Deep Learning: Making It Learnable

A course companion in native Python and PyTorch

Heman Shakeri

2026-07-08

Table of contents

Preface

> ⚠ Warning
>
> **Draft.** This book is being written in the open as the companion text for DS 6050 Deep Learning at UVA's School of Data Science. Chapters appear as they mature; everything here runs — every figure and result is produced by the code on the page.

Deep learning did not appear out of nowhere. Nearly every idea that powers modern AI is a classical idea — a line of best fit, a hand-designed image filter, a kernel-weighted average — that someone dared to ask one question about:

> *What if we made this* ***learnable****?*

That question is the spine of this book. We will ask it over and over:

- Take **linear regression**, let its features be learned, and you get the multilayer perceptron.
- Watch the MLP **fail to generalize** on images — see the failure in pictures — and the cure (respecting the structure of the data) becomes obvious: an **inductive bias**.
- Take the **fixed filters** of classical computer vision, make them learnable, and you get the convolutional network.
- Take **kernel regression** — a century-old way to average nearby evidence — make the similarity function learnable, and you get **attention**, then self-attention, then the Transformer.
- Then comes the **BERT moment**: stop training from scratch at all, and adapt what has already been learned.

Each part of the book replays this same move at a larger scale. By the end, the modern stack should feel less like a zoo of architectures and more like one idea, compounding.

How to read this book

Every chapter follows the course's learning loop: **read → run → check**. The code is native Python and PyTorch, written to run on an ordinary laptop CPU — small data, honest experiments. Watch the lecture videos, read the chapter, run the cells, and test yourself against the self-checks on the course site.

Acknowledgments

To the students of DS 6050, whose questions shaped every explanation here.

Part I.

I · From Lines to Networks

1. Linear Regression, the Mother Model

The core task of everything we will do in this course is to teach a computer to learn a function from examples. Before we can appreciate what is *deep* about deep learning, we need one complete, honest example of learning — a model, a loss, and a way to improve. Linear regression is that example. It is the smallest model that contains the entire supervised learning story, and by the end of this chapter we will have built it three times: once in closed form, once by gradient descent, and once the way you will build everything else in this book — with PyTorch modules. All three will agree, and you will know exactly why.

1.1. Learning from examples

We start with a dataset of examples,

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n},$$

where each $\mathbf{x}_i \in \mathbb{R}^d$ is an input with d features and y_i is the desired output — the *supervision signal*. Stack the inputs as rows and you get the data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$: n samples down, d features across, with the targets collected in a vector $\mathbf{y}$.

Our goal is a flexible function f such that

$$f(\mathbf{x}_i) \approx y_i \quad \text{and, crucially, } f \text{ generalizes to unseen data.}$$

That second clause is the whole game. We do not care how well f recites the training examples; we care how it behaves on a new $\mathbf{x}$ it has never seen — one drawn from the same distribution as the training data.

Supervised learning, three flavors

The range of y decides the task and, as we will see, the natural loss:

Task	Range of y	Typical loss
Regression	$\mathbb{R}$	Mean squared error
Classification	$\{0, \dots, C-1\}$	Cross-entropy
Structured prediction	sequences, images, graphs	task-specific

This chapter is regression; Chapter 2 handles classification with the same machinery.

1.1.1. Why is this hard?

This task sounds simple, but it is fundamentally hard: for any finite set of examples, there are *infinitely many* functions that fit them perfectly. We want the one that is most suited to our problem — so the learning algorithm needs a nudge in the right direction. That guiding assumption is called its **inductive bias**, and it is a thread we will pull on for the rest of the book.

- A **linear model** has a *strong* bias: it assumes the world is linear. Simple, stable — and systematically wrong when the data is not linear (high bias, low variance).
- A **deep neural network** has a *weak* bias. Its flexibility lets it learn complex patterns, but it can memorize the training data instead of the underlying rule (low bias, high variance). Some networks can memorize an entire training set — and then perform poorly the day you deploy them.

The key to modern deep learning is to use a flexible model and fight overfitting with data, regularization, and — the deepest idea of all — inductive biases matched to the task. That last idea gets its own chapter (Chapter 6).

To make any of this concrete, we parameterize a family of functions $f_{\mathbf{w}}$ by a set of tuning parameters $\mathbf{w}$ — think of each parameter as a knob. *Learning* is finding a good setting $\hat{\mathbf{w}}$ of the knobs; *inference* is using $f_{\hat{\mathbf{w}}}$ to predict on unseen data. That is the vocabulary we will use all course.

1.2. The linear model and its geometry

Linear regression assumes a linear relationship:

$$y = \mathbf{w}^\top \mathbf{x} + b.$$

Geometrically, $\mathbf{w}$ sets the orientation — the slope — of a hyperplane, and the bias b shifts it away from the origin. In two dimensions this is the familiar line through a scatter of points; in d dimensions it is a d-dimensional plane, but nothing about the algebra changes.

1.2.1. The augmentation trick

Carrying b separately clutters the algebra, so we will often fold it into the weights: append a column of ones to the data matrix, making $\mathbf{X} \in \mathbb{R}^{n\times(d+1)}$, and append b to the weight vector, making $\mathbf{w} \in \mathbb{R}^{d+1}$. Then

$$y = \mathbf{w}^\top \mathbf{x} \qquad \text{(bias included)}.$$

Whenever you see a linear model written without a bias — here or inside a neural network — the bias has not disappeared; it is living inside $\mathbf{w}$.

1.3. The loss: what makes a hyperplane "bad"?

To find the best hyperplane we must first quantify error. The standard choice for regression is the **mean squared error (MSE)**:

$$\mathcal{L}(\mathbf{w}) \;=\; \frac{1}{n}\sum_{i=1}^{n}(y_i - \mathbf{w}^\top \mathbf{x}_i)^2 \;=\; \frac{1}{n}\,\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2\,. \tag{1.1}$$

The loss is the guiding principle of training: it is how we tell the model *you are a bad model, and here is by how much* — and the model has to find a way to improve. The choice of loss is not arbitrary, and we will justify this one honestly at the end of the chapter.

> The first "make it learnable" seed
>
> Look closely at $y = \mathbf{w}^\top \mathbf{x} + b$. A single neuron in a neural network computes *exactly this*, followed by a nonlinear squashing function $\sigma(\mathbf{w}^\top \mathbf{x} + b)$. Linear regression **is** a neuron with the identity activation. Everything we learn here — loss, gradients, optimization — transfers directly. The rest of this book is, in a precise sense, the story of what happens when we take this fixed, simple recipe and progressively let more of it be *learned*.

1.4. Finding the best weights, method 1: solve it exactly

For this one special problem we can find the exact minimizer. Set the gradient of Equation 1.1 to zero and you get the **normal equations**:

$$\hat{\mathbf{w}}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}. \tag{1.2}$$

There is a picture behind this formula worth carrying for the rest of your career. Our predictions $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ can only ever live in the **column space** of $\mathbf{X}$ — the span of its feature columns. If the true $\mathbf{y}$ lies outside that space (it almost always does), the best we can do is **project** $\mathbf{y}$ onto it. The leftover error $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ is what our model cannot explain, and at the optimum it is *perpendicular* to the column space — no adjustment of the knobs can reduce it further.

Elegant — so why is this formula almost absent from deep learning? Two reasons. Solving it costs $O(d^3)$, which is hopeless when d runs to millions or billions. And it only exists because the model is linear; the moment we add a nonlinearity, there is no closed form to solve. Our datasets are large and our models are not linear — we have neither luxury. We need another way.

1.5. Finding the best weights, method 2: walk downhill

Here is the geometric intuition. Picture the loss as a landscape over the parameter space, and imagine standing on it *blindfolded*, trying to find the lowest valley. You cannot see the valley —

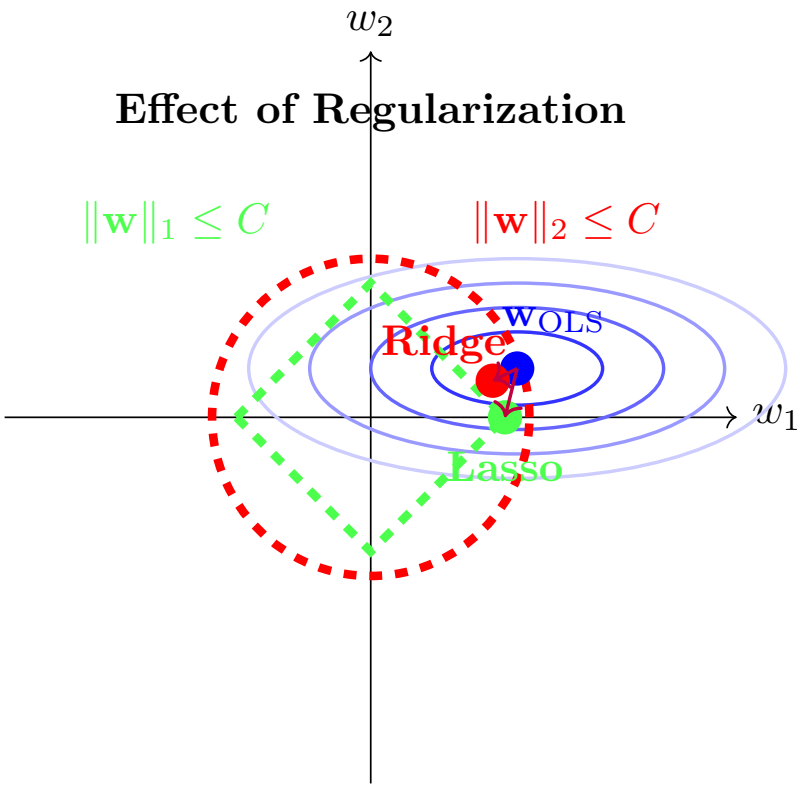

Figure 1.1.: The normal equations, geometrically: the optimal prediction $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}}$ is the projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$; the residual $\mathbf{e}$ is orthogonal to it.

but you can feel the slope under your feet. So you feel the slope, take a small step downhill, and repeat.

The mathematical form of this hill-descending intuition is **gradient descent**:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \, \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(t)}), \tag{1.3}$$

where the *learning rate* η sets the step size. For the MSE loss the gradient has a clean closed form — with $\mathbf{X} \in \mathbb{R}^{n \times d}$, $\mathbf{w} \in \mathbb{R}^d$, and $\mathbf{y} \in \mathbb{R}^n$:

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}) = \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) \ \in \ \mathbb{R}^d. \tag{1.4}$$

This iterative loop — predict, measure the loss, feel the gradient, step — is exactly what modern deep learning frameworks automate, at billion-parameter scale. When the dataset itself is huge we will not even use all of it per step: a small random *batch* gives a good-enough gradient. That refinement (stochastic gradient descent) matters enough to get its own treatment in Chapter 4.

1.6. Build it three times

Enough theory — let us implement linear regression, and let us do it in PyTorch. You have already built this in NumPy in earlier courses, so we will use it as an excuse to get comfortable with tensors, while still doing everything from scratch.

> **Coding hygiene**
>
> Two habits worth forming from the very first cell: **fix your seeds** so runs are reproducible, and **annotate your functions with types** — not critical for Python to run, but it makes code readable and debugging far easier.

```
import torch
import matplotlib.pyplot as plt

torch.manual_seed(6050)  # reproducible runs, always
```

```
<torch._C.Generator at 0x10fe8c3d0>
```

First, synthetic data. We control the ground truth — the true weights and bias — so we can check whether each method actually recovers them.

```python
def make_synthetic_data(
    weights: torch.Tensor,   # (d,) true weights
    bias: float,             # true bias
    n_samples: int,
    noise: float = 0.1,      # std of Gaussian noise
) -> tuple[torch.Tensor, torch.Tensor]:
    """y = X w + b + eps, with eps ~ N(0, noise^2)."""
    d = weights.shape[0]
    X = torch.randn(n_samples, d)              # (n, d) feature matrix
    y = X @ weights + bias                     # (n,)  clean targets
    y += noise * torch.randn(n_samples)        # add irreducible noise
    return X, y

true_w = torch.tensor([2.0, -3.4])
true_b = 4.2
X, y = make_synthetic_data(true_w, true_b, n_samples=200)
X.shape, y.shape   # (200, 2), (200,) - always check your shapes
```

```
(torch.Size([200, 2]), torch.Size([200]))
```

1.6.1. Take 1: the closed form

The normal equations, Equation 1.2, on the *augmented* matrix (the column of ones carries the bias):

```python
X_aug = torch.cat([X, torch.ones(X.shape[0], 1)], dim=1)   # (n, d+1)
w_ols = torch.linalg.solve(X_aug.T @ X_aug, X_aug.T @ y)   # solve, don't invert
print(f"closed form:      w = {w_ols[:2].numpy().round(3)},  b = {w_ols[2]:.3f}")
print(f"ground truth:     w = {true_w.numpy()},  b = {true_b}")
```

```
closed form:      w = [ 2.01  -3.402],  b = 4.196
ground truth:     w = [ 2.  -3.4],  b = 4.2
```

Two lines, and we recover the truth up to the noise floor. Remember what this cost us: $O(d^3)$, and the special grace of a linear model. Now the method that scales.

1.6.2. Take 2: gradient descent, from scratch

We write the model as a small class — a forward pass, a loss, manually derived gradients (Equation 1.4), and an update step. No autograd yet: we want to feel the mechanics once with our own hands.

```python
class LinearRegressionScratch:
    """Linear regression with manually derived gradients."""

    def __init__(self, input_size: int, lr: float = 0.1):
        self.w = 0.01 * torch.randn(input_size)   # small random start
        self.b = torch.zeros(1)
        self.lr = lr

    def forward(self, X: torch.Tensor) -> torch.Tensor:
        return X @ self.w + self.b                # (n,)

    @staticmethod
    def loss(y_hat: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        return ((y_hat - y) ** 2).mean()

    def step(self, X: torch.Tensor, y: torch.Tensor) -> float:
        y_hat = self.forward(X)                   # 1. predict
        loss = self.loss(y_hat, y)                # 2. measure
        err = y_hat - y                           # (n,) residuals
        grad_w = 2.0 * X.T @ err / len(y)         # 3. feel the slope  (d,)
        grad_b = 2.0 * err.mean()                 #    (bias grad = mean error)
        self.w -= self.lr * grad_w                # 4. step downhill
        self.b -= self.lr * grad_b
        return float(loss)

model = LinearRegressionScratch(input_size=2)
losses = [model.step(X, y) for _ in range(100)]
print(f"gradient descent: w = {model.w.numpy().round(3)},  b = {model.b.item():.3f}")
```

```
gradient descent: w = [ 2.01  -3.402],  b = 4.196
```

```python
plt.figure(figsize=(5.5, 3.2))
plt.plot(losses)
plt.xlabel("step")
plt.ylabel("MSE loss")
plt.yscale("log")
plt.tight_layout()
plt.show()
```

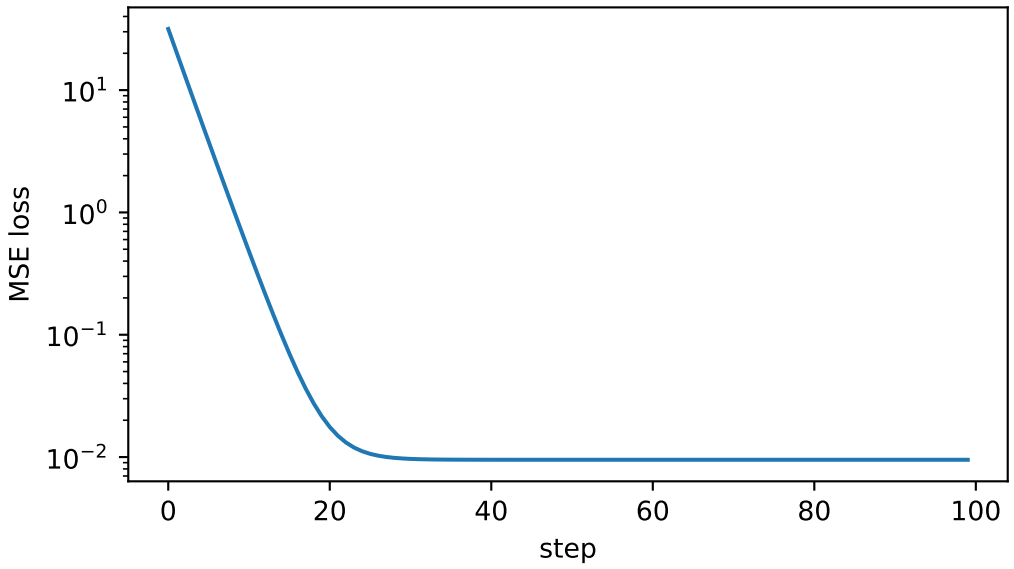

Figure 1.2.: The loss falls fast at first — steep slope underfoot — then flattens as we approach the valley floor set by the irreducible noise.

1.6.3. Take 3: the framework way

Finally, the idiom you will use for every model from here on: `nn.Linear` holds the knobs, autograd feels the slope for us, and the optimizer takes the step.

```
from torch import nn

net = nn.Linear(in_features=2, out_features=1)
optimizer = torch.optim.SGD(net.parameters(), lr=0.1)
loss_fn = nn.MSELoss()

for _ in range(100):
    optimizer.zero_grad()               # clear old gradients - they accumulate!
    y_hat = net(X).squeeze(-1)          # (n, 1) -> (n)
    loss = loss_fn(y_hat, y)
    loss.backward()                     # autograd computes the gradient for us
    optimizer.step()

w_nn = net.weight.detach().squeeze()
b_nn = net.bias.item()
print(f"nn.Linear:        w = {w_nn.numpy().round(3)},  b = {b_nn:.3f}")
```

```
nn.Linear:        w = [ 2.01  -3.402],  b = 4.196
```

Three implementations, one answer:

```
grid = torch.linspace(X[:, 0].min(), X[:, 0].max(), 50)
line = lambda w, b: w[0] * grid + w[1] * X[:, 1].mean() + b

plt.figure(figsize=(5.5, 3.6))
plt.scatter(X[:, 0], y, s=12, alpha=0.4, label="data")
plt.plot(grid, line(w_ols, w_ols[2]), lw=3, label="closed form")
plt.plot(grid, line(model.w, model.b.item()), "--", lw=2, label="gradient descent")
plt.plot(grid, line(w_nn, b_nn), ":", lw=2, label="nn.Linear")
plt.xlabel("$x_1$"); plt.ylabel("$y$"); plt.legend()
plt.tight_layout()
plt.show()
```

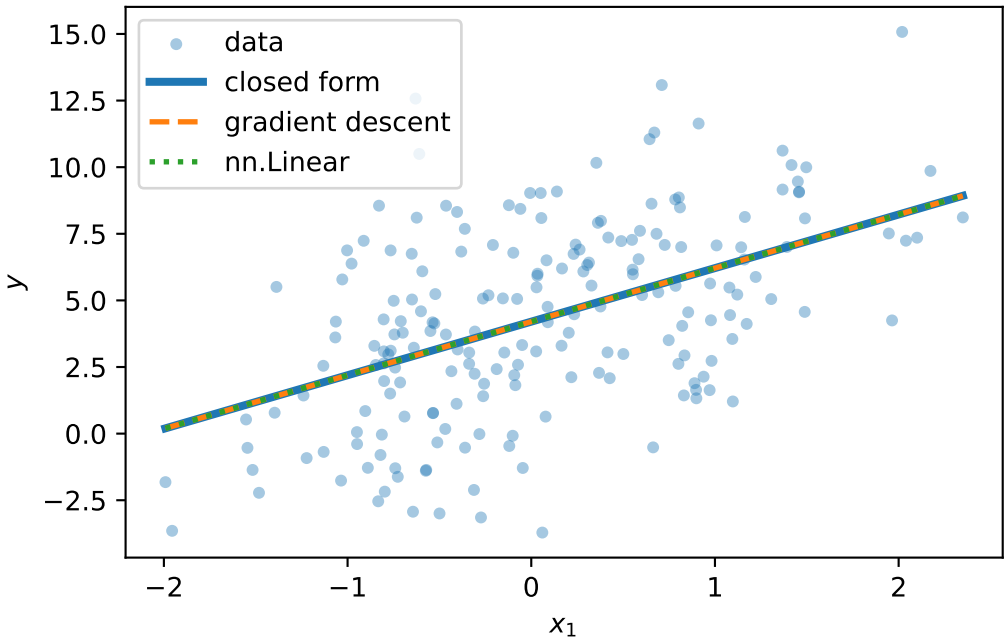

Figure 1.3.: All three methods find the same line (shown against feature x_1, second feature at its mean).

1.7. Why squared error? The maximum-likelihood view

The MSE is not an arbitrary choice. Assume the data really is generated by a linear process plus Gaussian noise:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

Then the probability of seeing output y given input $\mathbf{x}$ is a Gaussian centered on the model's prediction. **Maximum likelihood estimation** asks: which parameters make the observed data

most likely? Take the log of the likelihood over all n samples and the Gaussian's exponent comes down:

$$\log \mathcal{L}(\mathbf{w}, b) = C - \frac{1}{2\sigma^2} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^\top \mathbf{x}_i + b)\right)^2.$$

Maximizing the log-likelihood is *exactly* minimizing the sum of squared errors. The loss encodes an assumption about the noise. This is a pattern to remember: when we meet cross-entropy in Chapter 2, it will be the same story with a different distribution.

1.8. A first look at bias and variance

Our learned $\hat{\mathbf{w}}$ depends on the training data — and the training data is a random sample. Collect a fresh dataset and you get a different $\hat{\mathbf{w}}$. So the model's expected error on a new point decomposes into three parts:

$$\text{Err} = \underbrace{\left(\mathbb{E}[\hat{y}] - y\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[(\hat{y} - \mathbb{E}[\hat{y}])^2\right]}_{\text{Variance}} + \underbrace{\sigma_\varepsilon^2}_{\text{Irreducible noise}}. \tag{1.5}$$

Bias measures how far the *average* prediction is from the truth, because of limiting assumptions like linearity — a simple model on complex data is systematically wrong (underfitting). **Variance** measures how much the prediction would change if we collected a fresh training set — the model's sensitivity to sampling noise; a complex model can fit the noise itself (overfitting). The **irreducible noise** is inherent to the data-generating process and cannot be learned away.

This is why we split data into training, validation, and test: the model sees the training set, the validation set referees the bias–variance trade-off, and the test set stays untouched until the end.

> ⚠ The U-shaped curve is a cartoon, not a law of nature
>
> The textbook picture — validation error falling then rising as complexity grows — is a helpful first mental model, and you should have it. But bias and variance are not a mechanical see-saw. More data shrinks variance (at a rate of roughly $1/n$, while leaving bias untouched); regularization, and inductive biases matched to the task, can buy capacity without exploding variance. Modern over-parameterized networks live in regimes where this cartoon bends in surprising ways — we will look at those pictures properly in Chapter 6.

1.8.1. Regularization, briefly

One lever we can pull right now: penalize complexity in the loss itself. **Ridge regression** adds an L_2 penalty,

$$\mathcal{L}_\lambda(\mathbf{w}) = \frac{1}{n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2, \qquad \hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^\top\mathbf{X} + \lambda I)^{-1}\mathbf{X}^\top\mathbf{y},$$

nudging the model toward smaller, more stable weights — a little more bias for a lot less variance. Its cousin **lasso** uses the L_1 norm, $\lambda\|\mathbf{w}\|_1$, and does something qualitatively different: it drives the least informative weights to *exactly zero*, performing automatic feature selection.

Let us see that difference, not just assert it. We build a problem designed to punish an unregularized model: 20 features of which only 5 matter, noisy targets, and — the cruel part — barely more samples than parameters. This is exactly the high-dimensional regime where regularization shines:

```
from sklearn.linear_model import Lasso   # coordinate descent for L1 (no closed form)

n, d = 25, 20                            # barely more samples than knobs!
w_true = torch.zeros(d)
w_true[:5] = torch.tensor([3.0, -2.0, 1.5, 2.5, -1.0])   # only 5 real features
Xh, yh = make_synthetic_data(w_true, bias=0.0, n_samples=n, noise=2.0)

# OLS and ridge in closed form
w_ols_h = torch.linalg.solve(Xh.T @ Xh, Xh.T @ yh)
lam = 10.0
w_ridge = torch.linalg.solve(Xh.T @ Xh + lam * torch.eye(d), Xh.T @ yh)
w_lasso = torch.tensor(
    Lasso(alpha=0.4).fit(Xh.numpy(), yh.numpy()).coef_, dtype=torch.float32
)

def report(name: str, w_hat: torch.Tensor) -> None:
    err = ((w_hat - w_true) ** 2).mean()
    zeros = int((w_hat.abs() < 1e-3).sum())
    print(f"{name:>6}:  weight MSE = {err:.4f}   near-zero weights = {zeros}/20")

report("OLS", w_ols_h)
report("ridge", w_ridge)
report("lasso", w_lasso)
```

```
   OLS:  weight MSE = 1.2347   near-zero weights = 0/20
 ridge:  weight MSE = 0.4400   near-zero weights = 0/20
 lasso:  weight MSE = 0.3489   near-zero weights = 12/20
```

OLS, with all its freedom and almost no data to discipline it, assigns large, confident, *wrong* weights to the 15 noise features — pure variance. Ridge shrinks everything and cuts the damage several-fold, but keeps every feature alive. Lasso does the one thing the others cannot: it identifies the noise features and sets them *exactly* to zero — a sparse, more interpretable model that has effectively performed feature selection for us. (Rerun this cell with `n = 100` and watch OLS become respectable again: variance decays like $1/n$.)

1.9. Okay, so — what did we just build?

We taught a computer a function from examples, completely:

1. **A model family** — hyperplanes, parameterized by knobs $\mathbf{w}, b$ (with the augmentation trick to fold b into $\mathbf{w}$).
2. **A loss** — MSE, which is maximum likelihood under Gaussian noise; the loss is how we tell the model how bad it is.
3. **An optimizer** — the exact projection when linearity permits, and gradient descent (feel the slope, step downhill) when it does not — which is always, from here on.
4. **The generalization lens** — bias vs. variance, the data splits that referee them, and regularization as a first counter-measure to overfitting.

And one reframing that will carry the whole book: linear regression is a single neuron with the identity activation,

$$\underbrace{y = \mathbf{w}^\top \mathbf{x} + b}_{\text{this chapter}} \quad \xrightarrow{\text{add a nonlinearity}} \quad \underbrace{y = \sigma(\mathbf{w}^\top \mathbf{x} + b)}_{\text{a neuron}}.$$

First, in Chapter 2, that squashing function turns our regressor into a classifier. Then, in Chapter 3, we stack neurons — and discover why the nonlinearity is not optional.

Exercises

1. **(Pencil.)** Derive the normal equations Equation 1.2 by expanding $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$, taking the gradient with respect to $\mathbf{w}$, and setting it to zero. Where exactly does the assumption that $\mathbf{X}^\top \mathbf{X}$ is invertible enter?
2. **(Pencil.)** Repeat the derivation with the ridge penalty $\lambda \|\mathbf{w}\|_2^2$ and show the solution becomes $(\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1} \mathbf{X}^\top \mathbf{y}$. Why does the λI term guarantee invertibility for any $\lambda > 0$?
3. **(Code.)** In `make_synthetic_data`, raise `noise` to 1.0 and rerun all three implementations 10 times with different seeds. How much do the learned weights vary across runs? Which term of Equation 1.5 are you watching?
4. **(Code.)** Set the learning rate in `LinearRegressionScratch` to 1.0 and rerun. Describe what the loss curve does, and explain it using the blindfolded-descent picture.
5. **(Code.)** In the ridge/lasso experiment, sweep $\lambda \in \{0.01, 0.1, 1, 10, 100\}$ for ridge and plot weight MSE against λ. Where is the sweet spot, and what happens at the extremes? Connect your answer to bias and variance.

2. Linear Models that Classify: Logistic and Softmax

Regression predicts numbers; most of what the world wants predicted is *categories* — cat or dog, spam or not, which of ten digits. This chapter converts the linear machinery of Chapter 1 into a classifier without discarding a single idea: the same weighted sums, the same maximum-likelihood recipe for the loss, the same gradient descent. What changes is one squashing function at the output, and it changes less than you might think.

2.1. What actually changes

In regression, $y \in \mathbb{R}$ and the Gaussian noise model handed us the MSE (Chapter 1). In classification, y is a label from $\{0, \dots, K-1\}$. Could we just regress onto the labels with MSE anyway? We could, and it fails for a principled reason: **the loss is a noise model, and Gaussian is the wrong model for a coin flip.** A binary outcome deviates from its probability like a Bernoulli variable, not like additive Gaussian noise → maximum likelihood demands a different loss. Everything in this chapter falls out of taking that seriously.

What we need is two things:

1. a way to turn the model's raw score (its **logit**, $o \in \mathbb{R}$) into a probability,
2. a loss that measures how wrong a *probability* is.

2.2. Binary classification

2.2.1. From score to probability: the sigmoid

Our linear model produces $o = \mathbf{w}^\top \mathbf{x} + b$, an unbounded score. The **sigmoid** squashes it into a probability:

$$\sigma(o) = \frac{1}{1 + e^{-o}}. \tag{2.1}$$

```
import torch
import matplotlib.pyplot as plt

o = torch.linspace(-6, 6, 300)
plt.figure(figsize=(5.8, 3))
plt.plot(o, torch.sigmoid(o), color="#232D4B", lw=2.2)
plt.axhline(0.5, ls=":", color="#B8B8A8"); plt.axvline(0, ls=":", color="#B8B8A8")
plt.annotate("decision boundary\n$o = 0$", xy=(0, 0.5), xytext=(1.6, 0.32),
             color="#E57200", arrowprops=dict(arrowstyle="-|>", color="#E57200"))
plt.xlabel(r"logit $o = \mathbf{w}^\top \mathbf{x} + b$")
plt.ylabel(r"$\hat{p} = \sigma(o)$")
plt.tight_layout(); plt.show()
```

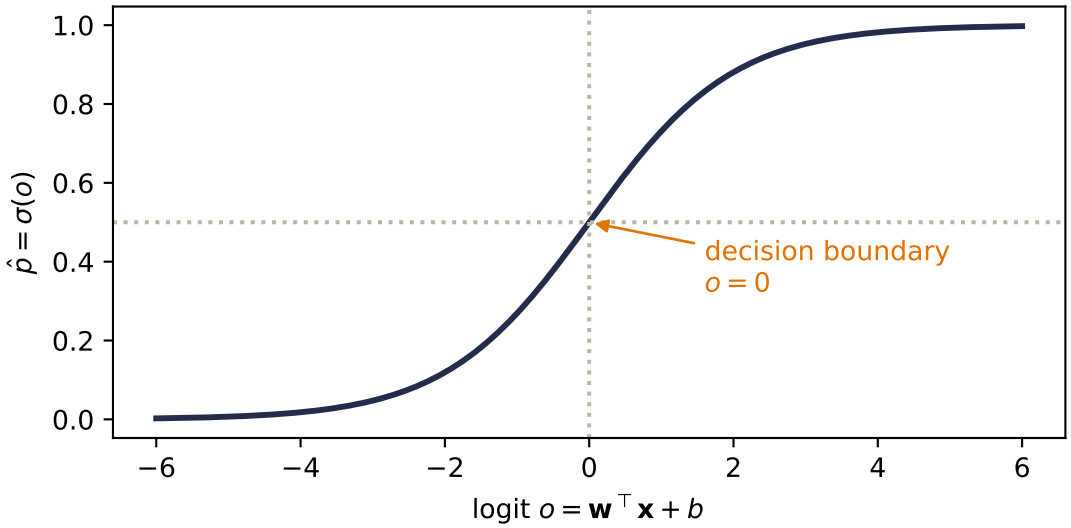

Figure 2.1.: The sigmoid maps any real score to $(0, 1)$, crossing $\frac{1}{2}$ exactly at $o = 0$. The model's uncertainty lives in the middle; its confident regions saturate at the tails.

Notice what did *not* change: the decision boundary $\hat{p} = \frac{1}{2}$ is exactly $o = 0$, i.e. $\mathbf{w}^\top \mathbf{x} + b = 0$ — the same hyperplane geometry from Chapter 1, with $\mathbf{w}$ still the template the input is scored against. The sigmoid does not bend the boundary; it grades our confidence on either side of it.

2.2.2. The loss: cross-entropy from maximum likelihood

Run the maximum-likelihood recipe with the correct noise model. If the label is a coin flip with $P(y{=}1 \mid \mathbf{x}) = \hat{p}$, the likelihood of the dataset is $\prod_i \hat{p}_i^{y_i}(1-\hat{p}_i)^{1-y_i}$; taking the negative log gives the **binary cross-entropy**:

$$\mathcal{L}_{\text{BCE}} = -\big[\, y \log \hat{p} + (1-y) \log(1-\hat{p}) \,\big]. \tag{2.2}$$

Read it as *surprise*: if the true label is 1, the loss is $-\log \hat{p}$ — small when the model believed in the outcome, exploding as the model's belief goes to zero. A confident wrong answer is punished without mercy, which is exactly the pressure a probability deserves.

2.2.3. The beautiful gradient

Chain the sigmoid into the cross-entropy and differentiate with respect to the logit, and something remarkable drops out (Exercise 1 walks you through it):

$$\frac{\partial \mathcal{L}}{\partial o} = \hat{p} - y. \tag{2.3}$$

The gradient is *literally the prediction error*, the same form MSE gave us for regression. All the logs and exponentials annihilate each other. This is not luck, and it is not the last time you will see it.

2.3. Multiclass classification: softmax

With K classes, run K templates at once: $\mathbf{W} \in \mathbb{R}^{K \times d}$ produces a score vector $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$, one logit per class. To turn K scores into a probability distribution, exponentiate (making everything positive) and normalize:

$$\hat{y}_j = \mathrm{softmax}(\mathbf{o})_j = \frac{e^{o_j}}{\sum_{k=1}^{K} e^{o_k}}. \tag{2.4}$$

Softmax has three properties worth committing to memory: every output is positive, the outputs sum to one, and — less obviously — it is **invariant to constant shifts**: adding the same c to every logit changes nothing, because $e^{o_j + c} = e^c e^{o_j}$ and the e^c cancels top and bottom. Only the *differences* between scores matter.

```
logits = torch.tensor([2.0, 0.5, -1.0, 1.0])
fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.9), sharey=True)
for ax, shift in [(axes[0], 0.0), (axes[1], 100.0)]:
    p = torch.softmax(logits + shift, dim=0)
    ax.bar(range(4), p, color="#5379AA", width=0.55)
    for j, v in enumerate(p):
        ax.text(j, v + 0.015, f"{v:.2f}", ha="center", fontsize=9)
    ax.set_xticks(range(4))
    ax.set_xticklabels([f"$o_{j}$ = {v:+.1f}" for j, v in enumerate(logits + shift)],
                       fontsize=8)
    ax.set_title("original logits" if shift == 0 else "all logits + 100")
axes[0].set_ylabel("probability")
plt.tight_layout(); plt.show()
```

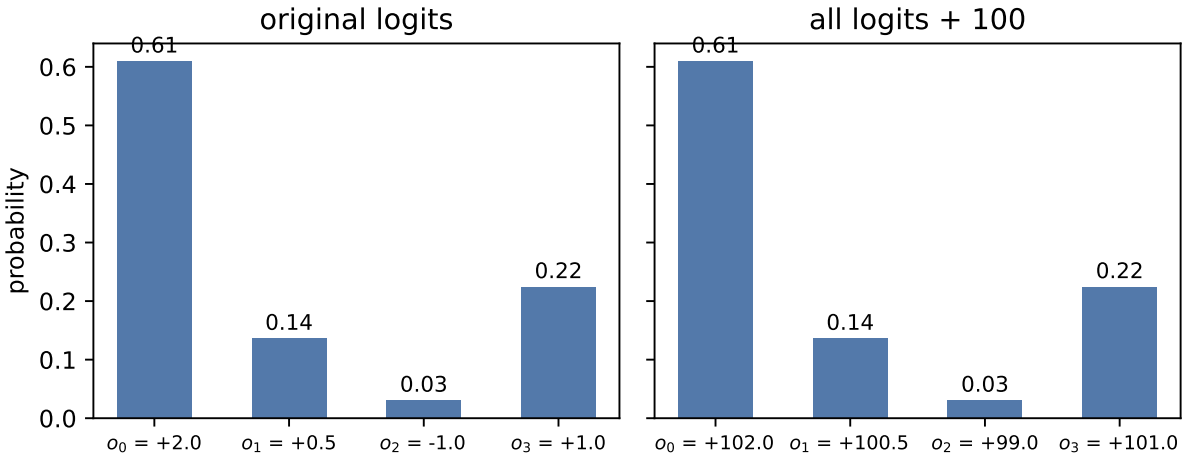

Figure 2.2.: Softmax turns scores into a distribution. Shifting all logits by a constant (right) changes the probabilities not at all — only score differences carry information.

Remember this machine: scores → weights

Step back from classification for a moment. What softmax really is: a differentiable device that takes any list of scores and returns **positive weights that sum to one**, favoring the large scores without silencing the small ones. Today the scores are class logits. In Part IV, the scores will be *similarities between a query and a set of keys*, and this exact same machine will turn them into **attention weights** (Chapter 13). One normalizer, the whole book long.

2.3.1. Cross-entropy, multiclass edition

With one-hot labels $\mathbf{y}$ (all zeros except a 1 at the true class), the cross-entropy is

$$\mathcal{L}_{\text{CE}} = -\sum_{j=1}^{K} y_j \log \hat{y}_j = -\log \hat{y}_{\text{true class}}, \tag{2.5}$$

the surprise at the true class, again. And the gradient repeats its trick:

$$\frac{\partial \mathcal{L}}{\partial o_j} = \hat{y}_j - y_j, \tag{2.6}$$

prediction minus target, one more time. This recurrence is no coincidence — sigmoid/BCE and softmax/CE are both members of the exponential family paired with their natural losses, and that pairing always collapses the gradient to the error.

> **The information-theory reading**
>
> Entropy $H[P] = -\sum_j p_j \log p_j$ measures the uncertainty in a distribution; cross-entropy $H(P,Q) = -\sum_j p_j \log q_j$ is the cost of encoding outcomes from P using codes built for Q. Minimizing our loss is minimizing that encoding cost — equivalently, maximizing likelihood, equivalently driving the KL divergence from truth to prediction toward zero. Three vocabularies, one objective.

2.3.2. One numerical landmine

Computing e^{o_j} overflows for logits as small as a few hundred. The fix exploits shift invariance: subtract the max logit before exponentiating (the *log-sum-exp trick*), which changes nothing mathematically and everything numerically. We will use it in the code below; the deeper story of floating-point limits lives in Appendix D.

2.4. Build it: three classes, from scratch

The full classifier, with the pieces named: K=3 templates, stable softmax, cross-entropy, and — because Equation 2.6 is so clean — the *manually derived* gradient, no autograd needed. On three Gaussian blobs:

```
torch.manual_seed(6050)
centers = torch.tensor([[-2.0, 0.0], [2.0, 0.0], [0.0, 2.5]])
X = torch.cat([c + 0.7 * torch.randn(150, 2) for c in centers])   # (450, 2)
y = torch.arange(3).repeat_interleave(150)                        # (450,)
Y = torch.nn.functional.one_hot(y, 3).float()                     # (450, 3)

def stable_softmax(logits: torch.Tensor) -> torch.Tensor:
    z = logits - logits.max(dim=-1, keepdim=True).values   # shift invariance at work
    e = torch.exp(z)
    return e / e.sum(dim=-1, keepdim=True)

W, b = torch.zeros(3, 2), torch.zeros(3)
for step in range(400):
    P = stable_softmax(X @ W.T + b)          # (450, 3) predicted probabilities
    G = (P - Y) / len(X)                     # the beautiful gradient, eq. (6)
    W -= 1.0 * G.T @ X                       # blame out x signal in
    b -= 1.0 * G.sum(0)

ce = -(Y * torch.log(P)).sum(1).mean()
acc = ((X @ W.T + b).argmax(1) == y).float().mean()
print(f"cross-entropy {ce:.3f}   accuracy {acc:.1%}")
```

```
cross-entropy 0.033   accuracy 98.9%
```

```
import numpy as np

gx, gy = np.meshgrid(np.linspace(-4.5, 4.5, 300), np.linspace(-2.5, 5, 300))
grid = torch.tensor(np.stack([gx.ravel(), gy.ravel()], 1), dtype=torch.float32)
Pg = stable_softmax(grid @ W.T + b)
cls = Pg.argmax(1).reshape(gx.shape).numpy()
conf = Pg.max(1).values.reshape(gx.shape).numpy()

plt.figure(figsize=(6, 4.4))
plt.contourf(gx, gy, cls, levels=[-0.5, 0.5, 1.5, 2.5],
             colors=["#DCE6F2", "#FDE3C8", "#E3EEE3"])
plt.contourf(gx, gy, 1 - conf, levels=12, cmap="Greys",
             alpha=0.25, antialiased=True)
for k, color in enumerate(["#232D4B", "#E57200", "#6B8E6B"]):
    plt.scatter(X[y == k, 0], X[y == k, 1], s=8, c=color, label=f"class {k}")
plt.xlabel("$x_1$"); plt.ylabel("$x_2$"); plt.legend(loc="lower right")
plt.tight_layout(); plt.show()
```

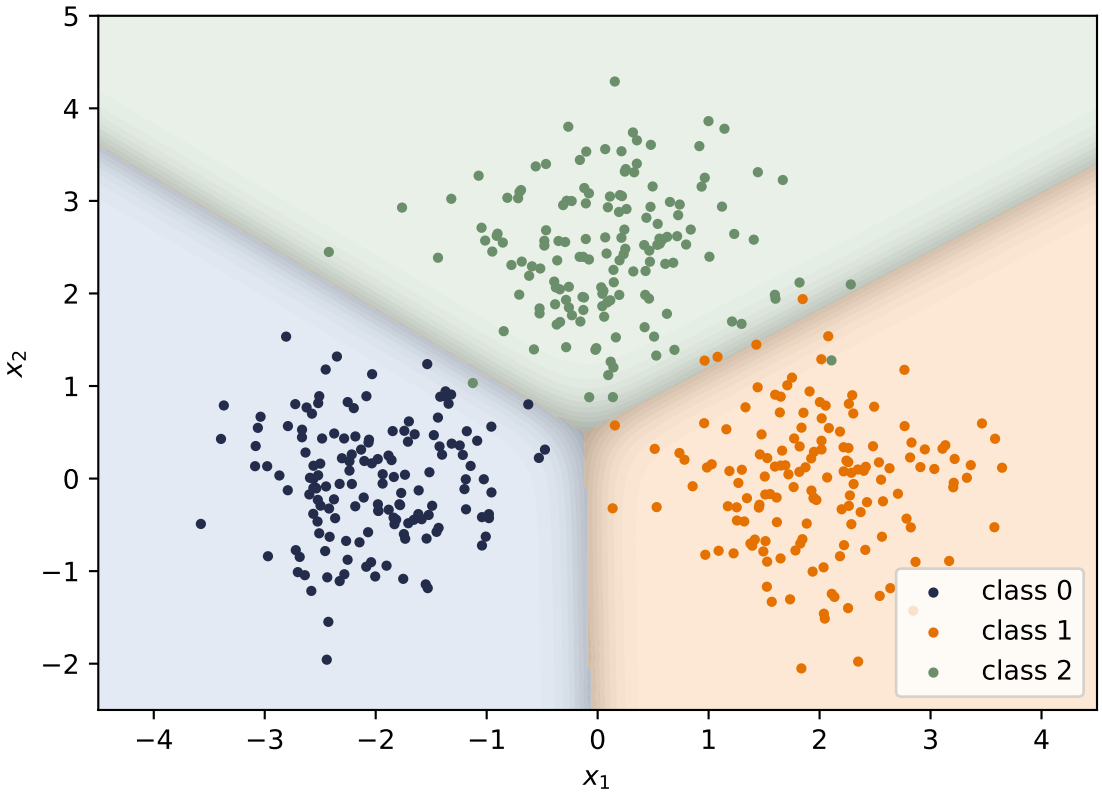

Figure 2.3.: Three templates, three linear boundaries. Color shows the predicted class; shading shows confidence (the max softmax probability) — crisp deep in each region, washed out along the boundaries where classes compete.

The framework version is two lines, with one trap worth a warning:

```
loss_fn = torch.nn.CrossEntropyLoss()
print(f"ours: {ce:.6f}   torch: {loss_fn(X @ W.T + b, y):.6f}")
```

```
ours: 0.033438   torch: 0.033425
```

> ⚠ The most common classification bug
>
> `nn.CrossEntropyLoss` takes **raw logits**, not softmax outputs — it applies (log-)softmax internally, with the stability trick built in. Feed it probabilities and your model still trains, just badly, with no error message. If a classifier is mysteriously mediocre, check this first.

2.5. Okay, so — classification is regression plus a normalizer

1. **The geometry survived**: logits are still template similarities $\mathbf{w}^\top \mathbf{x} + b$, boundaries are still hyperplanes. Sigmoid and softmax grade confidence; they do not bend anything.
2. **The loss came from the noise model**, exactly as in Chapter 1: Bernoulli → BCE, categorical → cross-entropy, both read as *surprise at the truth*.
3. **The gradient is the error**, $\hat{y} - y$, both times — the exponential-family pairing at work.
4. **Softmax is a scores-to-weights machine**, shift-invariant, numerically tamed by log-sum-exp — and it will return, unchanged, as the normalizer of attention.

Next, we let the templates themselves be built out of other templates: Chapter 3 stacks these linear units — and discovers why a bend between them is not optional.

Exercises

1. **(Pencil.)** Derive Equation 2.3: write $\mathcal{L}_{\text{BCE}}$ as a function of o through $\hat{p} = \sigma(o)$, use $\sigma'(o) = \sigma(o)(1-\sigma(o))$, and simplify. Which factors cancel, and why is the result independent of the sigmoid's saturation?
2. **(Pencil.)** Prove softmax's shift invariance from Equation 2.4, then explain why subtracting the max logit makes the computation overflow-proof: what is the largest value ever exponentiated?
3. **(Code.)** Temperature: replace `logits` with `logits / T` in the softmax figure for $T \in \{0.5, 1, 2, 10\}$. Describe what happens to the distribution at both extremes. (Keep this dial in mind — it returns when we scale attention scores in Part IV.)
4. **(Code.)** Train the blobs classifier with MSE on one-hot targets instead of cross-entropy (same manual loop; the gradient changes). Compare accuracy and the confidence shading near boundaries. What does the wrong noise model cost?
5. **(Code.)** Move the three blob centers closer together (`0.7 → 1.2` noise, or centers at distance 1) and retrain. Where does the confidence shading collapse, and what does the cross-entropy value tell you compared to the accuracy?

3. Nonlinearity and the MLP

Our linear models can regress (Chapter 1) and classify (Chapter 2), and it is tempting to think we could get more power by simply stacking them: feed the output of one linear layer into another. Watch what happens:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\,\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2) = \mathbf{W}_{\text{new}}\,\mathbf{x} + \mathbf{b}_{\text{new}}. \tag{3.1}$$

The stack *collapses*. Two linear layers, or twenty, are exactly one linear layer with different knobs → no amount of stacking buys any new expressive power. To learn the patterns of the real world we need a magic ingredient: **nonlinearity**. This chapter is about what one small bend does to everything.

3.1. The tiny dataset that embarrassed a field

You do not need real-world data to expose the linear ceiling. Four points suffice. The XOR function outputs 1 exactly when its two binary inputs *differ*:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Plot the four points and try to separate the 1s from the 0s with a single line. The positive examples sit on one diagonal, the negatives on the other; any line you draw puts at least one point on the wrong side. A linear classifier cannot represent this function, no matter how it is trained.

> **i** A historical scar
>
> This is not a toy observation. The XOR limitation of single-layer perceptrons, publicized in 1969, helped freeze neural-network research for years — part of the first "AI winter." The thaw came with multi-layer networks and backpropagation (Chapter 5), which is to say: with the contents of this chapter and the next two.

Let us confirm the ceiling empirically, with the tools we already have:

```
import torch
from torch import nn

torch.manual_seed(6050)
X_xor = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y_xor = torch.tensor([0., 1., 1., 0.])

def train(model, X, y, steps=4000, lr=0.5):
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    for _ in range(steps):
        opt.zero_grad()
        loss = nn.functional.binary_cross_entropy_with_logits(model(X).squeeze(-1), y)
        loss.backward()
        opt.step()
    return model

linear = train(nn.Linear(2, 1), X_xor, y_xor)
mlp = train(nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1)),
            X_xor, y_xor)

for name, model in [("linear", linear), ("MLP (4 hidden)", mlp)]:
    acc = ((model(X_xor).squeeze(-1) > 0).float() == y_xor).float().mean()
    print(f"{name:>15}: accuracy {acc:.0%}")
```

```
         linear: accuracy 25%
 MLP (4 hidden): accuracy 100%
```

The linear model lands at 25%: *worse than guessing*, because XOR is adversarial to lines. The little network with one bend in it is perfect. The rest of the chapter explains what that bend actually buys.

```
import numpy as np
import matplotlib.pyplot as plt

gx, gy = np.meshgrid(np.linspace(-0.6, 1.6, 220), np.linspace(-0.6, 1.6, 220))
grid = torch.tensor(np.stack([gx.ravel(), gy.ravel()], 1), dtype=torch.float32)

fig, axes = plt.subplots(1, 2, figsize=(7.4, 3.4), sharey=True)
for ax, model, title in [(axes[0], linear, "linear model"),
                         (axes[1], mlp, "MLP with ReLU")]:
    with torch.no_grad():
        Z = (model(grid).squeeze(-1) > 0).float().reshape(gx.shape)
    ax.contourf(gx, gy, Z, levels=[-0.5, 0.5, 1.5],
                colors=["#DCE6F2", "#FDE3C8"], alpha=0.9)
    for cls, marker, color in [(0, "o", "#232D4B"), (1, "s", "#E57200")]:
```

```
        pts = X_xor[y_xor == cls]
        ax.scatter(pts[:, 0], pts[:, 1], marker=marker, s=120, c=color,
                   edgecolors="white", linewidths=1.5, zorder=3,
                   label=f"class {cls}")
    ax.set_title(title); ax.set_xlabel("$x_1$")
axes[0].set_ylabel("$x_2$"); axes[0].legend(loc="center right")
plt.tight_layout(); plt.show()
```

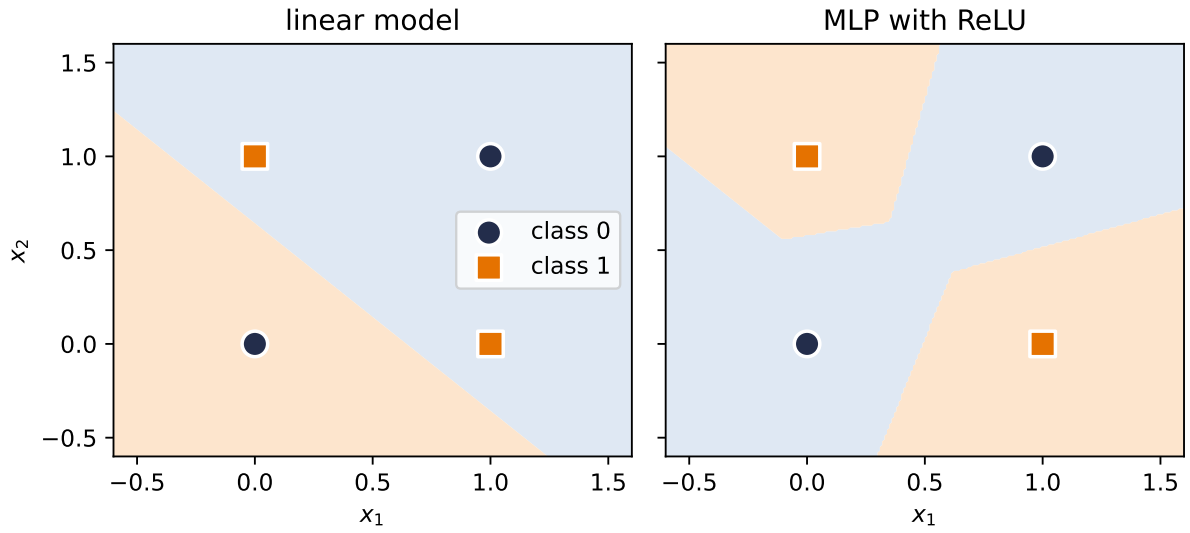

Figure 3.1.: Left: no line separates XOR — whatever the linear model learns, a diagonal pair is misclassified. Right: the trained 4-hidden-unit MLP carves the plane so that both diagonals get their own region.

3.2. The neuron: a threshold with a bend

The artificial neuron takes its inspiration from biology: a neuron receives signals, and it *fires* only when the accumulated stimulus crosses a threshold. The modern, computationally friendly version of that thresholding is the **rectified linear unit**:

$$\mathrm{ReLU}(z) = \max(0, z), \qquad f(\mathbf{x}) = \mathrm{ReLU}(\mathbf{w}^\top \mathbf{x} + b). \tag{3.2}$$

The weighted input $\mathbf{w}^\top \mathbf{x} + b$ is the stimulus — and recall from Chapter 1 that it is a *similarity score* against the template $\mathbf{w}$. Below threshold, the neuron is silent (output 0); above it, the neuron fires with intensity proportional to the stimulus.

In one dimension a single neuron turns a line into a **hinge**, and in two dimensions it draws a line through the plane: an OFF region and an ON region, with $\mathbf{w}$ perpendicular to the boundary. That boundary is the fundamental unit of computation in everything that follows.

(What about the sharp corner at $z = 0$, where the derivative is undefined? In practice, never a problem: libraries assign it 0, and the chance of a floating-point number landing exactly on 0 is essentially nil. The gradient story of this choice — why this open-or-shut gate is *the* reason deep networks train — is told in Chapter 5.)

3.3. Layers of hinges: bumps and polytopes

One neuron makes one boundary. A layer of k neurons makes k boundaries, and their intersections partition the input space into cells — each cell a **convex polytope**, and within each cell the layer computes a plain linear function. With d-dimensional inputs, k neurons can create on the order of k^d regions.

The simplest constructive example lives in 1D. Take two hinges and subtract:

```
xs = torch.linspace(-3, 5, 400)
h1 = torch.relu(0.67 * xs + 1)
h2 = torch.relu(0.8 * xs - 0.4)
bump = h1 - 1.25 * h2

fig, axes = plt.subplots(1, 2, figsize=(7.4, 3))
axes[0].plot(xs, h1, color="#5379AA", label=r"$h_1 = \mathrm{ReLU}(0.67x + 1)$")
axes[0].plot(xs, h2, color="#6B8E6B", label=r"$h_2 = \mathrm{ReLU}(0.8x - 0.4)$")
axes[0].set_title("two hidden units"); axes[0].legend(fontsize=8)
axes[1].plot(xs, bump, color="#E57200", lw=2.5, label=r"$y = h_1 - 1.25\,h_2$")
axes[1].set_title("their combination: a bump"); axes[1].legend(fontsize=9)
for ax in axes: ax.set_xlabel("$x$")
plt.tight_layout(); plt.show()
```

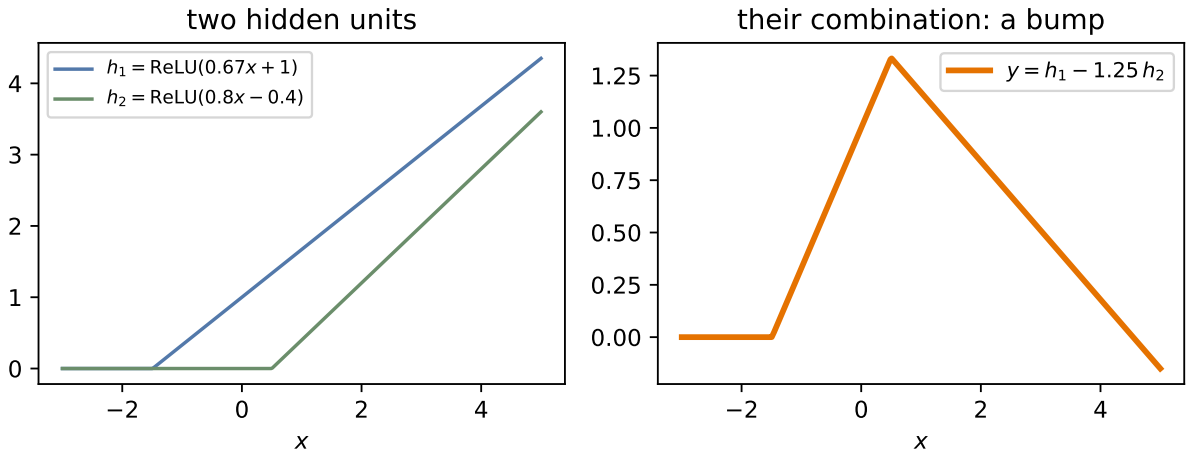

Figure 3.2.: The lecture's construction, exactly: two hinges (h_1, h_2) and one subtraction make a bump. Bumps are the brush strokes of function approximation.

Follow the pieces: before the first hinge, both units are silent and the output is zero. Between the hinges, only h_1 is on, so the output climbs. After the second hinge, h_2's negative contribution takes over and the output descends back to zero. Three linear pieces → one bump.

3.4. The universal approximation theorem

Bumps are the whole secret. If you can make one bump, you can make many, at any position and height → with enough hidden units you can *tile* any continuous function, the way an artist paints a shape with brush strokes. That is the intuition behind the **universal approximation theorem**: a network with a single hidden layer can, in principle, approximate any continuous function on a bounded region to any desired accuracy.

Watch the theorem materialize as width grows:

```
xs1 = torch.linspace(-3, 3, 200)[:, None]
target = torch.sin(2 * xs1) + 0.5 * torch.sign(xs1)

def fit(width: int, steps: int = 3000, lr: float = 0.05):
    torch.manual_seed(6050)
    net = nn.Sequential(nn.Linear(1, width), nn.ReLU(), nn.Linear(width, 1))
    opt = torch.optim.SGD(net.parameters(), lr=lr)
    for _ in range(steps):
        opt.zero_grad()
        ((net(xs1) - target) ** 2).mean().backward()
        opt.step()
    with torch.no_grad():
        return net(xs1).squeeze(-1)

plt.figure(figsize=(6.6, 3.6))
plt.plot(xs1, target, color="#232D4B", lw=2.5, label="target")
for width, color in [(2, "#B8B8A8"), (8, "#5379AA"), (64, "#E57200")]:
    plt.plot(xs1, fit(width), color=color, lw=1.6, label=f"width {width}")
plt.xlabel("$x$"); plt.legend(); plt.tight_layout(); plt.show()
```

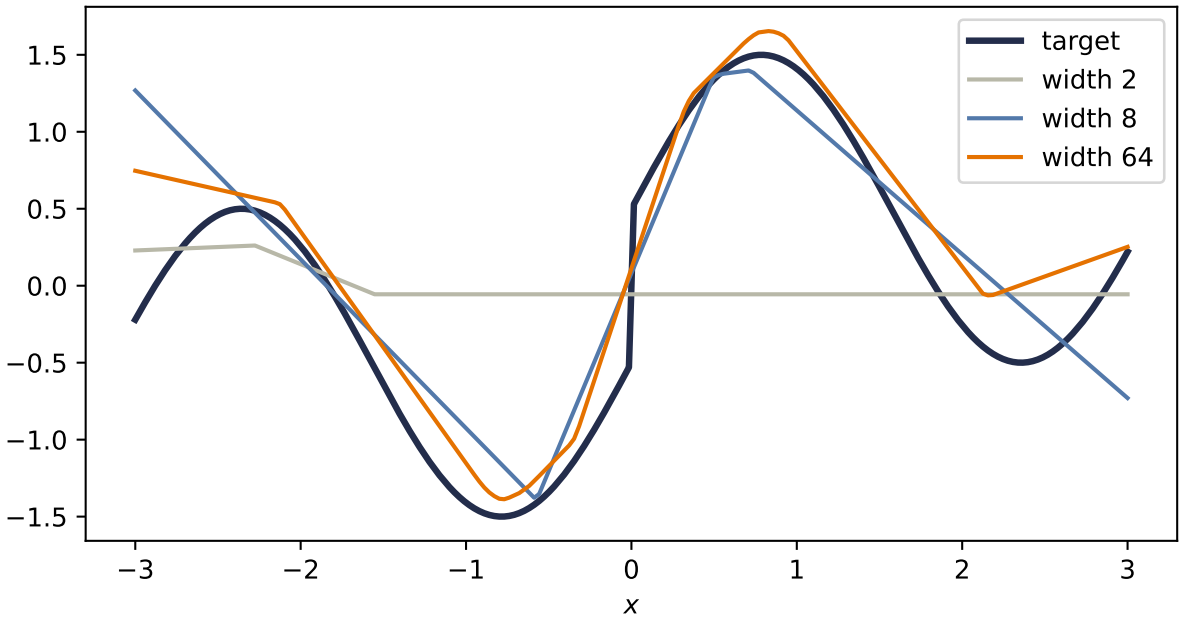

Figure 3.3.: One hidden layer approximating a wiggly target with a jump. Width 2 manages a single hinge-pair; width 8 catches the shape; width 64 traces it closely — except at the discontinuity, which no finite (or infinite) width fully conquers: the theorem promises continuous functions only.

Now read the theorem's fine print, because it is where beginners over-promise:

⚠ What the theorem does *not* say

1. It does not tell you **how to find the weights** — existence of a good network is not a training algorithm. Finding the weights is the job of the loss and the optimizer, and nothing guarantees gradient descent lands on the theorem's solution.
2. It does not say the network is **small**. Tiling a complicated function with one-layer bumps can require astronomically many units.
3. It speaks of **continuous functions on bounded regions** — note our jump refusing to be captured — and says nothing about how the fit behaves *between* or *beyond* your samples. Approximation is not generalization; that distinction gets its own chapter (Chapter 6).

3.5. Why depth: boundaries that bend

If one hidden layer suffices in principle, why is the field called *deep* learning? Efficiency, and geometry. A second hidden layer receives, as its inputs, the piecewise-linear outputs of the first → its boundaries are no longer straight lines. They *bend* wherever the first layer's boundaries cut the space. Boundaries composed of boundaries: each layer folds the space the previous layer drew.

```
torch.manual_seed(6050)
net2 = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 8), nn.ReLU())

gx2, gy2 = np.meshgrid(np.linspace(-2, 2, 500), np.linspace(-2, 2, 500))
G = torch.tensor(np.stack([gx2.ravel(), gy2.ravel()], 1), dtype=torch.float32)
with torch.no_grad():
    h1_ = torch.relu(net2[0](G))
    h2_ = torch.relu(net2[2](h1_))
    pattern = torch.cat([(h1_ > 0), (h2_ > 0)], 1).float()
    code = (pattern @ (2 ** torch.arange(16).float())).reshape(gx2.shape)

plt.figure(figsize=(5.4, 4.6))
plt.imshow(code, extent=[-2, 2, -2, 2], origin="lower", cmap="tab20", alpha=0.85)
plt.xlabel("$x_1$"); plt.ylabel("$x_2$")
plt.tight_layout(); plt.show()
```

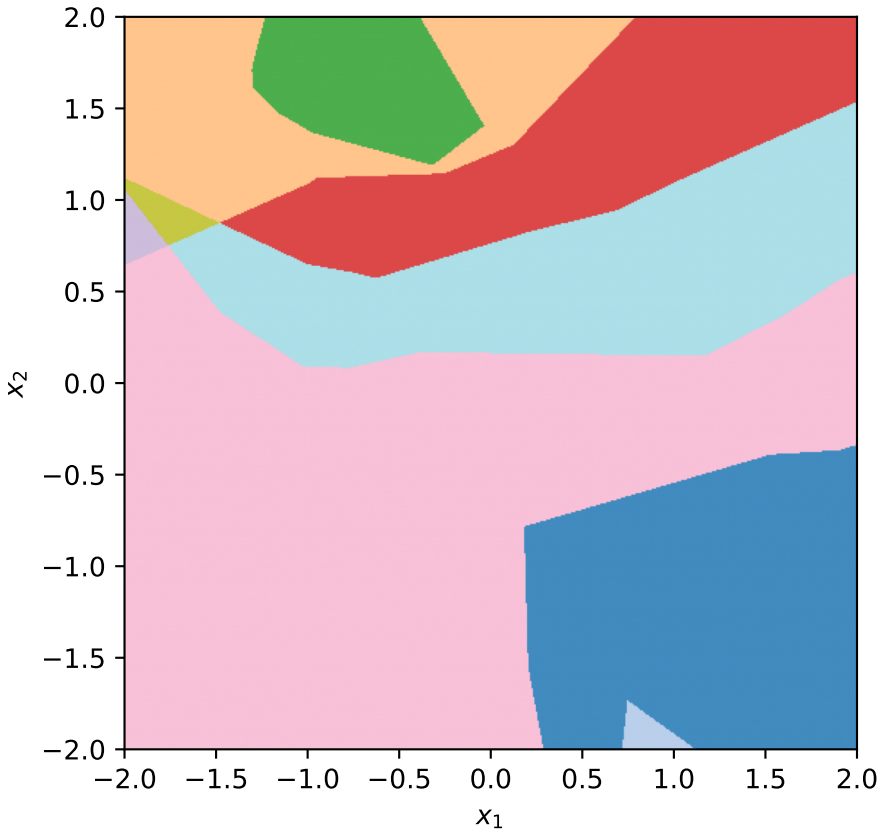

Figure 3.4.: The plane, colored by which ReLUs are on/off in a random 2-layer network ($2 \to 8 \to 8$): every cell is a region where the network is exactly linear. First-layer boundaries are straight; second-layer boundaries visibly bend where they cross them.

This bending compounds: theory shows the number of linear regions grows **exponentially with**

depth but only polynomially with width. A deep, narrow network can carve a partition that a shallow network would need exponentially many units to match. That is the economic argument for depth — the same expressive budget, spent far more efficiently.

3.6. Your first real MLP

Time to build the thing properly, the way the live session does: as a class.

> Coding hygiene: the house and its foundation
>
> Every model you write from now on subclasses `nn.Module`, and the first line of your `__init__` is always `super().__init__()`. Think of it as pouring the foundation before building the house: it is what wires up parameter tracking, `.parameters()`, and everything the optimizer and autograd need. Forget it and PyTorch fails loudly — the one favor a framework can do you.

```
class MLP(nn.Module):
    """A multilayer perceptron: linear layers with bends between them."""

    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int):
        super().__init__()                      # the foundation
        self.hidden = nn.Linear(input_dim, hidden_dim)
        self.out = nn.Linear(hidden_dim, output_dim)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.out(torch.relu(self.hidden(x)))
```

And a problem worthy of it: two interleaved moons, the classic shape no line can separate well. Same training loop as always — the model is the only thing that changed:

```
torch.manual_seed(6050)
n = 200
t = torch.rand(n) * torch.pi
moon1 = torch.stack([torch.cos(t), torch.sin(t)], 1) + 0.12 * torch.randn(n, 2)
moon2 = torch.stack([1 - torch.cos(t), 0.4 - torch.sin(t)], 1) + 0.12 * torch.randn(n, 2)
X_m = torch.cat([moon1, moon2])
y_m = torch.cat([torch.zeros(n), torch.ones(n)])

torch.manual_seed(6050)
lin_m = train(nn.Linear(2, 1), X_m, y_m, steps=3000, lr=0.8)
torch.manual_seed(6050)
mlp_m = train(MLP(2, 16, 1), X_m, y_m, steps=3000, lr=0.8)

gx3, gy3 = np.meshgrid(np.linspace(-1.6, 2.6, 240), np.linspace(-1.1, 1.6, 240))
```

```
grid3 = torch.tensor(np.stack([gx3.ravel(), gy3.ravel()], 1), dtype=torch.float32)

fig, axes = plt.subplots(1, 2, figsize=(7.6, 3.3), sharey=True)
for ax, model, title in [(axes[0], lin_m, "linear"), (axes[1], mlp_m, "MLP (16 hidden)")]:
    with torch.no_grad():
        acc = ((model(X_m).squeeze(-1) > 0).float() == y_m).float().mean()
        Z = (model(grid3).squeeze(-1) > 0).float().reshape(gx3.shape)
    ax.contourf(gx3, gy3, Z, levels=[-0.5, 0.5, 1.5],
                colors=["#DCE6F2", "#FDE3C8"], alpha=0.9)
    ax.scatter(X_m[y_m == 0, 0], X_m[y_m == 0, 1], s=8, c="#232D4B")
    ax.scatter(X_m[y_m == 1, 0], X_m[y_m == 1, 1], s=8, c="#E57200")
    ax.set_title(f"{title}: {acc:.1%}"); ax.set_xlabel("$x_1$")
axes[0].set_ylabel("$x_2$")
plt.tight_layout(); plt.show()
```

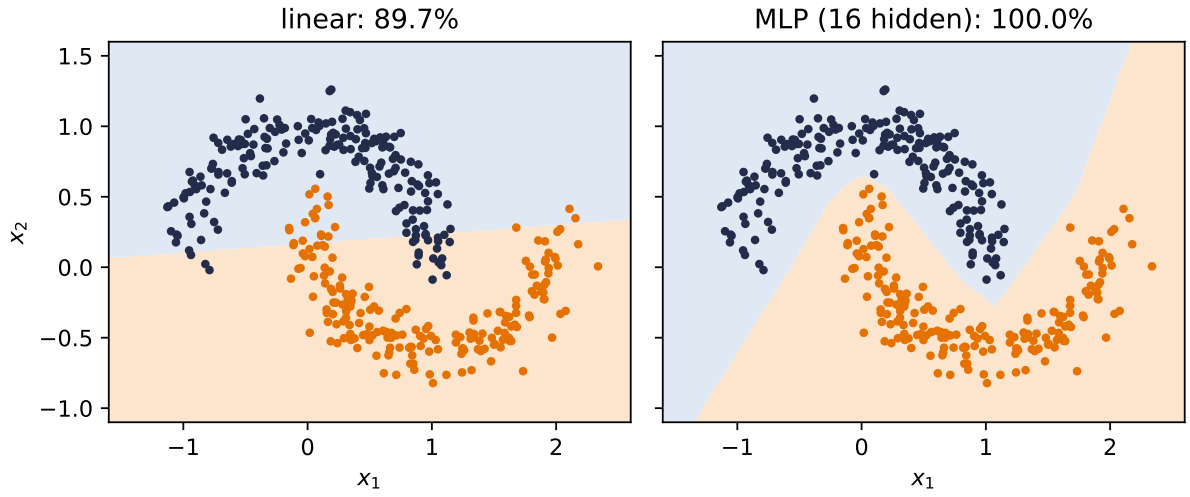

Figure 3.5.: Two moons: the linear model does what it can with one line (about 90% here); the MLP bends its boundary through the gap and separates the classes completely.

One honest footnote on the XOR network: in principle *two* hidden units suffice, but such minimal networks are temperamental — across random seeds they frequently get stuck at 50% accuracy in poor local minima (you will verify this in Exercise 3, and the noisy escape story of Chapter 4 is part of the cure). A few spare units cost little and buy reliability.

3.7. A cautionary tale, and a look ahead

There is a seductive story about what the layers of an MLP learn: the first layer detects edges, the next combines them into shapes, the next into objects — a tidy hierarchy of increasingly abstract features. It is the *idealized* story, and for fully-connected networks on raw images it is mostly **not what happens**. A fully-connected layer sees its input as one long, structureless

vector; it has no notion that pixel potentially relates to its neighbor. Nothing in the architecture encourages the tidy hierarchy, and with every pixel connected to every unit, the parameter count explodes long before the hierarchy emerges.

We now hold real expressive power — networks that can, in principle, represent almost anything. Chapter 4 trains them at scale and Chapter 5 supplies their gradients. And then Chapter 6 asks this chapter's uncomfortable question: what happens when all that flexibility meets data it does not understand the *structure* of? The answer — watch an MLP fail, in pictures, then fix it by building knowledge into the architecture — is the turn the whole book pivots on.

3.8. Okay, so — one bend changed everything

1. **Linear stacks collapse** (Equation 3.1); XOR proves four points can defeat any line.
2. **A neuron is a thresholded similarity score**: below the boundary silent, above it proportional. One neuron → one boundary; a layer → a polytope partition; two hinges → a bump.
3. **Bumps tile functions** (universal approximation) — but the theorem promises existence, not learnability, economy, or generalization.
4. **Depth bends boundaries**: regions grow exponentially with depth, polynomially with width. That is why the field is *deep* learning.
5. **The MLP is a class now** (`nn.Module`, foundation first), and it separates what no line can.

Exercises

1. **(Pencil.)** Generalize Equation 3.1: show by induction that any stack of L linear layers equals a single linear layer. Where exactly does inserting ReLU between layers break your proof?
2. **(Pencil.)** Using the bump construction, give explicit weights for a one-hidden-layer network that outputs approximately 1 on $[0, 1]$ and 0 outside $[-0.2, 1.2]$. How many hidden units did you need?
3. **(Code.)** Retrain the XOR network with `hidden_dim = 2` across ten different seeds. How often does it reach 100%? Inspect a failed run: what happened to its hidden units? (Hint: check how many are permanently silent.)
4. **(Code.)** In the polytope figure, count distinct activation patterns as you vary width at depth 1 versus depth at width 4 (keep total units comparable). Which grows the region count faster?
5. **(Code.)** Replace the moons MLP's ReLU with `nn.Identity()` and retrain. Explain the resulting boundary in one sentence using Equation 3.1.

4. Training: Loss and Stochastic Gradient Descent

We now have models worth training: linear regressors (Chapter 1), classifiers (Chapter 2), and multilayer perceptrons (Chapter 3). We have losses that tell each of them how bad they are, and we know the direction of improvement is the negative gradient. What remains is the question every practitioner faces daily: *how, exactly, do you run the descent* when the dataset has a million examples, the model has millions of knobs, and the loss landscape is no longer a friendly bowl?

This chapter is about the workhorse answer, stochastic gradient descent, and the small set of refinements (learning rates, batch sizes, momentum, Adam) that turn it from a theoretical idea into the algorithm that trains essentially everything in this book.

4.1. Where losses come from, one more time

Before optimizing a loss, remember why it is *that* loss. In Chapter 1 we saw that assuming Gaussian noise on a linear process makes maximum likelihood collapse into least squares; in Chapter 2 the same argument with a categorical distribution produced cross-entropy. This is the general pattern: **a loss function is a noise model in disguise.** Choose how you believe the data deviates from your model, and maximum likelihood hands you the loss. The loss is how we tell the model it is doing badly → choosing it well is not a detail, it is the supervision itself.

With the loss fixed, training is one line:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \; \mathcal{L}(\mathbf{w}) = \arg\min_{\mathbf{w}} \; \frac{1}{n} \sum_{i=1}^{n} \ell_i(\mathbf{w}), \tag{4.1}$$

where ℓ_i is the loss on example i. Everything that follows is about minimizing this average when n is enormous.

4.2. The computational bottleneck

Gradient descent, as we left it in Chapter 1, updates with the *full* gradient:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\mathbf{w}^{(t)}). \tag{4.2}$$

Look at the cost. One step touches every example; modern datasets have millions to billions of them, modern models have millions to billions of parameters, and training needs thousands of steps. One exact gradient per step is a luxury we cannot afford → we need to change strategy to scale up.

4.3. Stochastic gradient descent

The idea is almost cheeky: we do not need the exact gradient. A good approximation is enough. Instead of the expensive sum over all n examples, average over a small random **mini-batch** $\mathcal{B}$ — think 32 examples against a million:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla \ell_i(\mathbf{w}^{(t)}). \tag{4.3}$$

Why does this work? Because the mini-batch gradient is **unbiased**: if each example is sampled uniformly, the expected contribution of example i to a batch equals its contribution to the full average, so

$$\mathbb{E}\big[\nabla \mathcal{L}_{\mathcal{B}}(\mathbf{w})\big] = \nabla \mathcal{L}(\mathbf{w}). \tag{4.4}$$

In expectation, the cheap step points exactly where the expensive step would have pointed. Each step is noisy; on average, the walk is right.

In practice the algorithm is:

1. Shuffle the training data.
2. Split it into mini-batches of size B.
3. For each batch: compute the average gradient, update the parameters.
4. One pass through all batches is an **epoch**; repeat for multiple epochs.

> **i** Honest fine print
>
> The unbiasedness proof assumes sampling *with* replacement. The algorithm above samples *without* replacement (shuffle, then walk the batches), for practical reasons: we want to visit every example and spread the contributions evenly. That mismatch breaks the simple proof, and making the theory rigorous for shuffled SGD is an active area of research. Keep the caveat in mind; keep using the shuffle.

4.4. The two zones of SGD

The noise gives SGD a characteristic two-phase behavior, and it is worth seeing rather than just hearing about. Far from the optimum, almost any batch tells you roughly the same thing → rapid, consistent progress. Close to the optimum, different batches start to disagree ("go left" — "no, go right"), and the iterate bounces around in what we will call the **region of confusion**:

```
import torch
import numpy as np
import matplotlib.pyplot as plt

torch.manual_seed(6050)
x1 = torch.randn(80)
y1 = 2.5 * x1 - 1.0 + 0.4 * torch.randn(80)

def descend(batch: int | None, lr: float = 0.12, steps: int = 60):
    w, b, path = torch.tensor(-0.5), torch.tensor(2.0), []
    for _ in range(steps):
        idx = torch.randperm(80)[:batch] if batch else slice(None)
        err = (w * x1[idx] + b) - y1[idx]
        path.append((float(w), float(b)))
        w = w - lr * 2 * (err * x1[idx]).mean()
        b = b - lr * 2 * err.mean()
    return np.array(path)

W, B = np.meshgrid(np.linspace(-1.5, 5.5, 90), np.linspace(-4, 3, 90))
L = ((y1.numpy()[None, None, :] -
      (W[..., None] * x1.numpy() + B[..., None])) ** 2).mean(-1)

plt.figure(figsize=(6.2, 4))
plt.contour(W, B, L, levels=25, cmap="Blues", alpha=0.7)
for batch, color, label in [(None, "#232D4B", "full batch"),
                            (4, "#E57200", "SGD, B = 4")]:
    p = descend(batch)
    plt.plot(p[:, 0], p[:, 1], "o-", ms=2.5, lw=1.2, color=color, label=label)
plt.plot(2.5, -1.0, "k*", ms=13, label="truth")
plt.xlabel("$w$"); plt.ylabel("$b$"); plt.legend(loc="upper right")
plt.tight_layout(); plt.show()
```

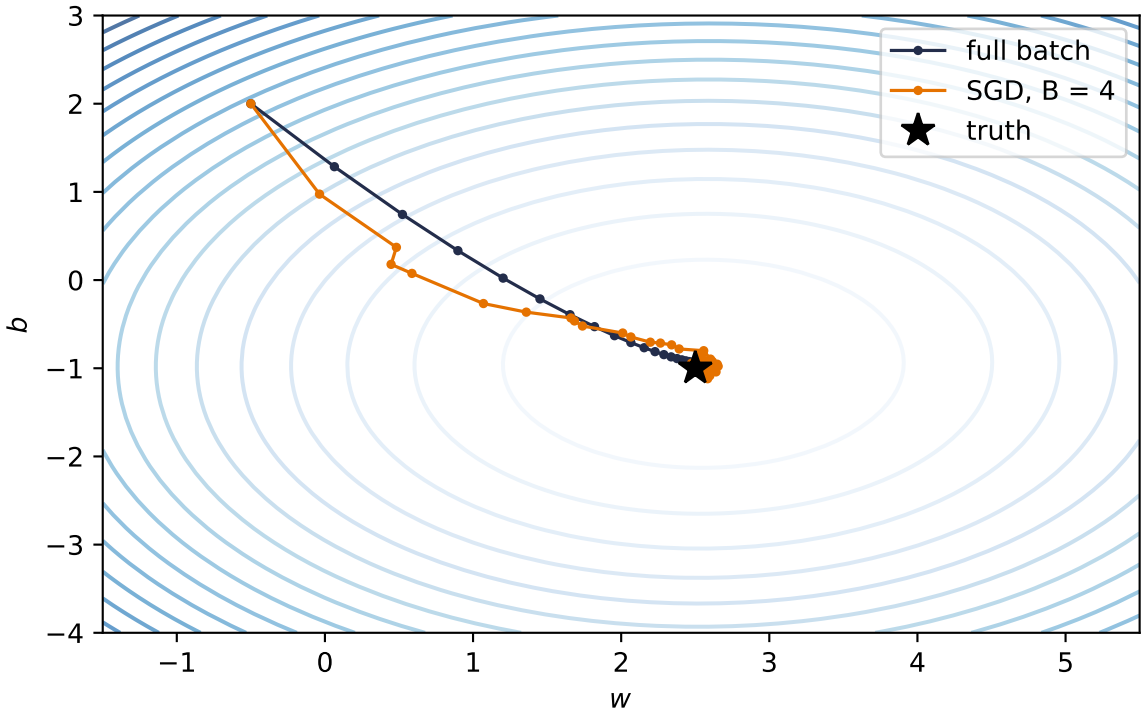

Figure 4.1.: Full-batch descent (blue) against mini-batch SGD (orange, $B = 4$) on the same landscape. Far out, the noisy steps agree with the true direction; near the optimum they scatter — the region of confusion.

Here is the surprise: the bouncing is not merely tolerable, it is often a **feature**. Loss landscapes of real networks are full of shallow traps, and a noisy step can hop out of a poor local minimum that would permanently capture exact gradient descent. The same jitter discourages the model from settling too perfectly into any one configuration of the training set; solutions found under noise tend to *generalize better*. We will give that observation its proper treatment in Chapter 6; for now, respect the noise. It is doing work for you.

4.5. The learning rate

One knob dominates all others. The learning rate α scales every step, and its failure modes are asymmetric: too small wastes your compute budget crawling; too large overshoots the valley and diverges.

```
def losses_for(lr: float, steps: int = 60):
    w, b, out = torch.tensor(-0.5), torch.tensor(2.0), []
    for _ in range(steps):
        err = (w * x1 + b) - y1
        out.append(float((err ** 2).mean()))
        w = w - lr * 2 * (err * x1).mean()
```

```
        b = b - lr * 2 * err.mean()
    return out

plt.figure(figsize=(6.2, 3.4))
for lr, style, color in [(0.005, "-", "#5379AA"), (0.12, "-", "#E57200"),
                         (1.1, "--", "#722F37")]:
    plt.semilogy(losses_for(lr), style, color=color, label=f"$\\alpha = {lr}$")
plt.xlabel("step"); plt.ylabel("loss (log scale)")
plt.legend(); plt.tight_layout(); plt.show()
```

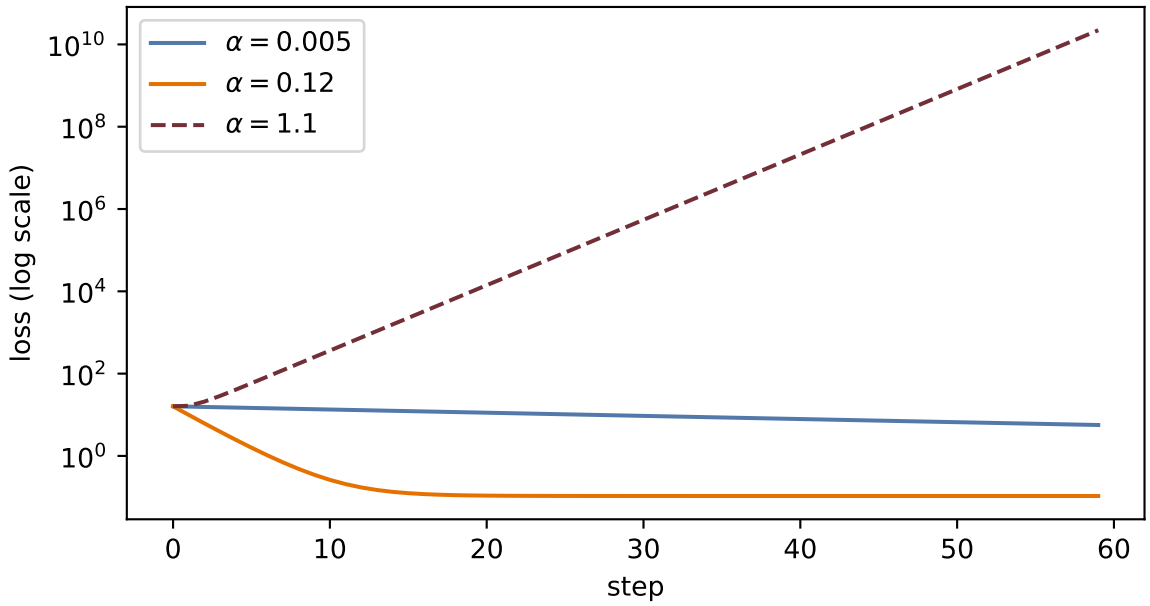

Figure 4.2.: Same problem, same steps, three learning rates. Too small crawls; too large overshoots back and forth and climbs; the middle one converges quickly.

> **Rule of thumb from the lectures**
>
> Start around $\alpha = 0.1$ and adjust based on what the training curves tell you: crawling → raise it; oscillating or exploding → lower it. Later in training it often pays to *decay* the learning rate so the fine-tuning steps get smaller — that is a **learning-rate schedule**. One exotic-sounding relative, *warmup* (starting tiny and ramping up), will matter enormously when we train Transformers in 1.

4.6. The batch size

The batch size B sets where you live on the noise–efficiency trade-off:

Batch size	Character	Trade-off
Small (1–32)	Noisy, exploratory	Better generalization; slow, underuses the GPU
Medium (32–256)	Balanced	The usual default; hardware-efficient
Large (256+)	Smooth, fast per epoch	Less exploration; can miss better minima and overfit

The noise level scales like $1/\sqrt{B}$: quadrupling the batch only halves the jitter. There is no free lunch here, only a dial — and the medium setting is the default for a reason.

4.7. Momentum: give the ball some mass

Vanilla SGD has exactly one control, α, and in narrow, curved valleys that is not enough: the iterate zigzags across the steep direction while inching along the shallow one. The fix is to give the update a memory. Keep a running **velocity** that accumulates gradients, and move along the velocity instead of the raw gradient:

$$\mathbf{v}^{(t)} = \beta\,\mathbf{v}^{(t-1)} + \nabla\mathcal{L}(\mathbf{w}^{(t)}), \qquad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\,\mathbf{v}^{(t)}, \tag{4.5}$$

with $\beta \in [0.9, 0.99]$. This is the ball rolling downhill: in directions where gradients keep agreeing, speed builds; in directions where they flip sign every step, they cancel. The zigzag damps itself, and small bumps in the landscape get rolled through rather than obeyed.

4.8. Adam: adaptive steps per knob

Momentum treats every parameter alike, but some knobs sit on steep cliffs while others sit on plains. **Adam** (adaptive moment estimation) combines three ideas from the lecture: momentum (accumulate past gradients), per-parameter adaptive learning rates (scale each knob's step by its own gradient history), and bias correction (account for both accumulators starting at zero). Formally, with elementwise operations:

$$\mathbf{m}^{(t)} = \beta_1\mathbf{m}^{(t-1)} + (1-\beta_1)\,\mathbf{g}^{(t)}, \quad \mathbf{s}^{(t)} = \beta_2\mathbf{s}^{(t-1)} + (1-\beta_2)\,\mathbf{g}^{(t)\,2}, \quad \mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\,\frac{\hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{s}}^{(t)}} + \epsilon}, \tag{4.6}$$

where $\hat{\mathbf{m}}, \hat{\mathbf{s}}$ are the bias-corrected accumulators. When to use what, per the lecture: **SGD with momentum** is simple, reliable, and strong for large models; **Adam** is the good default that works out of the box.

4.9. The race, on a problem where we know the truth

Claims about optimizers deserve a test you can rerun. We build a least-squares problem with a deliberately *ill-conditioned* landscape — the second feature is ten times the scale of the first, so the loss surface is a long narrow valley — and race the three optimizers with the same budget:

```
torch.manual_seed(6050)
n = 256
Xr = torch.randn(n, 2) * torch.tensor([1.0, 10.0])   # ill-conditioned by design
w_true = torch.tensor([3.0, 0.3])
yr = Xr @ w_true + 0.1 * torch.randn(n)

def race(kind: str, steps: int = 120, B: int = 16):
    torch.manual_seed(1)                              # same batches for everyone
    w = torch.zeros(2, requires_grad=True)
    opt = {"SGD":      torch.optim.SGD([w], lr=3e-3),
           "momentum": torch.optim.SGD([w], lr=3e-3, momentum=0.9),
           "Adam":     torch.optim.Adam([w], lr=0.1)}[kind]
    hist = []
    for _ in range(steps):
        idx = torch.randint(0, n, (B,))
        opt.zero_grad()
        ((Xr[idx] @ w - yr[idx]) ** 2).mean().backward()
        opt.step()
        hist.append(((Xr @ w.detach() - yr) ** 2).mean().item())
    return hist

plt.figure(figsize=(6.2, 3.6))
for kind, color in [("SGD", "#5379AA"), ("momentum", "#232D4B"),
                    ("Adam", "#E57200")]:
    plt.semilogy(race(kind), color=color, label=kind)
plt.xlabel("step"); plt.ylabel("full-batch loss (log scale)")
plt.legend(); plt.tight_layout(); plt.show()
```

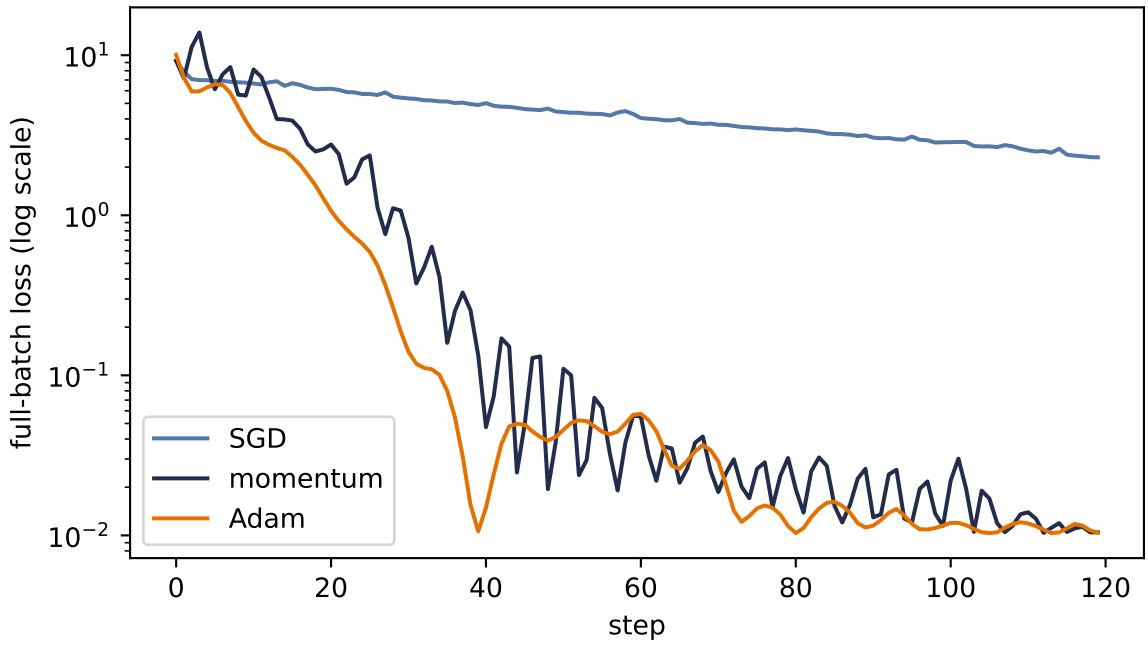

Figure 4.3.: The optimizer race on an ill-conditioned valley (feature scales 1 and 10). At the shared learning rate, plain SGD zigzags across the steep direction and crawls along the shallow one; momentum cancels the zigzag; Adam's per-parameter scaling neutralizes the conditioning almost entirely.

Read the result honestly. This is one problem, chosen to expose conditioning; it is not proof that Adam always wins (it does not — plain SGD with momentum, well tuned, still trains many of the best vision models). What the race does show is *mechanism*: when different directions of the landscape have wildly different steepness, per-direction memory (momentum) and per-parameter scaling (Adam) buy you orders of magnitude. It also whispers a practical lesson from the live session: much of this valley's narrowness came from unscaled features → *normalize your inputs* and you fight the optimizer less (Exercise 5).

4.10. Two regularizers that live inside training

Two ideas from Chapter 1 reappear here as training-loop citizens rather than loss-function algebra.

Weight decay. Ridge's L_2 penalty, seen from the optimizer's side, is one extra term in the update: shrink every weight slightly toward zero each step ($\mathbf{w} \leftarrow (1 - \alpha\lambda)\mathbf{w} - \alpha\nabla\mathcal{L}$). That is why every PyTorch optimizer takes a `weight_decay` argument — it is the same knob you met as ridge, now applied to any model, and you will see it in nearly every training recipe in this book.

Early stopping. Track the validation loss during training and stop when it turns upward, even though the training loss is still falling:

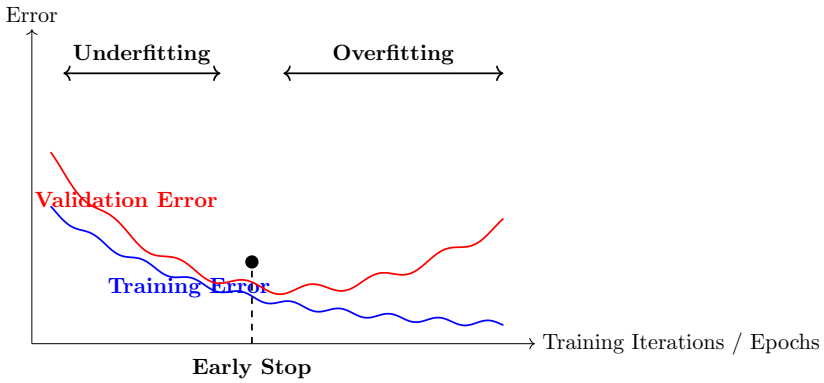

Figure 4.4.: Training error keeps falling; validation error turns. Stopping at the turn is regularization by clock — no penalty term required.

Both are cheap, both are everywhere, and both are previews: the full story of *why* constraining training improves generalization is the business of Chapter 6.

4.11. Okay, so — the training loop

Assemble the chapter into the loop you will run, in some form, for the rest of the book:

1. Start from (well-initialized) random parameters.
2. For each epoch: shuffle, split into mini-batches of size B.
3. For each batch: forward pass $\rightarrow$ loss (your noise model in disguise) $\rightarrow$ backward pass for the gradients $\rightarrow$ optimizer step (α, momentum or Adam, weight decay).
4. Watch the validation curve; schedule the learning rate; stop early when it turns.

One box in that loop is still magic: "backward pass for the gradients." Computing millions of partial derivatives at the cost of roughly one extra forward pass is the subject of Chapter 5.

Exercises

1. **(Pencil.)** Prove Equation 4.4 for sampling with replacement: if i is drawn uniformly from $\{1, \dots, n\}$, show $\mathbb{E}[\nabla \ell_i(\mathbf{w})] = \frac{1}{n} \sum_j \nabla \ell_j(\mathbf{w})$. Where exactly does the argument use uniformity?
2. **(Code.)** In the two-zones figure, sweep $B \in \{1, 4, 16, 80\}$ and plot the final 30 steps of each path. How does the radius of the region of confusion scale with B? Compare against the $1/\sqrt{B}$ rule.
3. **(Code.)** Add a learning-rate schedule to the race: halve α every 30 steps for plain SGD. How much of the gap to momentum does scheduling close? What does that tell you about what momentum is *really* fixing here?
4. **(Code.)** Sweep momentum's $\beta \in \{0, 0.5, 0.9, 0.99\}$ in the race. Explain the failure mode at the top end using the ball analogy.

5. **(Code.)** Standardize the race's features (divide each column of `Xr` by its standard deviation) and rerun all three optimizers. How much of Adam's advantage evaporates? State the practical moral in one sentence.

5. Backpropagation

Recall the update rule that ended Chapter 1: new parameters come from old parameters by stepping against the gradient, $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} \mathcal{L}$. For linear regression we could write that gradient down by hand. But a modern network has millions, sometimes billions, of knobs, tangled through layer after layer of composed functions → computing each partial derivative naively, one at a time, is hopeless.

Backpropagation is the algorithm that computes *all* of them, exactly, in one backward sweep whose cost is linear in the number of parameters. It is not a new kind of calculus. It is the chain rule, *organized*. By the end of this chapter you will have derived it, implemented it by hand, rebuilt it as a 60-line autograd engine, and then checked that `torch.autograd` agrees with you to machine precision.

5.1. One neuron, one chain

Start with the smallest possible case: one neuron on one training example. The previous activation $a^{(l-1)}$ is multiplied by a weight w, giving the pre-activation $z = w\,a^{(l-1)} + b$; the activation function produces $a = \sigma(z)$; and a feeds a loss L.

Now turn the knob w a little. How much does L change? Follow the chain: turning w changes z; the change in z changes a; the change in a changes L. The chain rule multiplies these local sensitivities:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}. \tag{5.1}$$

```
import matplotlib.pyplot as plt
import matplotlib.patches as mp

fig, ax = plt.subplots(figsize=(7.2, 2.4))
nodes = {r"$w$": 0, r"$z = w a^{(l-1)} + b$": 1, r"$a=\sigma(z)$": 2, r"$L$": 3}
for label, i in nodes.items():
    ax.add_patch(mp.FancyBboxPatch((i * 2.1, 0), 1.5, 0.7, boxstyle="round,pad=0.08",
                                   fc="#E8EEF7", ec="#232D4B", lw=1.4))
    ax.text(i * 2.1 + 0.75, 0.35, label, ha="center", va="center", fontsize=11)
fwd = dict(arrowstyle="-|>", color="#5379AA", lw=1.8)
bwd = dict(arrowstyle="-|>", color="#E57200", lw=1.8)
lbl = [r"$\frac{\partial z}{\partial w}$", r"$\frac{\partial a}{\partial z}$",
       r"$\frac{\partial L}{\partial a}$"]
```

```
for i in range(3):
    ax.annotate("", xy=(2.1 * (i + 1) - 0.1, 0.55), xytext=(2.1 * i + 1.6, 0.55),
                arrowprops=fwd)
    ax.annotate("", xy=(2.1 * i + 1.6, 0.12), xytext=(2.1 * (i + 1) - 0.1, 0.12),
                arrowprops=bwd)
    ax.text(2.1 * i + 1.85, -0.28, lbl[i], ha="center", fontsize=12, color="#E57200")
ax.set_xlim(-0.3, 8.3); ax.set_ylim(-0.7, 1.1); ax.axis("off")
plt.tight_layout(); plt.show()
```

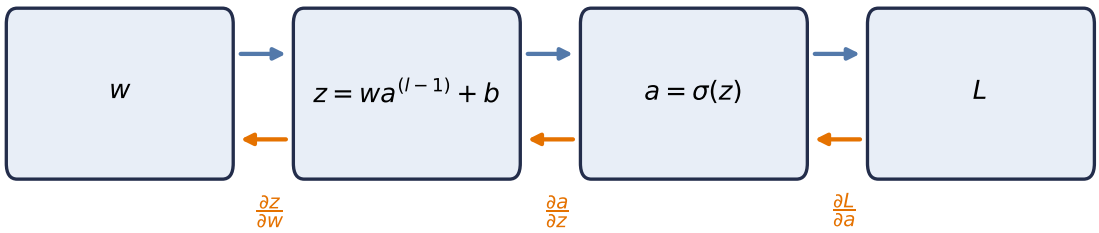

Figure 5.1.: Computation as a graph: the forward pass (top, blue) computes and caches values; the backward pass (bottom, orange) multiplies local derivatives along the same edges, in reverse.

Two things about this picture generalize to any network, however deep:

1. **The forward pass caches.** Computing L produces the intermediate values (z, a) along the way; we keep them, because the backward pass will need them.
2. **The backward pass reuses.** Each edge contributes one *local* derivative, and the derivative of L with respect to anything is the product of local derivatives along the path back to it. Nothing is recomputed → the whole sweep costs about as much as one forward pass.

5.2. The game of blame

For a full layer of neurons the bookkeeping threatens to become a tangled mess of sums. The cure is to pick the right intermediate quantity and track only that. At the output of the network the error is obvious: it is just the gap to the target. What is *not* obvious is how much of that error is the fault of a weight buried ten layers deep. Backpropagation sends this blame backward, from where it is obvious to where it is not.

The blame signal

Define, for each layer l, the **blame signal**

$$\delta^{(l)} := \frac{\partial L}{\partial \mathbf{z}^{(l)}},$$

the sensitivity of the loss to that layer's *pre-activations*. It answers: how much did each neuron's z contribute to the final loss?

Everything now unfolds in four short equations. With $\odot$ denoting elementwise (Hadamard) product:

1. Start where blame is obvious (output layer L):

$$\delta^{(L)} = \frac{\partial L}{\partial \mathbf{a}^{(L)}} \odot \sigma'(\mathbf{z}^{(L)}). \tag{5.2}$$

For squared error, $\partial L/\partial \mathbf{a}^{(L)}$ is literally the prediction error $\mathbf{a}^{(L)} - \mathbf{y}$: the error, scaled by how sensitive the activation was to its input.

2. Propagate blame one layer back:

$$\delta^{(l)} = \left((\mathbf{W}^{(l+1)})^\top \delta^{(l+1)}\right) \odot \sigma'(\mathbf{z}^{(l)}). \tag{5.3}$$

Blame flows backward through the same weights the signal flowed forward through, hence the transpose; then the local gate $\sigma'(\mathbf{z}^{(l)})$ scales it. This recursion runs from the last layer to the first.

3. Convert blame into weight gradients:

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} \, (\mathbf{a}^{(l-1)})^\top. \tag{5.4}$$

4. And into bias gradients:

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}. \tag{5.5}$$

Equation Equation 5.4 deserves a pause, because its shapes tell the story. With m neurons in layer l and n in layer $l-1$: $\delta^{(l)}$ is $(m \times 1)$, and $(\mathbf{a}^{(l-1)})^\top$ is $(1 \times n)$. Their **outer product** is an $m \times n$ matrix, exactly the shape of $\mathbf{W}^{(l)}$, and entry (i, j) is the blame of neuron i times the activation of neuron j that fed it. The update for a weight is *blame out times signal in*. No messy summations: the matrix form performs them all at once, and it maps directly onto the parallel matrix multiplies GPUs are built for. The same four equations govern a toy network and a billion-parameter model; only the matrix sizes change.

5.3. Build it, then check it against autograd

Let us implement the four equations on a two-layer network, with no autograd anywhere, and then ask `torch.autograd` to differentiate the same network. If our blame algebra is right, the two answers must agree to machine precision.

```
import torch

torch.manual_seed(6050)
```

```
<torch._C.Generator at 0x129a216b0>
```

```
n, d, h = 32, 5, 8                              # batch, input dim, hidden dim
X = torch.randn(n, d)
y = torch.randn(n)

W1, b1 = torch.randn(h, d) * 0.3, torch.zeros(h)
W2, b2 = torch.randn(1, h) * 0.3, torch.zeros(1)

# forward pass -- cache everything the backward pass will need
Z1 = X @ W1.T + b1                              # (n, h)
A1 = torch.sigmoid(Z1)                          # (n, h)
Z2 = A1 @ W2.T + b2                             # (n, 1)
L = ((Z2.squeeze() - y) ** 2).mean()

# backward pass -- the four equations (batch-averaged)
sig_prime = A1 * (1 - A1)                       # sigma'(z) = sigma(z)(1 - sigma(z))
delta2 = 2 * (Z2.squeeze() - y)[:, None] / n  # eq. 1 (identity output: no gate)
delta1 = (delta2 @ W2) * sig_prime              # eq. 2: blame through W^T, gated
grads = {                                       # eqs. 3 & 4: blame out x signal in
    "W2": delta2.T @ A1, "b2": delta2.sum(0),
    "W1": delta1.T @ X,  "b1": delta1.sum(0),
}

# same network, autograd's turn
params = {k: v.clone().requires_grad_(True) for k, v in
          {"W1": W1, "b1": b1, "W2": W2, "b2": b2}.items()}
A1_ = torch.sigmoid(X @ params["W1"].T + params["b1"])
L_ = (((A1_ @ params["W2"].T + params["b2"]).squeeze() - y) ** 2).mean()
L_.backward()

for k in grads:
    gap = (grads[k] - params[k].grad).abs().max()
    print(f"{k}: max |ours - autograd| = {gap:.2e}")
```

```
W2: max |ours - autograd| = 0.00e+00
b2: max |ours - autograd| = 0.00e+00
W1: max |ours - autograd| = 3.73e-09
b1: max |ours - autograd| = 1.86e-09
```

Zeros, to floating-point precision. The blame equations *are* what autograd computes; the framework simply builds the computation graph for us as we go and then walks it backward, caching and reusing exactly as we did.

5.4. A 60-line autograd engine

To remove the last trace of magic, let us build the graph ourselves. Each `Value` node remembers how it was made (its parents and the local derivative rule); calling `backward()` walks the graph in reverse topological order, accumulating blame into every node's `grad`.

```python
class Value:
    """A scalar that remembers its history -- a node in the computation graph."""

    def __init__(self, data: float, parents=(), backward_rule=lambda: None):
        self.data = data
        self.grad = 0.0
        self._parents = parents
        self._backward_rule = backward_rule

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other))
        def rule():                       # d(out)/d(self) = d(out)/d(other) = 1
            self.grad += out.grad
            other.grad += out.grad
        out._backward_rule = rule
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other))
        def rule():                       # product rule, one side at a time
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward_rule = rule
        return out

    def sigmoid(self):
        s = 1 / (1 + 2.718281828459045 ** (-self.data))
        out = Value(s, (self,))
```

```
        def rule():                         # sigma' = s (1 - s), the gate
            self.grad += s * (1 - s) * out.grad
        out._backward_rule = rule
        return out

    def backward(self):
        order, seen = [], set()
        def topo(v):                        # children before parents
            if v not in seen:
                seen.add(v)
                for p in v._parents:
                    topo(p)
                order.append(v)
        topo(self)
        self.grad = 1.0                     # dL/dL = 1: blame starts here
        for v in reversed(order):
            v._backward_rule()
```

```
w, x, b = Value(0.7), Value(2.0), Value(-0.5)
a = ((w * x) + b).sigmoid()             # one neuron, forward
loss = (a + (-0.3)) * (a + (-0.3))      # squared error vs target 0.3
loss.backward()

# analytic check: dL/dw = 2(a - y) * sigma'(z) * x
z = 0.7 * 2.0 - 0.5
s = 1 / (1 + 2.718281828459045 ** (-z))
print(f"micro-autograd: dL/dw = {w.grad:.6f}")
print(f"by hand:        dL/dw = {2 * (s - 0.3) * s * (1 - s) * 2.0:.6f}")
```

```
micro-autograd: dL/dw = 0.337801
by hand:        dL/dw = 0.337801
```

Sixty lines, and it is the real thing: PyTorch's autograd is this idea with tensors instead of scalars, a compiled graph walker, and thousands of derivative rules.

⚠ Autograd hygiene (the pitfalls that actually bite)

- **Gradients accumulate.** `backward()` *adds* into `.grad`; forget `optimizer.zero_grad()` and every step uses the sum of all past gradients.
- **Updates happen outside the graph.** Parameter updates go inside `torch.no_grad()`; otherwise the update itself is recorded and memory explodes.
- **`detach()` cuts the tape.** Use it when you want a value without its history (logging, plotting, building targets).

5.5. The gate, and the problem with deep chains

Look again at Equation 5.3. Every backward step multiplies by $\sigma'(\mathbf{z}^{(l)})$. That derivative acts as a **gate** on the blame: wide open, and the signal passes; nearly closed, and it dies.

Now examine the sigmoid's gate. From $\sigma'(z) = \sigma(z)(1-\sigma(z))$, its maximum value, at $z = 0$, is exactly 0.25; away from zero the neuron *saturates* and the gate approaches 0. The sigmoid gate is at best a quarter open, and often nearly closed.

```
import matplotlib.pyplot as plt

z = torch.linspace(-6, 6, 300)
sig = torch.sigmoid(z)
fig, axes = plt.subplots(1, 2, figsize=(7.4, 2.9), sharey=True)
axes[0].plot(z, sig * (1 - sig), color="#232D4B")
axes[0].axhline(0.25, ls="--", color="#E57200", lw=1)
axes[0].text(2.1, 0.26, "ceiling: 0.25", color="#E57200", fontsize=9)
axes[0].set_title(r"sigmoid gate $\sigma'(z)$"); axes[0].set_xlabel("$z$")
axes[1].plot(z, (z > 0).float(), color="#232D4B")
axes[1].set_title(r"ReLU gate"); axes[1].set_xlabel("$z$")
plt.tight_layout(); plt.show()
```

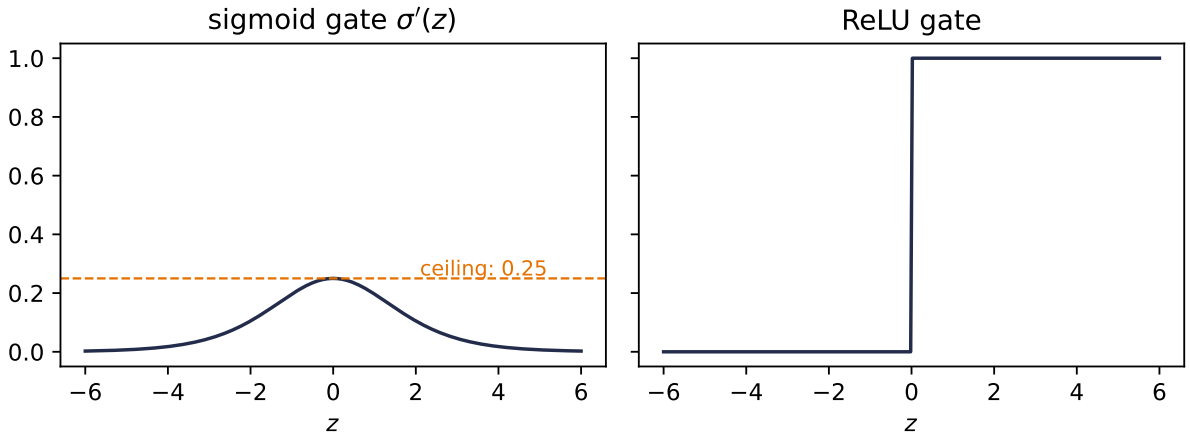

Figure 5.2.: Two gates. The sigmoid's derivative never exceeds 0.25 and vanishes under saturation; ReLU's is fully open (exactly 1) for every active neuron.

Here is the consequence, and it is the single most important failure mode in deep learning. The chain rule *multiplies* gates across layers → with sigmoid activations the backward signal shrinks by a factor of at most 0.25 per layer, so after k layers it is bounded by 0.25^k. Ten layers in, the ceiling is below one in a million. The blame reaching the early layers is too weak to update them: the **vanishing gradient** problem.

The fix is almost embarrassingly simple. $\mathrm{ReLU}(z) = \max(0, z)$ has derivative 1 for $z > 0$ and 0 otherwise (the kink at exactly zero never matters in floating point). The gate is either fully

open or fully closed. For every neuron that was active in the forward pass, blame passes through *unchanged*: a **gradient superhighway** through the network. This, more than anything, is why ReLU and its variants are the default in deep networks.

Do not take the argument on faith; measure it. We build the same 30-layer network twice (sigmoid vs. ReLU), push one batch through, and record how much gradient reaches each layer:

```
from torch import nn

def grad_by_layer(activation: type[nn.Module], depth: int = 30, width: int = 64):
    layers = []
    for _ in range(depth):
        layers += [nn.Linear(width, width), activation()]
    net = nn.Sequential(*layers, nn.Linear(width, 1))
    out = net(torch.randn(128, width)).mean()
    out.backward()
    return [lin.weight.grad.norm().item()
            for lin in net if isinstance(lin, nn.Linear)][:-1]

torch.manual_seed(6050)
plt.figure(figsize=(6.2, 3.4))
for act, color in [(nn.Sigmoid, "#232D4B"), (nn.ReLU, "#E57200")]:
    plt.semilogy(grad_by_layer(act), "o-", ms=3, color=color, label=act.__name__)
plt.xlabel("layer (0 = closest to input)")
plt.ylabel(r"$\|\partial L / \partial W^{(l)}\|$")
plt.legend(); plt.tight_layout(); plt.show()
```

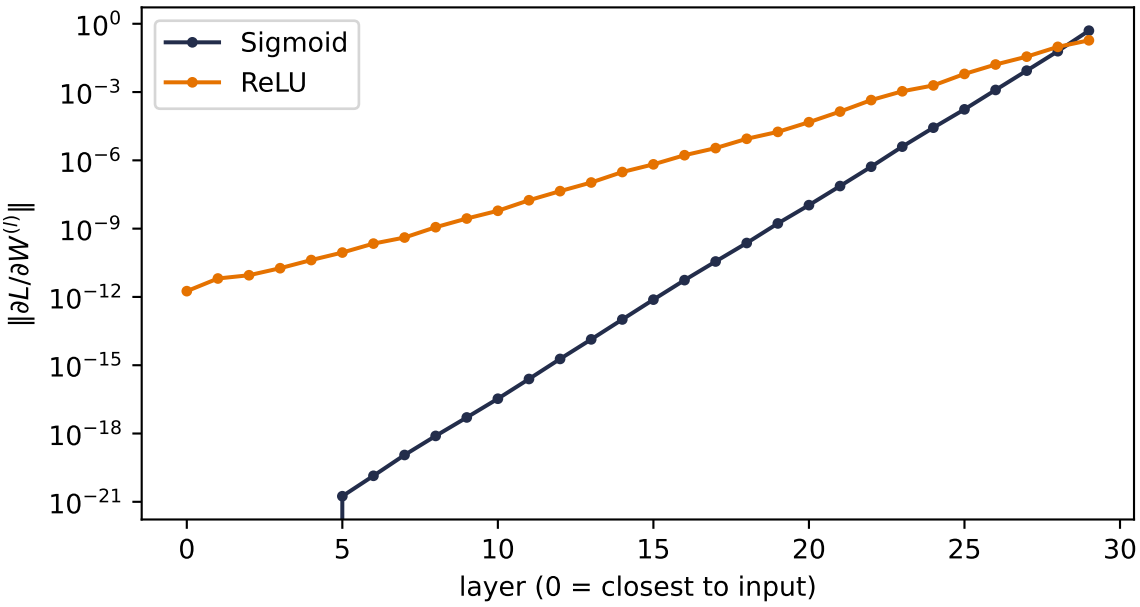

Figure 5.3.: Gradient reaching each layer of a 30-layer MLP (log scale). Depth attenuates both, but the sigmoid gates compound the decay so violently that below layer 5 the gradient falls beneath floating-point precision entirely, while ReLU's per-layer decay stays gentle enough for usable signal to reach the input.

Read the slopes: every step backward through a sigmoid layer multiplies the signal by its nearly-closed gate, and the compounding is so violent that by five layers from the input the gradient is numerically *zero* — those layers will never learn. ReLU is not magic (depth still attenuates the signal), but its open gates make the per-layer decay orders of magnitude gentler, and usable blame reaches all the way down. When we build deep MLPs for real in Chapter 3, this plot is the reason they train at all.

5.6. Initialization: setting the signal's energy

One more lever lives in this chapter, because it falls out of the same variance bookkeeping. Think of the network as a pipeline and the variance of the signal as its *energy*. If each layer multiplies the variance by more than one, activations explode after enough layers; by less than one, they vanish, and by Equation 5.3 the gradients follow. A good initialization keeps the energy roughly constant in both directions.

The analysis is one line. For $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ with independent, zero-mean inputs and weights, $\operatorname{Var}(z) = n_{\text{in}} \operatorname{Var}(w) \operatorname{Var}(x)$. Keeping $\operatorname{Var}(z) = \operatorname{Var}(x)$ demands

$$\operatorname{Var}(w) = \frac{1}{n_{\text{in}}} \quad \text{(forward)}, \qquad \operatorname{Var}(w) = \frac{1}{n_{\text{out}}} \quad \text{(backward)}.$$

Xavier initialization splits the difference for symmetric activations like tanh and sigmoid, $\mathrm{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$. **He initialization** accounts for ReLU discarding half its input distribution by doubling the forward recipe, $\mathrm{Var}(w) = 2/n_{\text{in}}$. PyTorch applies sensible defaults for you; the difference between a good and a careless choice is smooth training versus a dead or exploding network:

```
def signal_std(scale_fn, depth: int = 40, width: int = 256):
    x = torch.randn(512, width)
    stds = []
    for _ in range(depth):
        W = scale_fn(torch.randn(width, width), width)
        x = torch.relu(x @ W)
        stds.append(x.std().item())
    return stds

torch.manual_seed(6050)
schemes = {
    "naive  (std = 0.05)": lambda W, n: W * 0.05,
    "naive  (std = 0.12)": lambda W, n: W * 0.12,
    "He  (std = sqrt(2/n))": lambda W, n: W * (2 / n) ** 0.5,
}
plt.figure(figsize=(6.2, 3.4))
for (name, fn), color in zip(schemes.items(), ["#5379AA", "#722F37", "#E57200"]):
    plt.semilogy(signal_std(fn), color=color, label=name)
plt.xlabel("layer"); plt.ylabel("std of activations")
plt.legend(); plt.tight_layout(); plt.show()
```

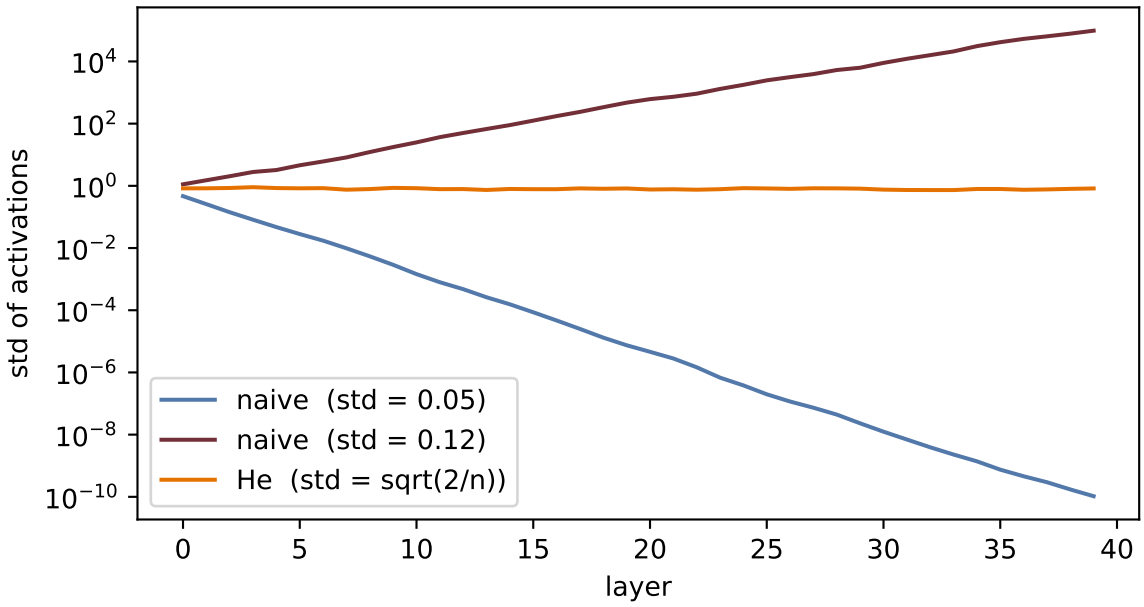

Figure 5.4.: Signal energy through 40 ReLU layers. Careless scales lose or amplify the signal exponentially; He initialization holds it steady.

Initialization gives a stable *start*. Keeping the signal stable *during* training, as the weights move, is the job of normalization layers; and for very deep networks even that is not enough, and gradients get an architectural bypass: skip connections that add a block's input straight to its output. Both belong to later chapters (normalization with training practice in Chapter 4 and Chapter 9; the residual idea in Chapter 9, and layer normalization where it becomes essential, inside the Transformer of 1). What you should carry from here is the invariant they all serve: *keep the signal, forward and backward, at constant energy.*

5.7. Okay, so — the chain rule, organized

1. **The graph.** Any network is a computation graph; the forward pass computes and caches, the backward pass multiplies local derivatives in reverse.
2. **The blame signal.** $\delta^{(l)} = \partial L/\partial \mathbf{z}^{(l)}$ turns the tangle into four equations: start at the output (Equation 5.2), propagate through $\mathbf{W}^\top$ and the gate (Equation 5.3), then read off weight and bias gradients (Equation 5.4, Equation 5.5). Cost: one extra forward pass, for *every* gradient at once.
3. **We verified it**: our hand-built equations match `torch.autograd` to 10^{-8}, and a 60-line `Value` class reproduces the machinery end to end.
4. **The gate rules the depth.** Multiplied sigmoid gates vanish like 0.25^k; ReLU's open gate is a gradient superhighway. Initialization sets the signal's energy so neither direction explodes or dies at the start.

Backpropagation is the engine under every remaining chapter. The next time a deep model fails to learn, your first questions now have names: *is the gradient reaching the early layers? what is*

gating it? what is its energy?

Exercises

1. **(Pencil.)** Derive the four blame equations for a network whose output layer uses the identity activation and squared-error loss (the network of our verification cell). Confirm that your $\delta^{(2)}$ matches the `delta2` line in the code.
2. **(Pencil.)** In Equation 5.3, why does the blame flow through $(\mathbf{W}^{(l+1)})^\top$ rather than $\mathbf{W}^{(l+1)}$? Argue from shapes: what are the dimensions of $\delta^{(l)}$, $\delta^{(l+1)}$, and $\mathbf{W}^{(l+1)}$?
3. **(Code.)** Extend the `Value` class with `tanh` and use it to train the one-neuron model from the check cell for 50 steps (write the update loop yourself). Verify the loss decreases.
4. **(Code.)** In the depth experiment, give the sigmoid network Xavier initialization (`nn.init.xavier_normal_` on each linear layer). How much of the vanishing does good initialization recover at depth 30? What does that tell you about why both fixes, initialization *and* ReLU, became standard?
5. **(Code.)** Comment out `optimizer.zero_grad()` in any training loop from Chapter 1 and describe what happens to the loss curve. Explain it with one sentence about `.grad` accumulation.

6. When It Fails: Generalization in Pictures, and Inductive Bias

Part I has given us everything a course could promise: models with universal expressive power (Chapter 3), losses grounded in maximum likelihood (Chapter 2), an optimizer that scales (Chapter 4), and gradients for anything (Chapter 5). This chapter takes all of it, aims it at real images, and succeeds completely.

Then we will look a little closer, and the wheels will come off. Watching *how* they come off — in pictures, not in theorems — is the most important lesson of Part I, because the cure is the idea the entire rest of this book is built on.

6.1. Victory

The data is a subset of Fashion-MNIST, the classic benchmark from our live sessions: 28×28 grayscale images of clothing in ten classes. The model and training loop hold no surprises — an MLP from Chapter 3, trained exactly as in Chapter 4:

```
import torch
from torch import nn

torch.manual_seed(6050)
train = torch.load("../../data/fashion-train.pt")
test = torch.load("../../data/fashion-test.pt")
X_tr = train["X"].float().reshape(-1, 784) / 255.0     # (1200, 784)
X_te = test["X"].float().reshape(-1, 784) / 255.0      # (600, 784)
y_tr, y_te, classes = train["y"], test["y"], train["classes"]

def train_mlp(X, y, hidden=256, epochs=40, seed=6050):
    torch.manual_seed(seed)
    net = nn.Sequential(nn.Linear(784, hidden), nn.ReLU(), nn.Linear(hidden, 10))
    opt = torch.optim.Adam(net.parameters(), lr=1e-3)
    for _ in range(epochs):
        perm = torch.randperm(len(X))
        for i in range(0, len(X), 64):
            idx = perm[i:i + 64]
            opt.zero_grad()
            nn.functional.cross_entropy(net(X[idx]), y[idx]).backward()
            opt.step()
```

```
    return net

def accuracy(net, X, y):
    with torch.no_grad():
        return (net(X).argmax(1) == y).float().mean().item()

net = train_mlp(X_tr, y_tr)
print(f"train accuracy: {accuracy(net, X_tr, y_tr):.1%}")
print(f"test accuracy:  {accuracy(net, X_te, y_te):.1%}")
```

```
train accuracy: 96.6%
test accuracy:  80.8%
```

Eighty percent on unseen images, from twelve hundred examples and a few seconds of CPU. The test set is honest, the loop is standard, the number is real. By every metric we have learned to compute, we are done.

The rest of this chapter is about why we are not.

6.2. Two experiments that should worry you

6.2.1. Experiment 1: move everything two pixels

Take the test images and shift each one two pixels to the right. Nothing about any garment changes — a coat is a coat, wherever it hangs in the frame. Any human would call the two test sets identical.

```
def shift_right(X, px: int):
    img = X.reshape(-1, 28, 28)
    out = torch.zeros_like(img)
    if px > 0:
        out[:, :, px:] = img[:, :, :-px]
    else:
        out = img.clone()
    return out.reshape(-1, 784)

for px in [0, 2]:
    print(f"shift {px}px: test accuracy {accuracy(net, shift_right(X_te, px), y_te):.1%}")
```

```
shift 0px: test accuracy 80.8%
shift 2px: test accuracy 42.0%
```

Two pixels, and the accuracy is nearly cut in half. Push further and the collapse completes:

```
import matplotlib.pyplot as plt

shifts = list(range(5))
accs = [accuracy(net, shift_right(X_te, s), y_te) for s in shifts]
plt.figure(figsize=(5.6, 3.3))
plt.plot(shifts, accs, "o-", color="#E57200", lw=2, ms=7)
plt.axhline(0.1, ls=":", color="#B8B8A8")
plt.text(0.1, 0.115, "chance (10 classes)", color="#8A8A7A", fontsize=9)
plt.xlabel("shift (pixels right)"); plt.ylabel("test accuracy")
plt.ylim(0, 0.9); plt.xticks(shifts)
plt.tight_layout(); plt.show()
```

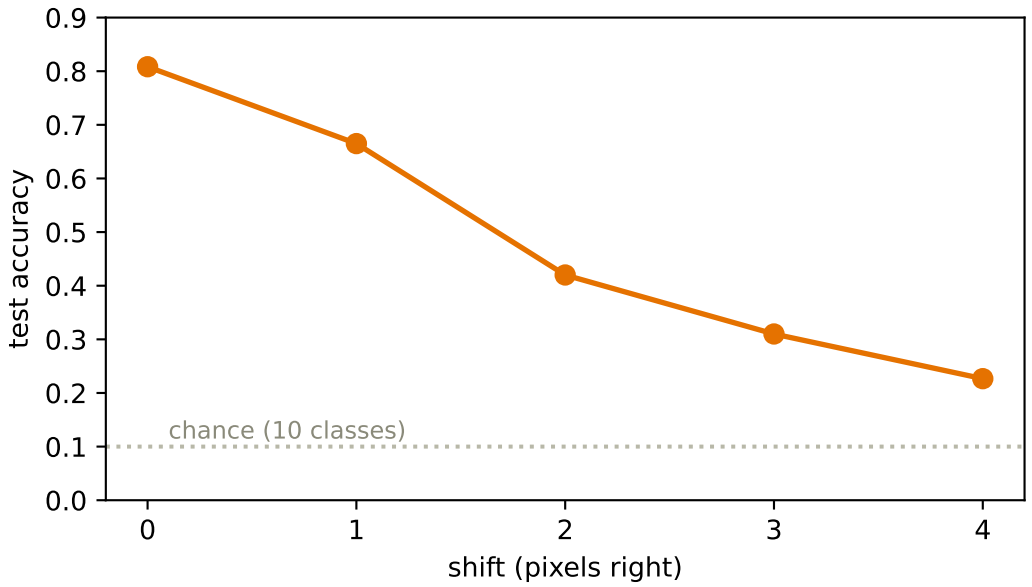

Figure 6.1.: The cliff. Each pixel of horizontal shift destroys accuracy the garments themselves never changed. By four pixels the model is approaching guesswork.

6.2.2. Experiment 2: destroy the images entirely

Now the stranger experiment, from the lecture's "fatal flaw" argument. Choose one fixed random permutation of the 784 pixel positions and apply it to *every* image, train and test alike. The results are not pictures anymore — no human could classify them. Every trace of spatial structure is gone. Retrain the same MLP on this scrambled world:

```
torch.manual_seed(0)
pixel_perm = torch.randperm(784)          # one fixed scrambling for all images
```

```
net_shuffled = train_mlp(X_tr[:, pixel_perm], y_tr)
print(f"original images:  test accuracy {accuracy(net, X_te, y_te):.1%}")
print(f"shuffled pixels:  test accuracy "
      f"{accuracy(net_shuffled, X_te[:, pixel_perm], y_te):.1%}")
```

```
original images:  test accuracy 80.8%
shuffled pixels:  test accuracy 82.2%
```

The accuracy is *the same* (the small difference is seed noise). Read that again: destroying everything that makes an image an image cost the MLP **nothing**.

Hold the two results side by side and the diagnosis writes itself:

> The model is **indifferent** to the structure that matters — pixel adjacency, shape, the entire geometry of the image — and **hypersensitive** to a nuisance that should not matter at all — where the garment happens to sit. It learned the dataset without ever learning what a picture is.

6.3. The autopsy, in pictures

Why exactly does shifting hurt a model that shuffling cannot? Look at what the model actually learned. Each row of the first layer's weight matrix is a 784-vector — which means we can reshape it into a 28×28 image and look at it. And we know from Chapter 1 what such a row *is*: a **template**, scored against the input by dot product.

```
W1 = net[0].weight.detach()                # (256, 784)
order = W1.norm(dim=1).argsort(descending=True)[:16]

fig, axes = plt.subplots(2, 8, figsize=(7.6, 2.2))
for ax, i in zip(axes.ravel(), order):
    ax.imshow(W1[i].reshape(28, 28), cmap="RdBu_r")
    ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()
```

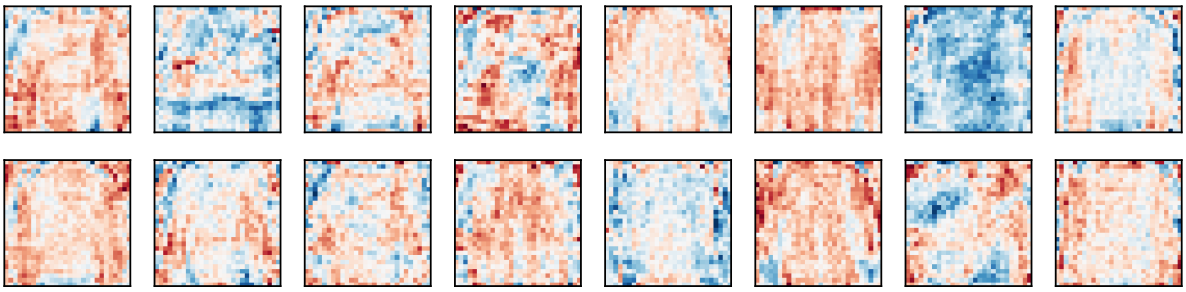

Figure 6.2.: Sixteen first-layer templates, reshaped into images. Each is a global, full-frame pattern: fabric-like textures and garment silhouettes smeared across the entire 28×28 canvas. A template that spans the whole frame breaks the moment its pattern moves — and never needed the frame to be a picture in the first place.

There is the whole story in one figure. Every template is **global** — a pattern stretched across the entire frame, tuned to where garments sat in the training images. Shift the input two pixels and every pixel of every template now scores against the wrong part of the garment; a thousand carefully tuned dot products all misfire at once. And conversely: a global bag-of-pixels template has no notion of *adjacency* to lose, so scrambling the pixels — consistently — just hands it a different bag, which it learns as happily as the first. Indifference and hypersensitivity are the same fact about the same templates.

Capacity was never the issue. By Chapter 3's universal approximation theorem, this network *can represent* a perfectly shift-tolerant classifier. It has no reason to find one: nothing in its architecture, loss, or data ever told it that position is a nuisance and adjacency is meaningful.

6.4. Naming the disease

With the failure in hand, the classical concepts of Part I snap into focus.

6.4.1. Hunting the U-curve

The textbook story (Chapter 1's cartoon) predicts that as capacity grows, validation error should fall and then *rise* as the model overfits. We now have everything needed to test the cartoon on real data — a width sweep at two dataset sizes:

```
widths = [2, 4, 8, 16, 64, 256]
plt.figure(figsize=(6, 3.5))
for n, color in [(300, "#5379AA"), (1200, "#232D4B")]:
    errs = [1 - accuracy(train_mlp(X_tr[:n], y_tr[:n], hidden=h, epochs=60), X_te, y_te)
            for h in widths]
    plt.semilogx(widths, errs, "o-", color=color, label=f"n = {n}", base=2)
plt.xlabel("hidden width (capacity)"); plt.ylabel("validation error")
plt.legend(); plt.tight_layout(); plt.show()
```

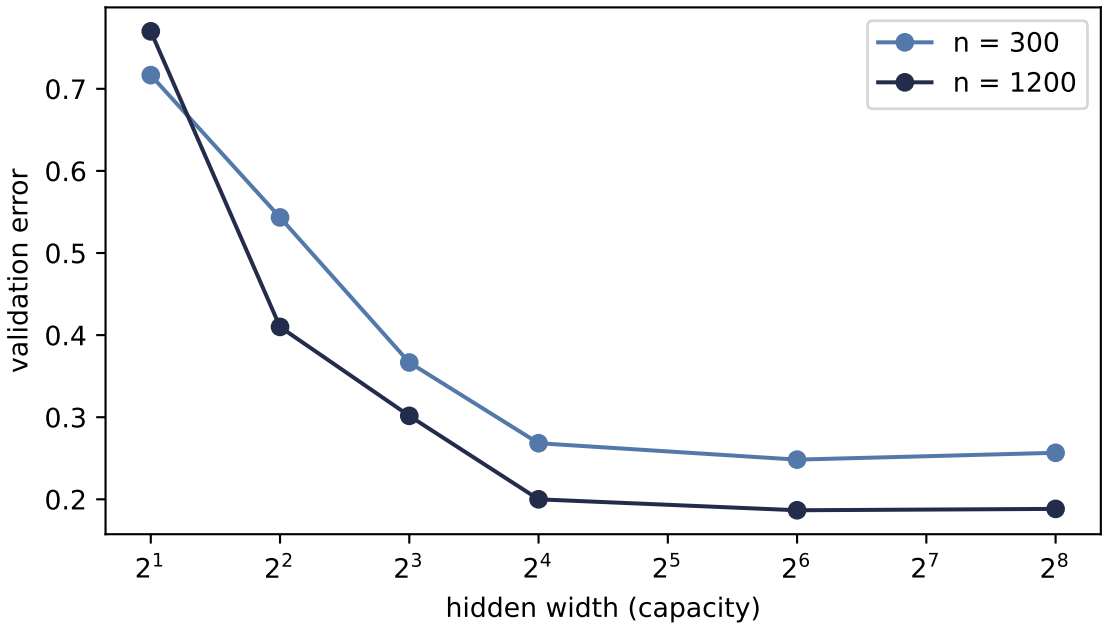

Figure 6.3.: Hunting the U-curve: validation error vs. hidden width at two training-set sizes. Error falls steeply as capacity grows — then flattens. At n=300 the predicted upturn is barely a whisper (width 64 → 256); at n=1200 it is absent. More data lowers and flattens the whole curve (variance ~ 1/n). The cartoon is real physics but weak medicine: capacity is not what is failing here.

Read it honestly. The left side of the cartoon is emphatically real: too little capacity underfits badly. But the dreaded right side barely materializes — a whisper of an upturn at the small dataset, nothing at the larger one. This is the modern experience of deep learning, and it is exactly the caveat planted in Chapter 1: the U is a helpful cartoon, not a law of nature. The noisy optimizer of Chapter 4 quietly regularizes, more data flattens the curve, and over-parameterized networks routinely generalize fine.

Which sharpens our problem instead of solving it: **our model does not fail for having too much capacity. It fails for having no knowledge.** No point on either curve survives a two-pixel shift.

6.4.2. The curse of dimensionality

Could we fix everything with data alone — just show the model garments at every position? Count what that costs. Our images live in 784 dimensions; a small color image (32×32×3) needs 3,072 numbers, a one-megapixel photo three million, an ImageNet input 150,528. And covering a d-dimensional space densely requires exponentially many examples: if 9 points cover a 1-D interval at some resolution, matching that density takes 9^2 points in 2-D and 9^d in d dimensions. For $d = 784$, there are not enough atoms in the universe, at any exchange rate.

High dimensionality also breaks the very idea of "similar examples":

```
torch.manual_seed(6050)
dims = [2, 10, 100, 784, 5000]
ratios = []
for d in dims:
    P = torch.rand(300, d)
    D = torch.cdist(P, P)
    ratios.append(((D + torch.eye(300) * 1e9).min(1).values / D.max(1).values).mean())

plt.figure(figsize=(5.6, 3.2))
plt.semilogx(dims, ratios, "o-", color="#722F37", lw=2)
plt.axvline(784, ls=":", color="#B8B8A8")
plt.text(830, 0.25, "Fashion-MNIST\n(d = 784)", fontsize=9, color="#8A8A7A")
plt.xlabel("dimension $d$"); plt.ylabel("nearest / farthest distance")
plt.ylim(0, 1); plt.tight_layout(); plt.show()
```

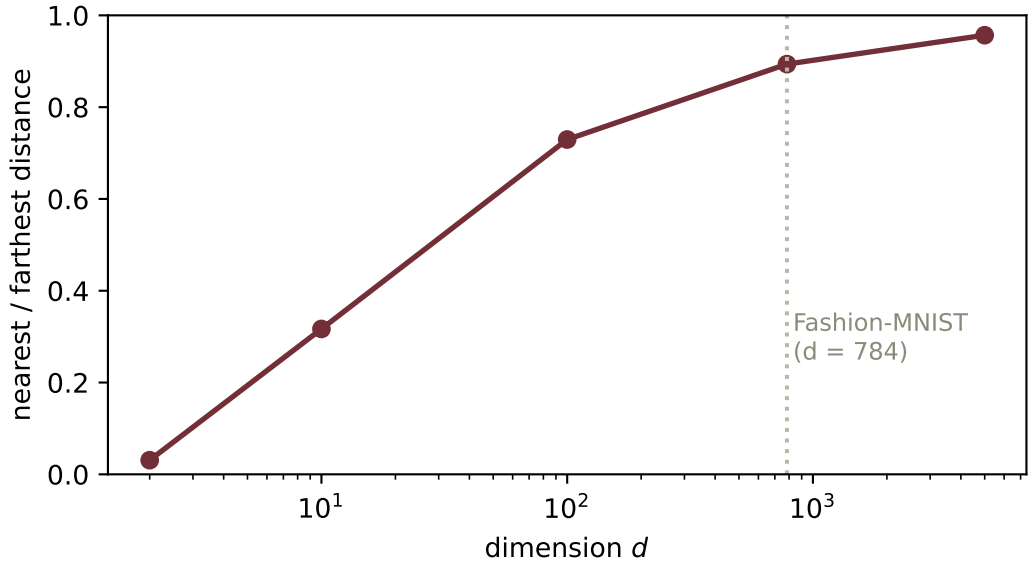

Figure 6.4.: Distance concentration: for 300 random points, the average ratio of nearest-neighbor to farthest-neighbor distance, by dimension. In 2-D your nearest neighbor is genuinely near (ratio 0.03). At d = 784 — our images' dimension — it is already 89% as far as the farthest point: 'nearby examples' have effectively ceased to exist.

Memorization-plus-interpolation — the strategy a structure-blind model is implicitly betting on — requires neighbors, and in 784 dimensions there are none. Brute-force data cannot buy back what the architecture refuses to know.

6.5. The cure has a name

Return to the word planted on the very first pages of this book (Chapter 1): **inductive bias** — the nudge that steers a learner toward the functions we want, among the infinitely many that fit. Our MLP's failure is not a capacity problem or (fixably) a data problem. It is a *knowledge* problem: we knew things about images that we never told the model.

Say them out loud — the two principles from the lecture:

1. **Locality.** Nearby pixels are related; a pixel and its neighbor jointly form edges, textures, parts. Distant pixels, far less so. (The shuffle experiment proved our MLP holds no such belief.)
2. **Translation invariance.** A sleeve is a sleeve anywhere in the frame; *what* something is does not depend on *where* it is. (The shift experiment proved our MLP believes the opposite.)

Here is the move that powers the rest of this book: **build these beliefs into the architecture itself** — as constraints. Constrain each template to look at a small local patch instead of the whole frame. Then *share* that template across every position, so what it detects cannot depend on where. Two constraints, and both pathologies are attacked at their root — and as the lecture's arithmetic shows, the parameter count falls (135k for our little MLP; a stronger convolutional model will use half that) because sharing is a massive economy.

> **Constraints are knowledge**
>
> A constraint looks like a *reduction* in power: the constrained model is a strict subset of the MLP. That is exactly the point. By Chapter 1's bias–variance lens, we are spending bias — assumptions we are confident of — to collapse the search space onto functions that respect the structure of images, slashing variance and sample complexity. Inductive bias is not a limitation of a model; done right, it is everything you know about the world, cast in architecture. And it is a dial, not a dogma: give a weakly-biased model enough data and it can win again — a trade we will meet at the frontier in Chapter 16.

6.6. The pivot

Look once more at the templates figure. The fix is almost visible in it: the templates should not span the frame. They should be **small and local** — and they should **slide**, scoring every neighborhood of the image with the same little pattern, so that a sleeve found anywhere is the same sleeve.

A local template, slid across the image, scoring each position by dot product. That machine has a name, it predates deep learning by decades, and Part II opens by building it with our bare hands — then asking this book's favorite question about it:

What if the template were learnable?

6.7. Okay, so — the lesson of Part I

1. **Every metric can say yes while the model has learned the wrong thing.** Our MLP aced its test set while being indifferent to structure (shuffle: unchanged) and hypersensitive to nuisance (two pixels: halved).
2. **The autopsy is visual**: global templates, tuned to position, blind to adjacency.
3. **Capacity is not the culprit** — the U-curve barely exists here — and **data alone cannot pay the bill**: the curse of dimensionality (no neighbors at $d = 784$) sees to that.
4. **The cure is inductive bias**: locality and translation invariance, built in as constraints. Knowledge, cast in architecture.

Deeper dive

For the curious: how deep this rabbit hole goes

Networks can memorize anything. Zhang et al. famously trained standard deep networks to 100% *training* accuracy on ImageNet with completely random labels — capacity is so abundant that fitting noise is easy (Exercise 4 reproduces this on our subset). Generalization therefore cannot be explained by capacity limits; the explanation must involve the optimizer's implicit bias and the architecture's inductive bias.

The bias–variance picture, updated. Our flattened U-curve is a mild case of a broader phenomenon: with modern training, test error can even *descend again* beyond the interpolation point — "double descent." The cartoon survives as intuition for the underfit regime; the overparameterized regime plays by subtler rules.

Further reading

- Zhang, Bengio, Hardt, Recht, Vinyals, "Understanding deep learning requires rethinking generalization" (ICLR 2017) — the random-labels bombshell.
- Neal, "On the Bias-Variance Tradeoff: Textbooks Need an Update" (2020) — the source of this book's "cartoon, not a law" stance.
- Belkin et al., "Reconciling modern machine-learning practice and the classical bias–variance trade-off" (PNAS 2019) — double descent.
- 3Blue1Brown, *Gradient descent, how neural networks learn* — the visual companion to this chapter's pacing, including the memorable demonstration that a trained MLP confidently classifies pure noise.

Exercises

1. **(Pencil.)** Explain algebraically why the pixel shuffle costs the MLP nothing: if $\mathbf{P}$ is a permutation matrix and we train on $\mathbf{Px}$ instead of $\mathbf{x}$, exhibit the weight matrix that makes the two networks identical. Why does no analogous argument work for the shift?

2. **(Code.)** Augment the training set with random shifts of ±3 pixels and retrain. Re-plot the cliff. How much robustness did augmentation buy, what did it cost in clean accuracy and compute, and — the real question — what *knowledge* did augmentation encode, compared to building invariance into the architecture?
3. **(Code.)** The U-curve hunt used widths up to 256. Push to 1024 and 4096 at $n = 300$. Does the upturn ever arrive convincingly? Relate what you see to the optimizer's implicit regularization from Chapter 4.
4. **(Code.)** Randomly permute the training *labels* and train a wide MLP for long enough. Report train and test accuracy, and state in one sentence what this proves about capacity as an explanation of generalization.
5. **(Pencil.)** Estimate the first-layer parameter count of an MLP with 1,000 hidden units on an ImageNet-scale input (224×224×3). At one float32 per parameter, how much memory is that — and how does the number change if each unit instead sees only a 7×7×3 patch?

Part II.

II · Vision: Learning the Filters

7. Filters and Convolution (Fixed Kernels)

Part I ended with a diagnosis and a prescription. The diagnosis: our MLP's templates span the whole frame, so they are blind to adjacency and brittle to position (Chapter 6). The prescription, in one sentence: *make the template local, and slide it.*

This chapter builds that machine with our bare hands — no learning anywhere. It is a piece of classical engineering, decades older than deep learning, and computer-vision experts spent careers designing its templates manually. Understanding the machine in its fixed, hand-crafted form is the whole point of the chapter, because Part II's revolution (Chapter 8) will consist of exactly one change to it.

Recall the primitive from Chapter 1: a dot product is a similarity score against a template. Everything below is that one primitive, applied locally, everywhere.

7.1. Two principles, one strategy

The failures of Chapter 6 tell us what knowledge to build in — the two principles from the lecture:

1. **Spatial locality.** Nearby pixels are more related than distant ones. Edges, textures, and patterns emerge from *local neighborhoods*, not global pixel arrangements.
2. **Translation equivariance.** The same feature detector should work regardless of position — a cat's ear is a cat's ear whether it appears top-left or bottom-right.

The strategy they suggest: instead of one massive network staring at the entire image, use a *small* detector that analyzes one patch at a time, and **slide it across the image** to look for its feature everywhere. To understand what such detectors do, we first look at how vision experts used to design them by hand.

7.2. Start in 1D: the moving average

The core operation appears in its friendliest form when smoothing a noisy signal. A moving average replaces each point by the mean of its neighborhood — locality, with *uniform* weights:

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
```

```
torch.manual_seed(6050)
t = torch.linspace(0, 4 * torch.pi, 300)
noisy = torch.sin(t) + 0.35 * torch.randn(300)
kernel = torch.full((9,), 1 / 9)                          # uniform local weights
smooth = F.conv1d(noisy[None, None], kernel[None, None]).squeeze()

plt.figure(figsize=(6.4, 3))
plt.plot(t, noisy, color="#B8B8A8", lw=0.8, label="noisy signal")
plt.plot(t[4:-4], smooth, color="#E57200", lw=2, label="moving average (9-wide)")
plt.xlabel("$t$"); plt.legend()
plt.tight_layout(); plt.show()
```

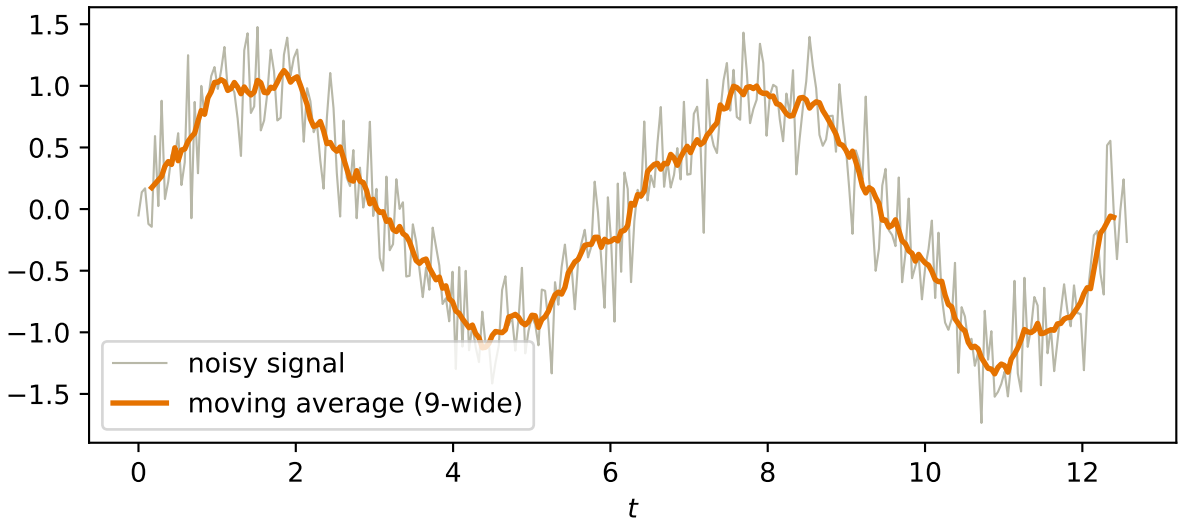

Figure 7.1.: A window of uniform weights, slid along a noisy signal, averaging as it goes: the simplest sliding-window operation, and already recognizably a denoiser.

7.3. To 2D: the recipe

The image version is the same idea with a small 2-D window, and it is worth stating as the five-step recipe from the lecture:

1. Define a small window of weights, the **kernel** (for a blur: all positive, summing to 1).
2. Place the kernel over a patch of the image.
3. Multiply elementwise — kernel weights against the pixels underneath.
4. Sum the products into a single output pixel. *(Steps 3–4 are one dot product: the patch, scored against a small template.)*
5. **Slide the window** to the next location and repeat, producing a new image.

Let us implement the recipe literally, and then check it against the framework's version — the build-then-verify habit of this book:

```python
def manual_conv2d(x: torch.Tensor, k: torch.Tensor) -> torch.Tensor:
    """The five-step recipe, verbatim: slide, multiply, sum."""
    kh, kw = k.shape
    H = torch.zeros(x.shape[0] - kh + 1, x.shape[1] - kw + 1)
    for i in range(H.shape[0]):
        for j in range(H.shape[1]):
            H[i, j] = (x[i:i + kh, j:j + kw] * k).sum()   # one dot product
    return H

def conv2d(x, k):                                          # the fast version
    return F.conv2d(x[None, None], k[None, None]).squeeze()

torch.manual_seed(6050)
x_test = torch.rand(10, 12)
k_test = torch.randn(3, 3)
gap = (manual_conv2d(x_test, k_test) - conv2d(x_test, k_test)).abs().max()
print(f"recipe vs. torch.conv2d: max |diff| = {gap:.1e}")
```

```
recipe vs. torch.conv2d: max |diff| = 4.8e-07
```

Also worth noticing before we move on: the output is smaller than the input. A $k \times k$ kernel on an $n \times n$ image produces $(n-k+1) \times (n-k+1)$ outputs, because the window must stay inside the frame. What to do about that (and about sliding in bigger steps) is Chapter 8's business.

7.4. The filter zoo

Here is the remarkable part: the recipe never changes — *only the numbers in the kernel do* — and different numbers produce completely different specialists. Three classics, applied to a synthetic test scene and to a boot from our Fashion-MNIST subset:

- **Blur**: uniform positive weights summing to 1 — the 2-D moving average.
- **Sharpen**: amplify each pixel's difference from its neighbors.
- **Edge detection** (Sobel): positive and negative weights summing to *zero* — over any flat region the products cancel and the output is silent; over an edge, a sharp change in values, the sum swings large. A detector that answers one question: *does intensity change here, in my direction?*

```python
def make_shapes(n: int = 64) -> torch.Tensor:
    img = torch.zeros(n, n)
    img[10:26, 8:30] = 0.9                                  # rectangle
    yy, xx = torch.meshgrid(torch.arange(n), torch.arange(n), indexing="ij")
    img[((yy - 44) ** 2 + (xx - 20) ** 2) < 100] = 0.6      # disk
    stripes = ((xx + yy) % 12 < 5).float() * 0.8
    img[8:56, 40:60] = stripes[8:56, 40:60]                 # diagonal stripes
```

```
    return img

test = torch.load("../../data/fashion-test.pt")
boot = test["X"][(test["y"] == 9).nonzero()[0, 0]].float() / 255.0

kernels = {
    "original": torch.tensor([[0., 0., 0.], [0., 1., 0.], [0., 0., 0.]]),
    "blur": torch.full((3, 3), 1 / 9),
    "sharpen": torch.tensor([[0., -1., 0.], [-1., 5., -1.], [0., -1., 0.]]),
    "Sobel (vert.)": torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]]),
    "Sobel (horiz.)": torch.tensor([[-1., -2., -1.], [0., 0., 0.], [1., 2., 1.]]),
}

fig, axes = plt.subplots(2, 5, figsize=(7.8, 3.4))
for row, image in enumerate([make_shapes(), boot]):
    for col, (name, k) in enumerate(kernels.items()):
        out = conv2d(image, k)
        axes[row, col].imshow(out, cmap="gray")
        axes[row, col].set_xticks([]); axes[row, col].set_yticks([])
        if row == 0:
            axes[row, col].set_title(name, fontsize=9)
plt.tight_layout(); plt.show()
```

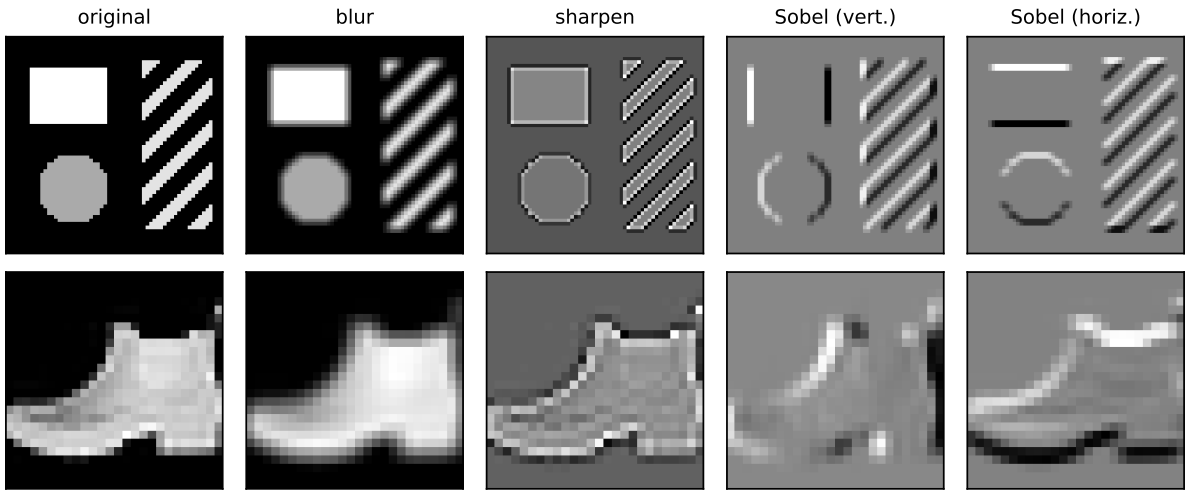

Figure 7.2.: One recipe, four specialists. Each column applies a different 3×3 kernel to the same inputs: identity, blur (local average), sharpen, and the two Sobel edge detectors — the vertical-edge kernel fires on vertical boundaries, the horizontal one on horizontal boundaries. The kernel's numbers are the entire difference.

Study the two Sobel columns for a moment. The vertical-edge detector lights up on the rectangle's sides and stays silent on its top and bottom; the horizontal detector does the reverse; the diagonal

stripes excite both. Nine numbers, and we have built a primitive *feature detector* — precisely the "small network analyzing one patch" of our strategy, except nobody has learned anything yet.

7.5. Naming the operation

This sliding sum-of-products has a proper name: **cross-correlation**. With kernel V of half-width Δ and image X:

$$[H]_{i,j} \;=\; \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [V]_{a,b}\, [X]_{i+a,\, j+b}. \tag{7.1}$$

> **i** "Convolution", with a small asterisk
>
> The deep learning community universally calls this operation a **convolution**, and so will we. A mathematician's convolution flips the kernel before sliding; since our kernels will soon be *learned* parameters, a flipped kernel is just another kernel, and the distinction is inconsequential in practice (Exercise 4 makes you confirm this).

7.6. The property we paid for: equivariance

Now collect the reward that Chapter 6's MLP could never have. Slide-and-score treats every position identically → if the input shifts, the output shifts *with it*, unchanged in content. That is **translation equivariance**, and it need not be taken on faith:

```
img = make_shapes()
sobel = kernels["Sobel (vert.)"]
s = 5

shifted = torch.zeros_like(img)
shifted[:, s:] = img[:, :-s]                  # input shifted right by 5

A = conv2d(shifted, sobel)                    # filter the shifted image
B = conv2d(img, sobel)                        # filter, THEN shift the result
B_shifted = torch.zeros_like(B)
B_shifted[:, s:] = B[:, :-s]

interior = (A - B_shifted)[:, s + 2:]         # compare away from the blank border
print(f"filter(shift(x)) vs shift(filter(x)): max |diff| = {interior.abs().max():.1f}")
```

```
filter(shift(x)) vs shift(filter(x)): max |diff| = 0.0
```

Exactly zero. The detector's report moves with the garment; nothing is *lost* by translation. Contrast Chapter 6: the MLP's global templates degraded under a 2-pixel shift because position was baked into every weight. Here, position is structurally incapable of mattering to *what* is detected — only to *where* the detection lands.

One precision worth keeping sharp, because the two words will both matter: **equivariant** means the output moves along with the input (what convolution gives us); **invariant** means the output does not change at all (what a classifier ultimately wants — "boot", regardless of position). Equivariance is the raw material; in Chapter 8, pooling will spend it to buy invariance.

7.7. The matrix view: a structured, weight-shared linear map

One more pair of glasses, from the lecture's preview box. Every output pixel is a dot product → the whole operation is *linear* → cross-correlation could be written as one enormous matrix multiplying the flattened image. But it is a matrix with a rigid structure: almost everywhere zero (locality: each row touches only one patch), and the same nine numbers repeating along its diagonals (sharing: every row is the same template, relocated).

That view makes the economics explicit. A dense layer from our 28×28 images to an equal-sized output would own $784 \times 784 \approx 615{,}000$ weights. The Sobel detector owns **nine** — reused at every position. This is Chapter 6's constraints-are-knowledge callout made concrete: locality zeroes out most of the matrix, equivariance ties the survivors together, and what remains is a linear map with almost nothing left to specify.

7.8. The bottleneck: someone has to design the kernel

So why did this classical machinery not solve vision decades ago? Because the kernels are only as good as their designers, and hand-crafted kernels have three limitations the lecture names:

1. **Limited expressiveness.** They detect simple, predefined patterns — gradients, blobs, corners. Nobody can write down nine numbers for "cat ear" or "car wheel."
2. **Manual feature engineering.** Choosing which kernels to use, and how to combine them, takes deep domain expertise and endless experimentation — for every new task.
3. **No adaptation.** A fixed kernel cannot adjust to a dataset. What works for one task fails for another.

An entire era of computer vision lived inside these constraints, stacking hand-designed detectors into elaborate pipelines. The machine was right; the templates were the bottleneck.

You can hear the question coming. We have a device that is local, weight-shared, and equivariant by construction — with a handful of numbers sitting inside it, currently chosen by hand. Nine numbers. We have spent four chapters building tools that tune numbers against a loss.

What if the template were learnable? That question is Chapter 8, and its answer has a name you already know.

7.9. Okay, so — the machine before the learning

1. **One operation, many specialists**: slide a small template, take a dot product at every position (Equation 7.1). Blur, sharpen, and edge detection differ only in the kernel's numbers.
2. **Locality and sharing are built in**: as a matrix, convolution is almost-all-zero with nine numbers repeating — 615,000 weights collapsed to 9.
3. **Equivariance is exact**, not approximate: filter-then-shift equals shift-then-filter, to the last bit. Invariance will be purchased from it in the next chapter.
4. **Hand-crafting is the bottleneck** — expressiveness, engineering cost, no adaptation — and it is exactly the kind of bottleneck this book knows how to break.

Exercises

1. **(Pencil.)** Compute by hand the full output of cross-correlating this 4×4 image with the vertical Sobel kernel: rows $(0,0,1,1)$, $(0,0,1,1)$, $(0,0,1,1)$, $(0,0,1,1)$. Before computing: where should the detector fire?
2. **(Pencil + code.)** Design a 3×3 kernel that detects *diagonal* edges (bottom-left to top-right). State your design rule (what must the weights sum to, and why?), then run it on `make_shapes()` and check it fires on the stripes.
3. **(Pencil.)** Prove translation equivariance from Equation 7.1: substitute the shifted image $X'_{i,j} = X_{i-s,j}$ and show $H'_{i,j} = H_{i-s,j}$. Why does the same argument fail for the MLP of Chapter 6?
4. **(Code.)** True convolution flips the kernel: run `conv2d` with `k` and with `k.flip(0, 1)` for the blur and for Sobel. Which outputs differ, which do not, and what property of the kernel decides it?
5. **(Code.)** The box blur is *separable*: show that convolving with the 3×3 uniform kernel equals convolving with a 3×1 column of $\frac{1}{3}$s and then a 1×3 row of $\frac{1}{3}$s. Count multiplications per output pixel for each route — this trick mattered enormously in the hand-crafted era.

8. CNNs: Making the Filters Learnable

Chapter 7 closed on a question it had spent a whole chapter earning: we built a machine that is local, weight-shared, and exactly equivariant — with nine hand-picked numbers sitting inside it. Nine numbers → nine knobs → we have spent half a book tuning knobs against a loss.

So: declare the kernel a **parameter**. That single change is this chapter, and it is the whole revolution. Everything else here — channels, padding, stride, pooling — is the supporting cast that turns one learnable filter into a working network. By the end we will have assembled LeNet, the first great convolutional architecture, trained it on our garments, and re-run the cruelest experiment of Chapter 6 to see what the inductive bias actually bought.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

torch.manual_seed(6050)
train = torch.load("../../data/fashion-train.pt")
test = torch.load("../../data/fashion-test.pt")
X_tr = train["X"].float().unsqueeze(1) / 255.0    # (1200, 1, 28, 28)
X_te = test["X"].float().unsqueeze(1) / 255.0     # (600, 1, 28, 28)
y_tr, y_te, classes = train["y"], test["y"], train["classes"]
```

Note the shapes. In Part I we flattened every image into a 784-vector before the model ever saw it. That stops now: the images stay rectangular, and they pick up a *channel* dimension, for reasons that will occupy us shortly.

> ⚠ Four dimensions, always
>
> PyTorch's convolution machinery expects every image batch in **NCHW** order: (batch, channels, height, width) — even a single grayscale image must be shaped `(1, 1, 28, 28)`. The kernels of a conv layer live in a 4-D tensor too: (out-channels, in-channels, height, width). This bookkeeping is the single most common source of errors in convolutional code: when a shape error greets you, count dimensions before touching anything else. Chapter 7's `conv2d` helper hid this with `x[None, None]`; from here on we handle it in the open.

8.1. Nine numbers, learned

Here is a game. I have a feature detector — one of Chapter 7's zoo, but I won't tell you which. I will only show you what it does: you hand it images, it hands back feature maps. Your job is to build a detector that matches it.

With the tools of Part I, this is not even hard. A kernel applied by Equation 7.1 is just nine numbers entering a dot product → the output is differentiable with respect to those numbers → gradient descent applies. Start from a random kernel and run the loop we have run since Chapter 1: predict → measure → step.

```
mystery = torch.tensor([[-1., 0., 1.],
                        [-2., 0., 2.],
                        [-1., 0., 1.]])            # (don't peek)
imgs = X_tr[:256]                                  # (256, 1, 28, 28)
with torch.no_grad():
    target = F.conv2d(imgs, mystery[None, None])   # what the detector does

torch.manual_seed(0)
K = 0.1 * torch.randn(1, 1, 3, 3)
K_init = K.clone().squeeze()
K.requires_grad_(True)

lr = 0.5
for step in range(601):
    pred = F.conv2d(imgs, K)                       # predict
    loss = F.mse_loss(pred, target)                # measure
    loss.backward()
    with torch.no_grad():                          # step
        K -= lr * K.grad
        K.grad.zero_()
    if step % 200 == 0:
        print(f"step {step:3d}   loss {loss.item():.2e}")

print(f"\nlearned kernel, rounded:\n{K.detach().squeeze().round(decimals=2)}")
print(f"max |learned - mystery| = {(K.detach().squeeze() - mystery).abs().max():.3f}")
```

```
step   0   loss 1.38e+00
step 200   loss 5.49e-04
step 400   loss 4.80e-05
step 600   loss 4.42e-06

learned kernel, rounded:
tensor([[-1.0100, -0.0000,  1.0100],
        [-1.9800, -0.0100,  1.9900],
        [-1.0100,  0.0100,  1.0000]])
max |learned - mystery| = 0.018
```

The loss falls more than five orders of magnitude, and the learned kernel is the vertical Sobel detector to within 0.02 in every entry. Gradient descent, given nothing but input–output pairs, has *reinvented* a filter that took computer-vision researchers years of squinting at image gradients to design.

```
fig, axes = plt.subplots(1, 3, figsize=(7, 2.6))
lim = 2.2
for ax, k, title in zip(axes, [K_init, K.detach().squeeze(), mystery],
                        ["random start", "learned", "Sobel (truth)"]):
    im = ax.imshow(k, cmap="RdBu_r", vmin=-lim, vmax=lim)
    ax.set_title(title, fontsize=10); ax.set_xticks([]); ax.set_yticks([])
fig.colorbar(im, ax=axes, shrink=0.8)
plt.show()
```

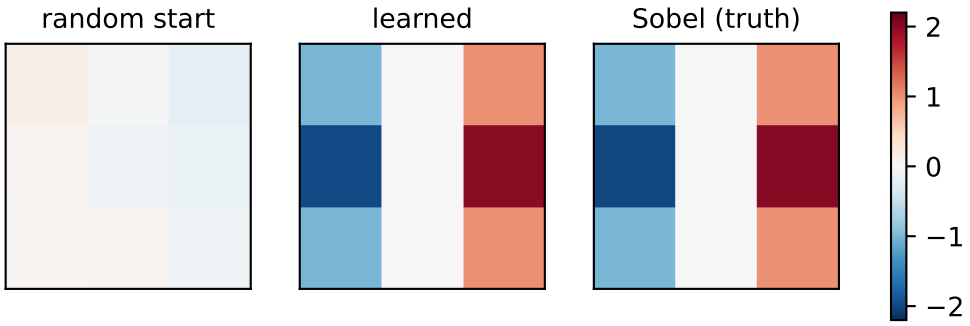

Figure 8.1.: The mystery-detector game. Left: the random starting kernel. Middle: the kernel gradient descent found from input–output pairs alone. Right: the hand-crafted vertical Sobel detector from Chapter 7's zoo. Nobody told the middle panel what an edge is.

The pivot of Part II

Read that cell again, because the entire deep-learning revolution in vision is that cell writ large. Chapter 7 ended with the bottleneck: hand-crafted kernels are limited to patterns a human can articulate in nine numbers — nobody can write down "cat ear." The fix is not better articulation. It is to stop articulating: put the numbers on the gradient tape and let the task pull the detector it needs out of the data. *What if the template were learnable?* It is, and the machinery has been in our hands since Chapter 4.

Two honest disclaimers about the game, both of which make the real thing *more* remarkable, not less. First, the game was rigged: the target came from a known kernel, so a perfect answer existed and the loss surface was a clean convex bowl (the output is linear in V, and squared error in a linear map is Chapter 1's setting all over again). Second, and more important: **in a real CNN, nobody supplies the target feature maps.** The only supervision is the classification loss at the far end of the network. Blame flows backward through every layer — exactly the mechanics of Chapter 5 — and the kernels that emerge are whatever detectors the task itself

demands. We chose Sobel here; a trained network might discover it needs something no one has named.

One backpropagation detail deserves a sentence, because it is the same fan-out rule we met in Chapter 5. The *same* nine numbers are used at every position of the slide → every position sends blame → the kernel's gradient is the **sum over all positions**. Weight sharing is not just a parameter economy; at training time it means every patch of every image is a lesson for the one shared template.

8.2. From one detector to a layer of detectives

One learnable kernel gives one feature map: one specialist, sweeping the image with one magnifying glass. But Chapter 7's zoo already told us one specialist is not enough — vertical edges, horizontal edges, blobs, textures all carry information. So we hire a *team*. The lecture's image is worth keeping: employ six detectives, each with a different expertise, each producing their own annotated map of the crime scene.

That is all the **channel** machinery is:

- **Out-channels: more detectives.** A conv layer with 6 output channels owns 6 independent kernels and produces a stack of 6 feature maps.
- **In-channels: detectives read the whole stack.** The *next* layer's input is that 6-deep stack, so each of its kernels grows a third dimension: a $6 \times 5 \times 5$ volume that looks at all six incoming maps *simultaneously* and sums the evidence into one output map, one bias per output channel at the end.

For an input stack X with C_{in} channels, output map o is

$$[H_o]_{i,j} \;=\; b_o \;+\; \sum_{c=1}^{C_{\text{in}}} \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [V_{o,c}]_{a,b}\, [X_c]_{i+a,\, j+b}, \tag{8.1}$$

which is Equation 7.1 run once per input channel and summed — followed, as always since Chapter 3, by a nonlinearity. A convolutional layer is Chapter 3's recipe with a structured linear map: dot products → add bias → ReLU. Nothing in the recipe changed; only the wiring of the linear part did.

```
conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, padding=2)
conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5)

x = X_tr[:1]                                    # (1, 1, 28, 28): one garment
h1 = F.relu(conv1(x))
h2 = F.relu(conv2(h1))
print(f"input   {tuple(x.shape)}")
print(f"conv1   {tuple(h1.shape)}   kernels: {tuple(conv1.weight.shape)}")
print(f"conv2   {tuple(h2.shape)}  kernels: {tuple(conv2.weight.shape)}")
```

```
input   (1, 1, 28, 28)
conv1   (1, 6, 28, 28)   kernels: (6, 1, 5, 5)
conv2   (1, 16, 24, 24)  kernels: (16, 6, 5, 5)
```

Look at `conv2.weight`: sixteen kernels, each $6 \times 5 \times 5$. This is where features start *composing*. Suppose detective 3 in the first layer is a vertical-edge expert and detective 5 hunts horizontal edges. A second-layer kernel that weights both maps positively in the same neighborhood fires precisely when both experts agree → a **corner detector**, built from evidence no single first-layer kernel could see. The cross-talk between channels is how edges become corners, corners become textures, and textures become parts. We promised this compositional ladder in Chapter 3; here is the hardware it runs on.

Chapter 7's zoo can stage the story before any learning enters. Give the first layer two hand-crafted experts — vertical and horizontal edge energy — then hand-build one second-layer kernel that reads both reports at once:

```
square = torch.zeros(28, 28)
square[7:21, 7:21] = 1.0
sobel_v = torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]])
v = F.conv2d(square[None, None], sobel_v[None, None], padding=1).abs()
h = F.conv2d(square[None, None], sobel_v.T.contiguous()[None, None], padding=1).abs()

experts = torch.cat([v, h], dim=1)              # (1, 2, 28, 28): two reports
corner_k = torch.full((1, 2, 5, 5), 1 / 50)     # one kernel, spanning both
corners = F.conv2d(experts, corner_k, padding=2).squeeze()

fig, axes = plt.subplots(1, 4, figsize=(9, 2.5))
panels = [(square, "input"), (v.squeeze(), "vertical-edge expert"),
          (h.squeeze(), "horizontal-edge expert"), (corners, "their reader: corners")]
for ax, (t, title) in zip(axes, panels):
    ax.imshow(t, cmap="gray"); ax.set_title(title, fontsize=9); ax.axis("off")
plt.tight_layout(); plt.show()
```

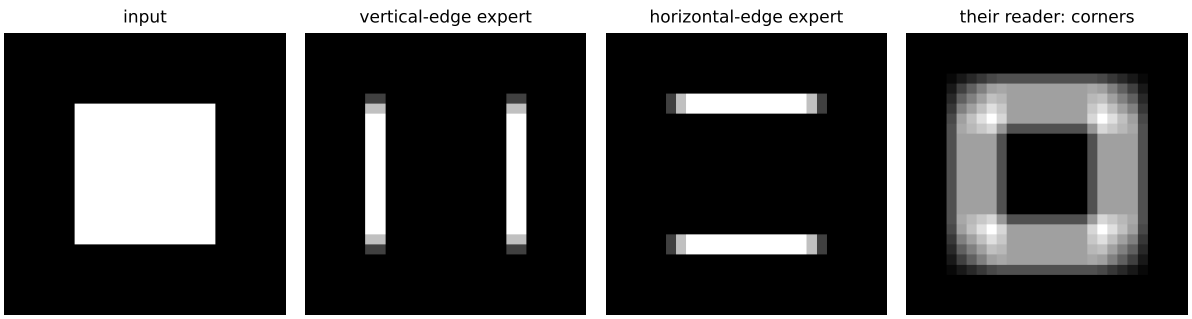

Figure 8.2.: Cross-talk staged by hand. Two first-layer experts report edge energy (absolute Sobel response) on a square. One second-layer kernel spanning both channels (all-positive weights; Equation 8.1 with two input channels) fires hardest exactly where both experts agree — the four corners, a feature neither channel could see alone.

The four brightest cells of the right panel are exactly the square's four corners: a corner detector, assembled from experts that individually know nothing about corners. In a trained CNN, gradient descent builds combinations like this — and far stranger ones — on its own.

> **`nn` versus `F`, appliances versus recipes**
>
> The lecture's analogy: `nn` modules are *appliances* — they have knobs and memory, and PyTorch carries their state around for you (`nn.Conv2d` owns its kernels and biases as `nn.Parameter`s, registered for autograd and visible to the optimizer). `F` functions are *recipes* — stateless, everything passed in by hand. In Chapter 7 the kernel was ours, so `F.conv2d` was the honest choice; now the kernel is the layer's business, so `nn.Conv2d` is. Use appliances for anything with parameters, recipes for everything else (`F.relu`, `F.max_pool2d`).

8.3. Padding and stride: the bookkeeping with a payoff

Chapter 7 left an administrative debt: a $k \times k$ kernel on an $n \times n$ image yields only $(n-k+1) \times (n-k+1)$ outputs, because the window must stay inside the frame. For one layer, a nuisance. For a *deep* stack, fatal: $28 \to 24 \to 20 \to \cdots$ and the feature map shrinks toward nothing, with edge pixels contributing to ever fewer windows along the way.

The fix is blunt: **padding**. Add a border of zeros around the input so the window can center itself on every real pixel. For a 5×5 kernel, a 2-pixel border returns exactly the input size $\to$ that is why `conv1` above says `padding=2`. Padding decouples "how deep can the network be" from "how fast do the maps shrink," and that decoupling is what makes deep stacks possible at all.

Sometimes, though, we *want* to shrink — coarser maps mean less computation and a wider view per pixel. Then we shrink on purpose with **stride**: slide the window s pixels at a time instead of one. Both knobs live in one formula, worth memorizing because you will run it in your head for every architecture you ever read:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1. \tag{8.2}$$

The numerator is the padded length minus the window, i.e. how far the window can travel; dividing by the step and flooring counts the stops; $+1$ counts the starting position. The lecture's three worked cases, verified:

```
x8 = torch.randn(1, 1, 8, 8)
for p, s in [(0, 1), (1, 1), (1, 2)]:
    out = F.conv2d(x8, torch.randn(1, 1, 3, 3), padding=p, stride=s)
    n_out = (8 + 2 * p - 3) // s + 1
    print(f"n=8, k=3, p={p}, s={s}:  formula {n_out}  torch {tuple(out.shape[2:])}")
```

```
n=8, k=3, p=0, s=1:  formula 6  torch (6, 6)
n=8, k=3, p=1, s=1:  formula 8  torch (8, 8)
n=8, k=3, p=1, s=2:  formula 4  torch (4, 4)
```

8.4. Pooling: spending equivariance to buy invariance

Now for the deepest design decision in the network, and it starts from the asset Chapter 7 banked. Convolution is exactly equivariant: shift the garment, and every feature map shifts in lockstep. The detective's map of clues faithfully tracks the scene. But the network's final job is not mapping clues — it is a verdict. *Is this a trouser?* The answer must not depend on where the trouser stands. The feature-finding stage wants **equivariance** (where things are matters); the decision stage wants **invariance** (only what was found matters). Equivariance is what we have; invariance is what the classifier head needs → something must convert one into the other.

That converter is **pooling**. Slide a window over each feature map (no parameters this time) and summarize each neighborhood by one number:

- **Max pooling** asks: *what is the strongest clue here?* Keep the loudest activation, discard the rest.
- **Average pooling** asks: *what is the general vibe of this neighborhood?* Smoother, but a single sharp clue gets diluted by quiet neighbors.

```
t = torch.arange(16.).reshape(1, 1, 4, 4)
print(f"input:\n{t.squeeze()}")
print(f"max pool 2x2:\n{F.max_pool2d(t, 2).squeeze()}")
print(f"avg pool 2x2:\n{F.avg_pool2d(t, 2).squeeze()}")
```

```
input:
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
max pool 2x2:
tensor([[ 5.,  7.],
        [13., 15.]])
avg pool 2x2:
tensor([[ 2.5000,  4.5000],
        [10.5000, 12.5000]])
```

The common configuration is the one above: 2×2 windows with stride 2, non-overlapping, halving each spatial dimension. Max is the default choice in practice, precisely because feature detection is about the strongest evidence, not the committee average.

Why does this buy invariance? Watch a shift become invisible:

```
scene = torch.zeros(1, 1, 4, 4)
scene[0, 0, 1, 0] = 9.0        # a strong clue, upper-left region
scene[0, 0, 2, 2] = 3.0        # a weaker clue, lower-right region

shifted = torch.zeros_like(scene)
shifted[0, 0, 1, 1] = 9.0      # both clues moved one pixel right
shifted[0, 0, 2, 3] = 3.0

print(f"pooled original: {F.max_pool2d(scene,   2).squeeze().tolist()}")
print(f"pooled shifted:  {F.max_pool2d(shifted, 2).squeeze().tolist()}")
```

```
pooled original: [[9.0, 0.0], [0.0, 3.0]]
pooled shifted:  [[9.0, 0.0], [0.0, 3.0]]
```

Both clues moved, yet the pooled maps are *identical*: each clue stayed inside its 2×2 window, and max pooling reports what it found, not where in the window it found it. The invariance is real but local — a shift that crosses a window boundary does change the output. One pooling stage buys tolerance to jitter of a pixel or so; stacking a second stage on the already-halved map compounds the tolerance. How far that compounding actually gets us is an empirical question, and the end of this chapter measures it.

> ⚠ Invariance has a price tag
>
> Pooling buys invariance by *throwing away position*. After two rounds, the network knows a strong vertical edge exists in a region; it no longer knows exactly where. For classification this is the right trade — but it is a trade, and it will come due twice in this book. Deeper in Part II, tasks that need precise locations will have to be more careful with resolution. And in Part IV the tables turn completely: self-attention (1) is so thoroughly position-agnostic that we will have to *pay to put position back in*. Remember, when we get there, that position was something we once discarded on purpose.

8.5. How far a deep neuron sees

There is one more consequence of stacking, and it quietly resolves the tension Chapter 6 left us with. Every kernel here is tiny — 5×5, resolutely local. Yet recognizing a garment is a *global* judgment. Where does globality come from?

From depth. A neuron in the first feature map sees a 5×5 patch of the image: its **receptive field**. A neuron one layer up sees a 5×5 patch *of feature maps*, each pixel of which already summarizes a patch below → its receptive field on the original image is wider. Pooling accelerates this: after a 2×2/stride-2 pool, neighboring pixels in the pooled map stand two original pixels apart, so every later window covers twice the ground. For our upcoming network the ledger reads: conv1 sees 5×5 → pooling nudges it to 6×6 → conv2's 5×5 window, at stride-2 spacing, expands it to 14×14 → the final pool reaches 16×16 — over half the frame in every direction, from nothing but 5×5 looks.

```
import matplotlib.patches as mpatches

fig, ax = plt.subplots(figsize=(4.2, 3.6))
ax.imshow(X_te[1, 0], cmap="gray")
for size, color, lab in [(5, "#E57200", "conv1: 5×5"),
                         (14, "#5379AA", "conv2: 14×14"),
                         (16, "#DDA0DD", "pool2: 16×16")]:
    lo = 14 - size / 2
    ax.add_patch(mpatches.Rectangle((lo, lo), size, size, fill=False,
                                    lw=2, edgecolor=color, label=lab))
ax.legend(fontsize=8, loc="center left", bbox_to_anchor=(1.02, 0.5))
ax.axis("off")
plt.tight_layout(); plt.show()
```

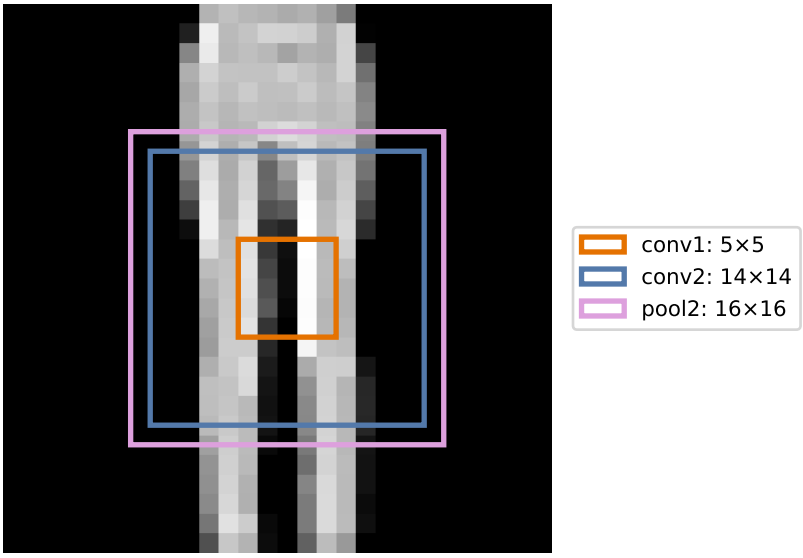

Figure 8.3.: What one neuron sees, by depth. Boxes mark the receptive field of a single unit in each stage of our network, centered on the same spot: conv1 sees a 5×5 patch, conv2 sees 14×14, the final pooled cell 16×16. Local wiring, increasingly global sight; the dense head then reads all 25 such overlapping views at once.

This is the compositional hierarchy of Chapter 3 finally given a reason to exist. The MLP *could* represent features-of-features but nothing encouraged it to; here the architecture leaves no alternative. Early layers, small fields, simple patterns: edges, gradients. Middle layers, wider fields, compositions: corners, textures. Late layers, fields spanning the object: parts. The CNN buys global sight gradually, paying for it with depth; hold that thought until Part IV, where attention will buy global sight in a single step and pay for it differently (Chapter 13).

8.6. LeNet: the whole machine

Time to assemble. The canonical blueprint is **LeNet**, Yann LeCun's digit-reading network, and its rhythm is the CNN design pattern to this day: *convolve, shrink, deepen; repeat; then decide.* Spatial size falls ($28 \to 14 \to 5$) while feature depth grows ($1 \to 6 \to 16$): the network trades "where" for "what" stage by stage, until a small dense head reads the final $16 \times 5 \times 5$ summary and votes.

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, padding=2)   # (1,28,28) -> (6,28,28)
        self.conv2 = nn.Conv2d(6, 16, 5)             # (6,14,14) -> (16,10,10)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = F.max_pool2d(F.relu(self.conv1(x)), 2)   # -> (6,14,14)
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)   # -> (16,5,5)
        x = x.flatten(1)                             # -> (400,)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)                           # logits, (10,)
```

Every line is a tool we already own: Equation 8.1 twice, Equation 8.2 silently fixing `padding=2`, pooling twice, then Chapter 3's MLP as the decision head — and, per Chapter 2's advice, the model returns raw logits and lets `F.cross_entropy` handle the probabilities.

> **i** LeNet, then and now
>
> The LeNet family (LeCun and colleagues, 1989–1998) began by reading handwritten ZIP codes for the US Postal Service, and its mature form, LeNet-5, went on to read a substantial share of America's bank checks — convolutional networks were literally cashing checks decades before they were fashionable. Our variant keeps the classic skeleton but upgrades the internals with Part I's verdicts: ReLU where the original used sigmoid-family activations (Chapter 3, Chapter 5), max pooling where it used average-flavored subsampling, Adam as the optimizer. The lecture's live session adds one more modern stabilizer, batch normalization, which we meet properly in Chapter 9.

Before training, the parameter audit — because parameter *economy* was half of Chapter 7's sales pitch, and Chapter 6's MLP is the yardstick:

```
def make_mlp():
    return nn.Sequential(nn.Flatten(),
                         nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
```

```
lenet, mlp = LeNet(), make_mlp()
for name, p in lenet.named_parameters():
    print(f"{name:14s} {str(tuple(p.shape)):16s} {p.numel():>7,}")
n_conv = sum(p.numel() for n, p in lenet.named_parameters() if "conv" in n)
n_lenet = sum(p.numel() for p in lenet.parameters())
n_mlp = sum(p.numel() for p in mlp.parameters())
print(f"\nLeNet: {n_lenet:,} parameters ({n_conv:,} of them convolutional)")
print(f"MLP:   {n_mlp:,} parameters")
```

```
conv1.weight   (6, 1, 5, 5)         150
conv1.bias     (6,)                   6
conv2.weight   (16, 6, 5, 5)      2,400
conv2.bias     (16,)                 16
fc1.weight     (120, 400)        48,000
fc1.bias       (120,)               120
fc2.weight     (84, 120)         10,080
fc2.bias       (84,)                 84
fc3.weight     (10, 84)             840
fc3.bias       (10,)                 10

LeNet: 61,706 parameters (2,572 of them convolutional)
MLP:   203,530 parameters
```

Two readings of this table. Against the MLP: LeNet carries 61,706 parameters to the MLP's 203,530 (less than a third) while performing far richer spatial computation. Weight sharing is doing exactly what Chapter 7's matrix view promised. Within LeNet: look where the parameters live. The two conv layers, the part that actually *sees*, own 2,572 parameters (about 4%) while the dense head hoards the rest. That imbalance is a design smell we will fix in Chapter 9, where modern architectures go deeper in convolution and lighter in the head.

Now train both models under one shared recipe (same data, same optimizer, same budget) → the only difference left is architecture. This time we use *minibatches* (Chapter 4's stochastic regime, batch size 128) rather than Chapter 6's full-batch loop, since the conv net needs more steps to converge:

```
def train_model(model_fn, epochs: int = 150, batch: int = 128, seed: int = 6050):
    torch.manual_seed(seed)
    model = model_fn()
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    n = len(X_tr)
    for _ in range(epochs):
        perm = torch.randperm(n)
        for i in range(0, n, batch):
            idx = perm[i:i + batch]
            loss = F.cross_entropy(model(X_tr[idx]), y_tr[idx])
```

```
            opt.zero_grad(); loss.backward(); opt.step()
    return model

@torch.no_grad()
def accuracy(model, X, y):
    return (model(X).argmax(1) == y).float().mean().item()

mlp = train_model(make_mlp)
lenet = train_model(LeNet)
for name, model in [("MLP  ", mlp), ("LeNet", lenet)]:
    print(f"{name}  train {accuracy(model, X_tr, y_tr):.1%}   "
          f"test {accuracy(model, X_te, y_te):.1%}")
```

```
MLP    train 100.0%   test 82.2%
LeNet  train 99.7%   test 82.5%
```

Read the clean-test column honestly: a near-tie. LeNet edges ahead, but within seed noise — this is *not* the landslide the architecture's press clippings promise. It shouldn't be, and Chapter 6 already told us why: on 1,200 centered garments, capacity was never the bottleneck. The MLP has parameters to spare and memorizes its way to the same score (both models sit at essentially 100% train accuracy). On the full 60,000-image dataset the gap opens decisively — the lecture's live session watches the CNN pull ahead early and stay ahead — but our small-data regime makes a sharper point: LeNet *matches* the MLP with a third of the parameters, and, as we are about to see, what it learned is a different kind of knowledge altogether.

8.7. The rematch

Chapter 6 convicted our MLP with one experiment: shift every test garment two pixels right, and accuracy fell off a cliff — the full-frame templates kept scoring sleeves against background. We then promised that an architecture built on locality and weight sharing would face the same trial. The rematch:

```
def shift_right(X, px: int):
    out = torch.zeros_like(X)
    if px == 0:
        return X.clone()
    out[..., px:] = X[..., :-px]
    return out

shifts = list(range(5))
acc_mlp = [accuracy(mlp, shift_right(X_te, s), y_te) for s in shifts]
acc_cnn = [accuracy(lenet, shift_right(X_te, s), y_te) for s in shifts]
for s in shifts:
    print(f"shift {s}px:   MLP {acc_mlp[s]:.1%}   LeNet {acc_cnn[s]:.1%}")
```

```
plt.figure(figsize=(5.5, 3.2))
plt.plot(shifts, acc_mlp, "o-", color="#E57200", lw=2, ms=7, label="MLP (Ch. 6)")
plt.plot(shifts, acc_cnn, "s-", color="#232D4B", lw=2, ms=7, label="LeNet")
plt.axhline(0.1, ls=":", color="#B8B8A8")
plt.xlabel("shift (pixels right)"); plt.ylabel("test accuracy")
plt.ylim(0, 0.9); plt.xticks(shifts); plt.legend()
plt.tight_layout(); plt.show()
```

```
shift 0px:   MLP 82.2%   LeNet 82.5%
shift 1px:   MLP 67.2%   LeNet 74.8%
shift 2px:   MLP 41.8%   LeNet 62.3%
shift 3px:   MLP 29.8%   LeNet 45.3%
shift 4px:   MLP 19.8%   LeNet 26.8%
```

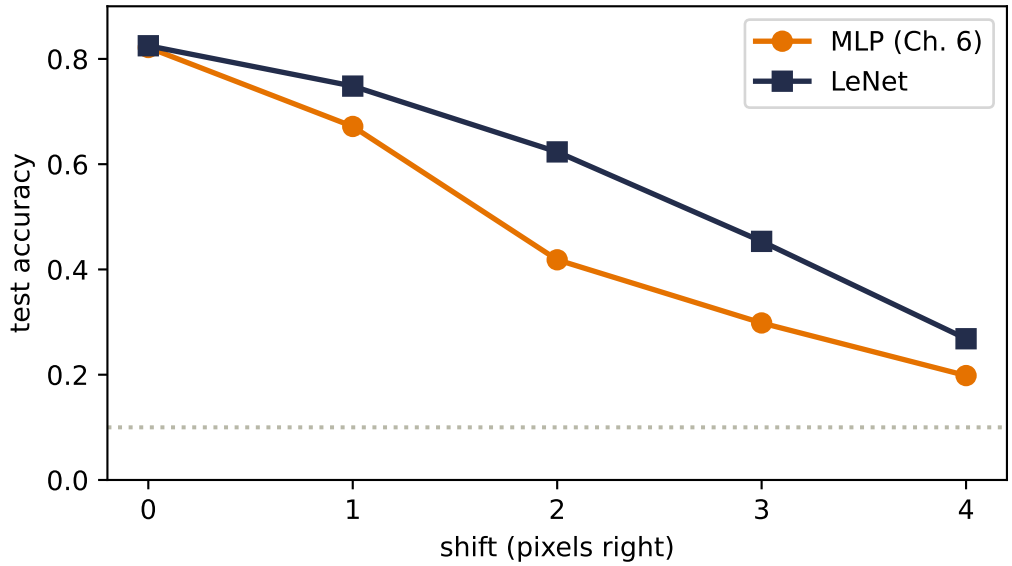

Figure 8.4.: The rematch. Chapter 6's experiment, run on both architectures trained under the identical recipe. The MLP repeats its cliff: half its accuracy gone by two pixels. LeNet degrades gracefully — the same two-pixel shift costs it a quarter. The dotted line is chance.

There is the payoff, and it is worth stating precisely. At two pixels the MLP is down to 41.8% — the same collapse as Chapter 6, reproduced under the new recipe — while LeNet holds 62.3%. At one pixel, LeNet gives up half as much as the MLP does (7.7 points against 15). The mechanism is everything this chapter built: the kernels travel with the garment (equivariance), so the *features are still found*; pooling forgives the sub-window jitter (local invariance); only the coarse layout entering the head changes at all.

And yet the navy curve falls too. Also worth stating precisely, because it teaches the architecture's fine print. Two rounds of 2× pooling buy tolerance measured in a few pixels, not unlimited; and

after `flatten(1)`, the dense head reads the final 5×5 grid *positionally* — a four-pixel input shift moves features roughly one full cell in that grid, and the head is as brittle to cell-level shifts as Chapter 6's MLP was to pixel-level ones. The inductive bias did not abolish the cliff; it moved it outward and flattened it into a slope. Getting rid of the rest is a solved problem, and the solutions are next chapter's business (going deeper; pooling *globally* — and exercise 5 lets you preview the idea today).

One last look inside, because Chapter 6 also showed us what the MLP's first layer *looked like* — global, smeared, garment-shaped templates (Chapter 6) — and promised that structure would change:

```
img = X_te[1:2]                                    # the trouser from earlier
with torch.no_grad():
    fmaps = F.relu(lenet.conv1(img)).squeeze(0)    # (6, 28, 28)
kernels = lenet.conv1.weight.detach().squeeze(1)   # (6, 5, 5)

fig, axes = plt.subplots(2, 6, figsize=(9, 3.2))
lim = kernels.abs().max()
for i in range(6):
    axes[0, i].imshow(kernels[i], cmap="RdBu_r", vmin=-lim, vmax=lim)
    axes[1, i].imshow(fmaps[i], cmap="gray")
    axes[0, i].axis("off"); axes[1, i].axis("off")
plt.tight_layout(); plt.show()
```

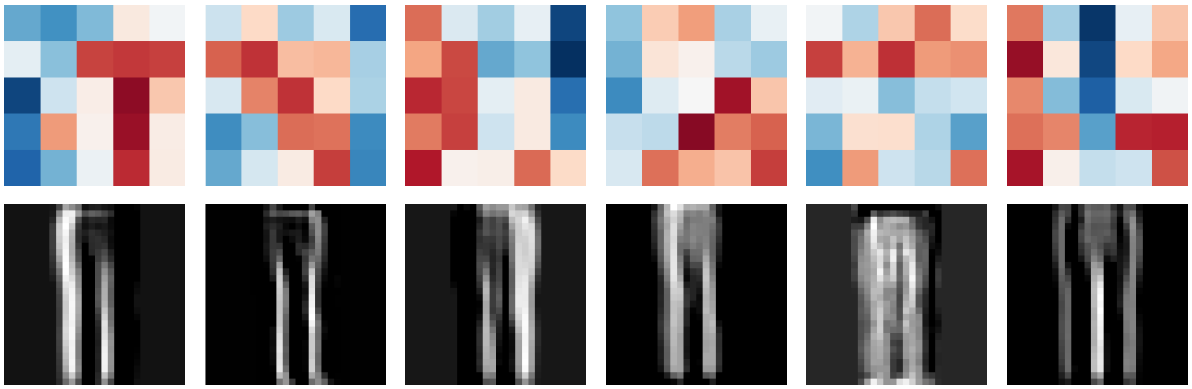

Figure 8.5.: What LeNet learned to look for. Top: the six first-layer 5×5 kernels (red positive, blue negative). Bottom: each kernel's feature map on a test trouser — different detectives report left leg edges, right edges, interior fill. Trained on only 1,200 images the kernels are grainier than their full-dataset cousins, but the division of labor is unmistakable. Compare Chapter 6's full-frame templates: same job, opposite philosophy.

Six small, local, oriented detectors, each lighting up a different aspect of the same trouser. Nobody designed them. The classification loss, pulling blame backward through Equation 8.1 for fifteen hundred gradient steps, decided that *this* is what the first layer of garment-reading should look for.

8.8. Okay, so —

1. **One change made the revolution**: the kernel's numbers became parameters. Gradient descent can recover a hand-crafted detector from data (our rigged game) and, more importantly, discover detectors nobody designed, supervised only by the task loss at the far end.
2. **Channels turn one detector into a team**: out-channels stack specialists' feature maps; in-channels let the next layer read all maps at once (Equation 8.1), which is how edges compose into corners and parts.
3. **Padding and stride are the shape budget**: $n_{\text{out}} = \lfloor (n + 2p - k)/s \rfloor + 1$ (Equation 8.2). Padding lets networks go deep without shrinking to nothing; stride shrinks on purpose.
4. **Pooling converts equivariance into (local) invariance** — the strongest clue per window, position discarded. It is a purchase, and position is the currency.
5. **Receptive fields grow with depth**: $5 \to 6 \to 14 \to 16$ in our LeNet. Local wiring earns global sight gradually; the hierarchy Chapter 3 could only hope for is here enforced by architecture.
6. **The verdict**: on our small dataset LeNet ties the MLP's clean accuracy with a third of the parameters — and where the MLP cliff-dives under a two-pixel shift (41.8%), LeNet degrades gracefully (62.3%). The bias did not add capacity; it added the *right constraints*, exactly Chapter 6's prescription.

Exercises

1. **(Pencil.)** Run Equation 8.2 through LeNet layer by layer, starting from 28×28: verify every shape comment in the `LeNet` class, then verify the parameter counts of `conv1` and `conv2` from the table (weights *and* biases).
2. **(Pencil.)** Verify the receptive-field ledger $5 \to 6 \to 14 \to 16$. (Track two numbers per stage: the field size, and the *jump* — the spacing in input pixels between neighboring units — which each stride-2 pool doubles.) Then show that no convolutional or pooling unit in LeNet ever sees the full 28×28 frame. Where does globality finally enter the network?
3. **(Code.)** Play the mystery-detector game with the 3×3 blur kernel as the target. Does gradient descent recover it as cleanly as Sobel? Now rerun with only 8 training images instead of 256. What happens to the recovered kernel, and what does that say about data volume versus parameter identifiability?
4. **(Code.)** Swap LeNet's max pooling for average pooling and retrain. Compare clean accuracy and the shift curve. Which changes more, and does the result match the "strongest clue vs. general vibe" story?
5. **(Code.)** Make the head positionless: replace `flatten` and the three dense layers with a *global* average pool over each of the 16 final feature maps, feeding `nn.Linear(16, 10)`. Retrain and re-plot the shift curve. What happened to the cliff — and what did the clean accuracy pay for it? Explain both effects in terms of what the head can no longer see.

9. Modern CNNs and Transfer Learning

Chapter 8 ended with a working machine and a report card. LeNet reads garments at 82.5% with 61,706 parameters; graceful under shift where the MLP cliff-dived. But the card has two demerits we wrote down explicitly: 96% of those parameters sit in the dense head rather than in the convolutions that do the seeing, and the shift cliff was softened, not abolished. And one IOU: the live session's batch normalization, promised for this chapter.

What happened after LeNet is a decade of architecture research, and it can drown you in named networks. The lecture compresses it into **four design questions**, and we will let them drive:

1. **Why 5×5?** Could smaller filters do the same job with fewer parameters?
2. **How do we go deeper** without gradients dying on the way down?
3. **Where do the parameters actually live**, and how do we evict them?
4. **Can we design reusable blocks** — optimized layer-combinations we stack, keeping a bird's-eye view instead of re-designing what already works?

Each question has a famous answer (VGG, ResNet, NiN's 1×1 + global pooling, and the block habit itself), and each answer is a constraint-shaped idea we can test on our own data. At the end, the practical superpower the lecture builds to — reusing a pretrained backbone — gets the most honest experiment in this book.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

torch.manual_seed(6050)
train = torch.load("../../data/fashion-train.pt")
test = torch.load("../../data/fashion-test.pt")
X_tr = train["X"].float().unsqueeze(1) / 255.0    # (1200, 1, 28, 28)
X_te = test["X"].float().unsqueeze(1) / 255.0     # (600, 1, 28, 28)
y_tr, y_te, classes = train["y"], test["y"], train["classes"]

def train_model(model_fn, epochs: int, batch: int = 128, seed: int = 6050,
                lr: float = 1e-3, data=None):
    torch.manual_seed(seed)
    model = model_fn()
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    X, y = data if data is not None else (X_tr, y_tr)
    n = len(X)
    for _ in range(epochs):
```

```
        perm = torch.randperm(n)
        for i in range(0, n, batch):
            idx = perm[i:i + batch]
            model.train()
            loss = F.cross_entropy(model(X[idx]), y[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    return model

@torch.no_grad()
def accuracy(model, X, y):
    model.eval()
    return (model(X).argmax(1) == y).float().mean().item()

count = lambda m: sum(p.numel() for p in m.parameters())
```

Two small novelties in the recipe, both because this chapter's networks contain batch normalization: `model.train()` before each step and `model.eval()` before each evaluation. Why that matters is the subject of the second section.

9.1. Question 1: why 5×5? Stack small kernels instead

VGG's answer (Simonyan & Zisserman, 2014) is the cleanest idea in this chapter. Recall the receptive-field ledger of Chapter 8: stacking grows the field. Two stacked 3×3 convolutions let the second layer see 5×5 of the input — the *same* receptive field as one 5×5 kernel. But compare the bills, for a layer with C channels in and C out:

$$\underbrace{25\,C^2}_{\text{one } 5\times5} \qquad \text{versus} \qquad \underbrace{2 \times 9\,C^2 = 18\,C^2}_{\text{two } 3\times3\text{s}},$$

a 28% saving — and the stacked pair fires a ReLU *twice* where the big kernel fires once. Same sight, fewer parameters, more nonlinearity. Three 3×3s reach a 7×7 field for $27C^2$ against $49C^2$: the deeper you take the idea, the better the deal gets.

```
C = 32
one_5x5 = nn.Conv2d(C, C, 5, padding=2)
two_3x3 = nn.Sequential(nn.Conv2d(C, C, 3, padding=1), nn.ReLU(),
                        nn.Conv2d(C, C, 3, padding=1))
print(f"one 5x5:  {count(one_5x5):,} parameters, 1 ReLU after")
print(f"two 3x3s: {count(two_3x3):,} parameters, 2 ReLUs")
```

```
one 5x5:  25,632 parameters, 1 ReLU after
two 3x3s: 18,496 parameters, 2 ReLUs
```

One honest caveat before you re-derive the field equations of vision from this: the $18C^2 < 25C^2$ arithmetic assumes the channel width stays C through the stack. When a layer *grows* channels (as LeNet's $6 \to 16$ did), splitting it into two growing 3×3s can cost more, not less. VGG's design sidesteps this by keeping width constant inside each block and changing it only between blocks — which is exactly the shape our code will take. (Exercise 1 makes you find the break-even point.)

9.2. The stabilizer we owe you: batch normalization

Before we stack deeper than ever, we need the tool Chapter 8's live session used and we deferred. Here is the problem it solves. Watch what happens to the *scale* of activations as a signal crosses twelve freshly initialized 3×3 conv layers:

```
def stack12(with_bn: bool, ch: int = 16):
    layers = [nn.Conv2d(1, ch, 3, padding=1)]
    for _ in range(11):
        if with_bn:
            layers.append(nn.BatchNorm2d(ch))
        layers += [nn.ReLU(), nn.Conv2d(ch, ch, 3, padding=1)]
    return nn.Sequential(*layers)

torch.manual_seed(0)
x = X_tr[:256]
for with_bn in [False, True]:
    h, stds = x, []
    with torch.no_grad():
        for layer in stack12(with_bn):
            h = layer(h)
            if isinstance(layer, nn.Conv2d):
                stds.append(h.std().item())
    tag = "with BN   " if with_bn else "without BN"
    print(f"{tag} layer stds: " + "  ".join(f"{s:.3f}" for s in stds[::2]))
```

```
without BN layer stds: 0.344  0.059  0.043  0.046  0.049  0.061
with BN    layer stds: 0.309  0.324  0.411  0.383  0.377  0.369
```

Without normalization the signal's spread collapses by an order of magnitude within a few layers and keeps sagging; recall from Chapter 5 that whatever happens to signals forward happens to gradients backward → layers deep in the stack learn at a completely different rate than shallow ones, if at all.

Batch normalization (Ioffe & Szegedy, 2015) re-standardizes at every layer: for each channel, compute the mean and standard deviation *across the current minibatch*, normalize the activations to zero mean and unit spread, then let the layer undo it if it wants to via two learnable knobs per channel:

$$\hat{x} = \frac{x - \mu_{\text{batch}}}{\sqrt{\sigma^2_{\text{batch}} + \epsilon}}, \qquad y = \gamma\,\hat{x} + \beta. \tag{9.1}$$

The γ, β pair matters: normalization is a *reset to a healthy scale*, not a straitjacket — the network can learn to restore any mean and spread that helps, but it starts every layer from sane numbers. In the printout above, the BN column holds near a constant spread through all twelve layers: every layer trains from day one. The standard placement, which the live session uses and we adopt, is conv → BN → ReLU, with `bias=False` on the convolution, since β already provides the shift.

> ⚠ BN is two different machines — tell PyTorch which one you're running
>
> At training time BN normalizes by the *current batch's* statistics. At evaluation time there may be no batch (one image!), so it uses running averages collected during training. `model.train()` and `model.eval()` switch between the two. Forgetting the switch is the classic BN bug: evaluate in train mode and your predictions depend on whatever else happens to be in the batch; train in eval mode and BN never learns its statistics. Our `train_model` recipe flips the switch in both directions — look for it. Corollary: BN needs real batches to estimate statistics, so it gets unreliable at tiny batch sizes.

> ℹ Normalize across *what*? A seed for Part IV
>
> Batch statistics are one choice among several. When we build transformers (1), batches of variable-length sequences will make per-batch statistics awkward, and the same standardize-then-rescale idea will reappear computed per *token* instead: **layer normalization**. Same equation, different axis — remember Equation 9.1 when you meet it.

9.3. Building with blocks: a small VGG

Now assemble Question 1's answer with Question 4's habit. Define the *atom* — conv, BN, ReLU — once; a block stacks atoms and pools; a network stacks blocks. This is the lecture's point about VGG versus its hand-tuned predecessors: you stop re-designing and start repeating.

```
def atom(c_in: int, c_out: int):
    return [nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
            nn.BatchNorm2d(c_out), nn.ReLU()]

class VGGSmall(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            *atom(1, 16), *atom(16, 16), nn.MaxPool2d(2),    # -> (16, 14, 14)
            *atom(16, 32), *atom(32, 32), nn.MaxPool2d(2),   # -> (32, 7, 7)
        )
```

```
        self.head = nn.Sequential(nn.Flatten(),
                                  nn.Linear(32 * 7 * 7, 128), nn.ReLU(),
                                  nn.Linear(128, 10))
    def forward(self, x):
        return self.head(self.features(x))

vgg = train_model(VGGSmall, epochs=60)
n_head = count(vgg.head)
print(f"VGGSmall: {count(vgg):,} parameters ({n_head:,} in the head, "
      f"{n_head / count(vgg):.0%})")
print(f"test accuracy {accuracy(vgg, X_te, y_te):.1%}")
```

```
VGGSmall: 218,586 parameters (202,122 in the head, 92%)
test accuracy 86.7%
```

Two readings again. The good: 86.7%, four points above LeNet — depth with small kernels genuinely sees better, and BN let sixty epochs suffice for a deeper network. The bad: 218,586 parameters, *triple* LeNet's total, and the head's share got worse — 92% of the network is a dense layer reading a flattened grid. The convolutional diet succeeded and the total got fatter anyway. Which forces the lecture's third question: where do the parameters live, and — his phrasing — where can we do the most damage to the count?

9.4. Question 3: 1×1 convolutions, and firing the flatten head

The bloat has one address: `nn.Linear(1568, 128)`. To evict it we need one more tool, odd-looking at first sight: a convolution whose kernel is 1×1.

A 1×1 kernel does no spatial mixing at all — it looks at a single pixel. But across *channels* it does exactly what Equation 8.1 says: at each position, take the C_{in} channel values and form C_{out} weighted combinations plus bias. Pause on what that is: a **linear model across the channels, run separately at every pixel** — Chapter 1's machine, miniaturized and stamped across the image. Add a ReLU and each pixel is running a tiny MLP over its own feature vector. The lecture's summary: we *summarize* channels, we don't discard them — compressing 64 feature maps into 32 loses far less than pooling them away, and each compression injects another nonlinearity almost for free.

That tool enables the **Network-in-Network** design (Lin et al., 2013), and with it the clean head. A NiN block is one 3×3 (some spatial mixing) followed by two 1×1s (per-pixel channel mixing). And the classifier becomes:

1. Let the last block produce a stack of feature maps.
2. **Global average pooling (GAP)**: average each map over all positions — 64 maps in, 64 numbers out. No parameters at all.
3. One small linear layer to the logits.

```
def nin_block(c_in: int, c_out: int):
    return [nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
            nn.BatchNorm2d(c_out), nn.ReLU(),
            nn.Conv2d(c_out, c_out, 1), nn.ReLU(),
            nn.Conv2d(c_out, c_out, 1), nn.ReLU()]

class NINSmall(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            *nin_block(1, 16), nn.MaxPool2d(2),
            *nin_block(16, 32), nn.MaxPool2d(2),
            *nin_block(32, 64),
        )
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(64, 10))
    def forward(self, x):
        return self.head(self.features(x))

nin = train_model(NINSmall, epochs=150)
print(f"NINSmall: {count(nin):,} parameters "
      f"(head: {count(nin.head):,})")
print(f"test accuracy {accuracy(nin, X_te, y_te):.1%}")
```

```
NINSmall: 35,034 parameters (head: 650)
test accuracy 76.2%
```

The parameter story is a rout: 35,034 total, a 650-parameter head, six times smaller than VGGSmall. The accuracy is 76.2% — and that dip is worth more attention than the win, so let us not rush past it. First, though, collect the promised payoff. Chapter 8 said the remaining shift-cliff was the flatten head's fault: it reads the final grid *positionally*. GAP averages over positions — there is nothing positional left to disturb. The rematch of the rematch:

```
class LeNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, 5, padding=2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(400, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
    def forward(self, x):
        x = F.max_pool2d(F.relu(self.conv1(x)), 2)
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = x.flatten(1)
```

```
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)

lenet = train_model(LeNet, epochs=150)          # Chapter 8's model, same recipe

def shift_right(X, px: int):
    if px == 0:
        return X.clone()
    out = torch.zeros_like(X)
    out[..., px:] = X[..., :-px]
    return out

shifts = list(range(5))
acc_lenet = [accuracy(lenet, shift_right(X_te, s), y_te) for s in shifts]
acc_nin = [accuracy(nin, shift_right(X_te, s), y_te) for s in shifts]
for s in shifts:
    print(f"shift {s}px:   LeNet {acc_lenet[s]:.1%}   NIN+GAP {acc_nin[s]:.1%}")

plt.figure(figsize=(5.5, 3.2))
plt.plot(shifts, acc_lenet, "s-", color="#232D4B", lw=2, ms=7, label="LeNet (flatten head)")
plt.plot(shifts, acc_nin, "^-", color="#E57200", lw=2, ms=7, label="NINSmall (GAP head)")
plt.axhline(0.1, ls=":", color="#B8B8A8")
plt.xlabel("shift (pixels right)"); plt.ylabel("test accuracy")
plt.ylim(0, 0.9); plt.xticks(shifts); plt.legend()
plt.tight_layout(); plt.show()
```

```
shift 0px:   LeNet 82.5%   NIN+GAP 76.2%
shift 1px:   LeNet 74.8%   NIN+GAP 70.3%
shift 2px:   LeNet 62.3%   NIN+GAP 69.3%
shift 3px:   LeNet 45.3%   NIN+GAP 67.5%
shift 4px:   LeNet 26.8%   NIN+GAP 66.5%
```

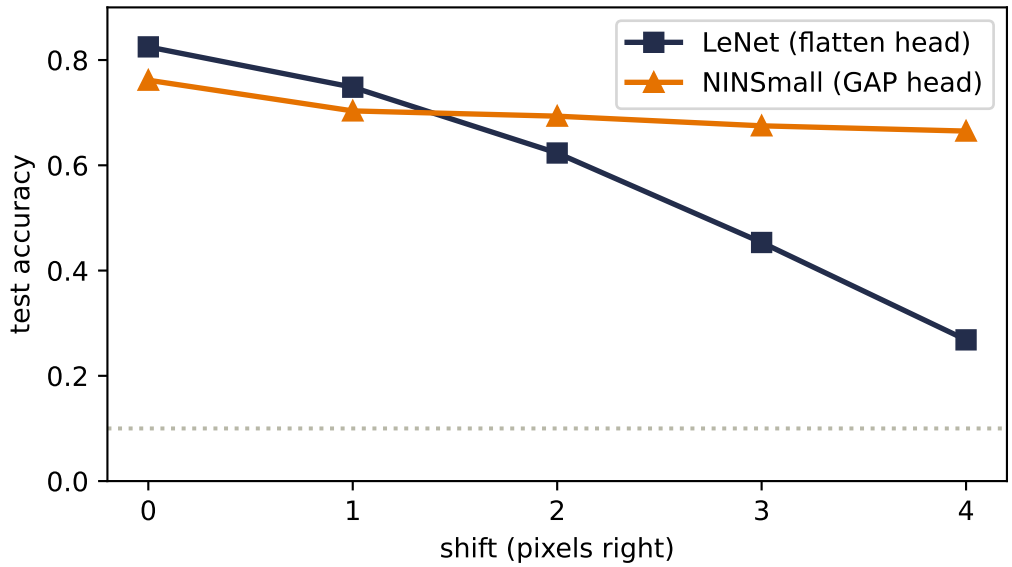

Figure 9.1.: Chapter 8's experiment, third round. LeNet's flatten head still slides below 30% by four pixels of shift. NINSmall with its global-average-pool head barely tilts: what the head reads — per-channel averages — is almost unchanged when features move. The remaining droop is content clipped at the image edge, not position sensitivity.

The curve that fell from 82% to 27% across this book's Part II is now a gentle slope from 76% to 67%. Position sensitivity, hunted since Chapter 6, is finally gone from the head.

Now the dip. On clean, centered test garments NINSmall trails LeNet by six points, and this is Chapter 8's warning callout collecting its debt with the sign flipped: **on centered data, position *is* information**, and GAP threw it away. A sleeve's location helps when every garment is framed identically; averaging it out costs accuracy exactly there. With the full 60,000-image dataset the gap closes — the lecture's NiN reaches about 90%, near its LeNet — because more data teaches richer channel features that compensate. At our scale you see the trade nakedly: invariance is bought with information, in both directions.

i Question 4's other famous block: Inception, in one breath

GoogLeNet's Inception block (2014) answers "which kernel size?" with "all of them": parallel 1×1, 3×3, 5×5, and pooling branches, concatenated. Its enabling trick is the 1×1 *bottleneck*: compress channels before the expensive spatial kernels. The seed's arithmetic for one 5×5 branch at 256 channels: direct, $5^2\times256\times128\approx819$k parameters; with a 1×1 squeeze to 32 first, $256\times32+5^2\times32\times128\approx111$k — an 86% cut for the same nominal operation. We won't build Inception here (the principle, channel compression before spatial expense, is the transferable part), but Exercise 2 walks the arithmetic and the course assignment has you build the block itself.

9.5. Question 2: going deeper, and the wall you hit

VGG says depth is the direction. So push it: stack twenty of our two-conv blocks (with BN, per the recipe, on a 14×14 grid so the experiment fits a laptop) and compare against the *identical* stack with one change we'll reveal after the numbers.

```
class Block(nn.Module):
    def __init__(self, ch: int, residual: bool):
        super().__init__()
        self.c1 = nn.Conv2d(ch, ch, 3, padding=1, bias=False)
        self.b1 = nn.BatchNorm2d(ch)
        self.c2 = nn.Conv2d(ch, ch, 3, padding=1, bias=False)
        self.b2 = nn.BatchNorm2d(ch)
        self.residual = residual
    def forward(self, x):
        y = self.b2(self.c2(F.relu(self.b1(self.c1(x)))))
        return F.relu(y + x) if self.residual else F.relu(y)

class DeepNet(nn.Module):
    def __init__(self, nblocks: int, residual: bool, ch: int = 12):
        super().__init__()
        self.stem = nn.Conv2d(1, ch, 3, padding=1)
        self.pool = nn.MaxPool2d(2)                          # 28 -> 14 early
        self.blocks = nn.Sequential(*[Block(ch, residual)
                                      for _ in range(nblocks)])
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(ch, 10))
    def forward(self, x):
        return self.head(self.blocks(self.pool(F.relu(self.stem(x)))))

plain = train_model(lambda: DeepNet(20, residual=False), epochs=30)
resid = train_model(lambda: DeepNet(20, residual=True), epochs=30)
for name, m in [("plain-20   ", plain), ("residual-20", resid)]:
    print(f"{name}  train {accuracy(m, X_tr, y_tr):.1%}   "
          f"test {accuracy(m, X_te, y_te):.1%}")
```

```
plain-20     train 60.8%   test 51.0%
residual-20  train 100.0%   test 77.8%
```

Look at the *training* column first, because it carries the whole lesson. The plain 40-conv-layer network cannot even fit the 1,200 images it stares at every epoch — 61% train accuracy — while its twin memorizes them completely and generalizes twenty-seven points better. This is the **degradation problem** exactly as He et al. reported it in 2015 (their 56-layer plain net trained *worse* than their 20-layer one): not overfitting, not vanishing capacity, but an *optimization* failure. The deep plain network is theoretically strictly more expressive — it could set twenty blocks to imitate the shallow net and coast — yet gradient descent cannot find even that.

The one change: each block computes $F(x)$ and outputs $F(x) + x$. A **residual connection** — the input skips over the block and is added back.

$$H(x) = F(x) + x \qquad \Longrightarrow \qquad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial H}\left(\frac{\partial F}{\partial x} + I\right). \tag{9.2}$$

Read the right-hand side with Chapter 5 eyes. Blame arriving at a block's output reaches its input through two routes: through the block's layers ($\partial F/\partial x$, which may mangle it), and through the identity I — untouched, unshrunk, unexploded, no matter what the block does. Chapter 5 called ReLU's open gate a *gradient superhighway* through a single unit; the skip connection is the same highway built as civic infrastructure, one lane per block, spanning the entire network. The block only needs to learn the *residual* — the correction on top of "pass it through" — and doing nothing is now the easiest thing in the world to learn: $F = 0$.

Here is the highway visible at initialization, extending Chapter 5's depth experiment from dense layers to conv stacks:

```
class PlainNoBN(nn.Module):
    def __init__(self, nconvs: int, ch: int = 12):
        super().__init__()
        self.stem = nn.Conv2d(1, ch, 3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.layers = nn.Sequential(*[m for _ in range(nconvs)
                                      for m in (nn.Conv2d(ch, ch, 3, padding=1),
                                                nn.ReLU())])
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(ch, 10))
    def forward(self, x):
        return self.head(self.layers(self.pool(F.relu(self.stem(x)))))

def stem_grad(make_net):
    torch.manual_seed(0)
    net = make_net()
    F.cross_entropy(net(X_tr[:128]), y_tr[:128]).backward()
    return net.stem.weight.grad.norm().item()

depths = [4, 12, 24]                                  # blocks (2 convs each)
curves = {
    "plain, no BN": [stem_grad(lambda d=d: PlainNoBN(2 * d)) for d in depths],
    "plain + BN": [stem_grad(lambda d=d: DeepNet(d, False)) for d in depths],
    "residual + BN": [stem_grad(lambda d=d: DeepNet(d, True)) for d in depths],
}
for name, ys in curves.items():
    print(f"{name:14s}: " + "   ".join(f"{v:.1e}" for v in ys))

plt.figure(figsize=(6.2, 3.4))
for (name, ys), color in zip(curves.items(),
```

```
                                ["#B8B8A8", "#E57200", "#232D4B"]):
    xs = [2 * d for d in depths]
    shown = [y if y > 0 else 1e-24 for y in ys]       # exact 0 -> sentinel to plot it
    plt.semilogy(xs, shown, "o-", ms=5, color=color, label=name)
    for x, y in zip(xs, ys):
        if y == 0:
            plt.annotate("exact 0 (underflow)", (x, 1e-24), fontsize=8,
                         color=color, textcoords="offset points", xytext=(-88, 4))
plt.xlabel("conv layers"); plt.ylabel(r"$\|\partial L / \partial W^{(\mathrm{stem})}\|$")
plt.legend(); plt.tight_layout(); plt.show()
```

```
plain, no BN  : 9.0e-06   3.3e-13   0.0e+00
plain + BN    : 7.6e-02   5.0e-01   8.3e+01
residual + BN : 1.6e-01   4.9e-01   5.0e-01
```

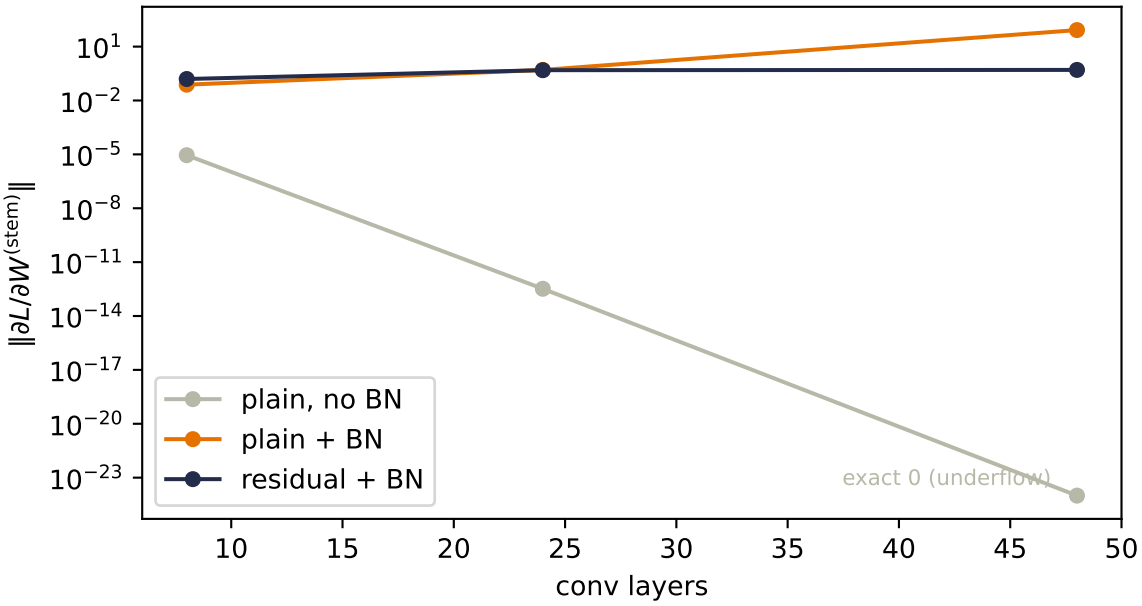

Figure 9.2.: Gradient magnitude reaching the stem (first layer) at initialization, versus depth, for three designs. Without BN the gradient underflows to *exactly zero* by 48 layers (annotated point): Chapter 5's float-precision death, now caused by nothing but depth. BN alone fixes the vanishing but overshoots — in plain stacks the gradient *grows* with depth, an instability of its own. Residual blocks hold the gradient within one order of magnitude at every depth: the highway delivers.

The most important seed this chapter plants

Residual connections are not a CNN trick. They are *the* enabling device of every large model you have heard of: when we assemble a transformer in 1, every attention layer and every

feed-forward layer will be wrapped as $x + F(x)$, and the freshly-met layer normalization will sit beside each skip. The picture to carry forward: a *residual stream* flowing unimpeded through the network, with each block reading from it and writing small corrections back. Attention will be one kind of correction. You now own both parts of that skeleton.

9.6. The scorecard: what a decade of design bought

The lecture closes its architecture tour with a scatter worth reproducing — every model of Part II on one canvas, accuracy against parameters, each trained to convergence under our shared recipe:

```
models_ = {
    "LeNet": (lenet, "#B8B8A8"),
    "VGGSmall": (vgg, "#E57200"),
    "NIN+GAP": (nin, "#232D4B"),
    "plain-20": (plain, "#DDA0DD"),
    "residual-20": (resid, "#5379AA"),
}
plt.figure(figsize=(6.2, 3.6))
for name, (m, color) in models_.items():
    p, a = count(m), accuracy(m, X_te, y_te)
    plt.scatter(p, a, s=70, color=color, zorder=3)
    plt.annotate(name, (p, a), textcoords="offset points",
                 xytext=(8, -3), fontsize=9)
plt.xscale("log")
plt.xlabel("parameters (log scale)"); plt.ylabel("test accuracy")
plt.ylim(0.45, 0.95)
plt.tight_layout(); plt.show()
```

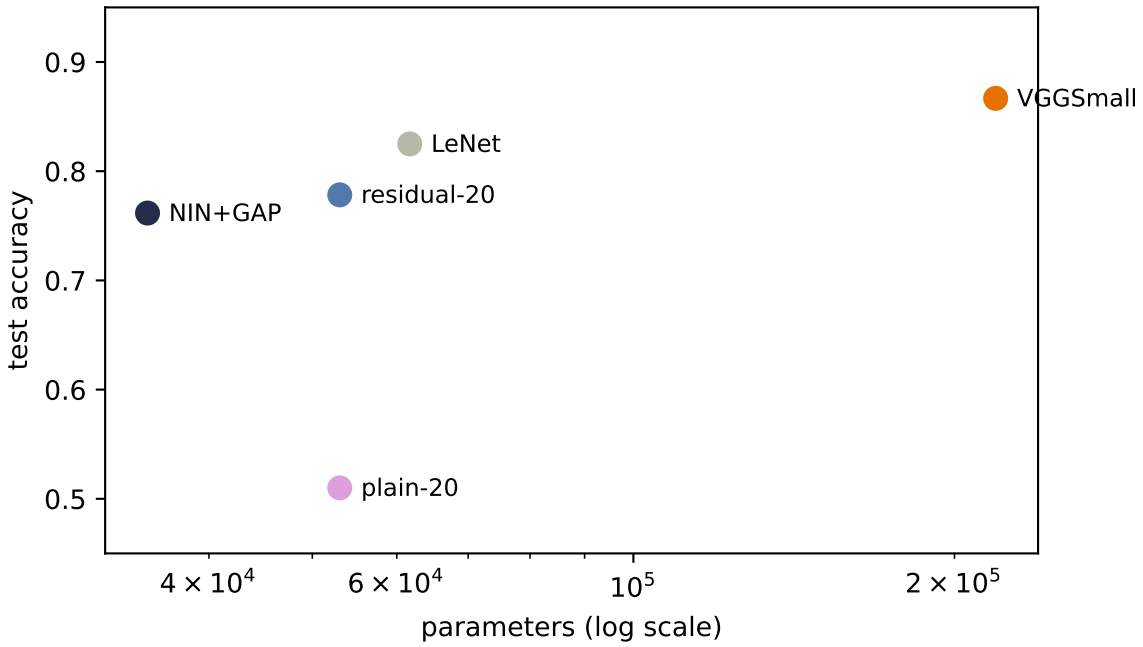

Figure 9.3.: Part II's models on one canvas (our 1,200-image subset; each model trained to convergence with the shared Adam recipe). VGG-style depth buys accuracy but hoards parameters in its flatten head; NiN+GAP evicts the head at a visible cost in clean accuracy at this data scale; residual blocks let a 47k-parameter network go 40 layers deep. The frontier the lecture reports on the full dataset — everything above 89% — needs data our subset does not have; the *shape* of the trade-offs is already exactly right.

9.7. Transfer learning: the mechanics, and an honest experiment

Everything so far trains from scratch. The lecture's final move is the one that defines practice today: most image features are *shared* — edges, textures, parts transfer across tasks — so train one strong backbone once, on enormous data, and reuse it everywhere. The mechanics come in two strengths:

- **Linear probe**: freeze the pretrained trunk entirely (`requires_grad = False`), extract its features, train only a new linear head on your labels.
- **Fine-tuning**: additionally unfreeze the last block or two, training them at a *much* smaller learning rate than the fresh head, so the pretrained features adapt gently instead of being trampled.

We will test the idea properly. The task: classify the three shoe classes (sandal, sneaker, ankle boot) from **ten labeled examples each**. The 30-image regime is exactly where transfer should shine — too few labels to learn features, goes the story, so imported features should dominate. Three contenders:

1. **From scratch**: our VGG-style trunk trained on the 30 images alone.

2. **Our own pretrained trunk**: the same trunk pretrained on the seven *non-shoe* classes (863 images), frozen, linear probe.
3. **A real pretrained backbone**: SqueezeNet 1.1 trained on ImageNet — 1.2 million photographs, 1,000 classes — loaded from weights committed with this book, frozen, linear probe on its 512-dimensional features. (Fashion images get upsampled to 96×96 and repeated to three channels to fit its expectations. That awkwardness is part of the experiment, and part of the lesson.)

```
from torchvision.models import squeezenet1_1

shoe = torch.tensor([5, 7, 9])
new_label = {5: 0, 7: 1, 9: 2}
is_shoe_tr = (y_tr[:, None] == shoe).any(1)
is_shoe_te = (y_te[:, None] == shoe).any(1)
X_shoe_te = X_te[is_shoe_te]
y_shoe_te = torch.tensor([new_label[int(c)] for c in y_te[is_shoe_te]])

# our own trunk: pretrain on the 7 non-shoe classes
src = [0, 1, 2, 3, 4, 6, 8]
remap = {c: i for i, c in enumerate(src)}
X_src = X_tr[~is_shoe_tr]
y_src = torch.tensor([remap[int(c)] for c in y_tr[~is_shoe_tr]])

class Trunk(nn.Module):
    def __init__(self, n_classes: int):
        super().__init__()
        self.features = nn.Sequential(
            *atom(1, 16), *atom(16, 16), nn.MaxPool2d(2),
            *atom(16, 32), *atom(32, 32), nn.MaxPool2d(2), *atom(32, 64))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(64, n_classes))
    def forward(self, x):
        return self.head(self.features(x))

own_trunk = train_model(lambda: Trunk(7), epochs=100, data=(X_src, y_src))
X_src_te = X_te[~is_shoe_te]
y_src_te = torch.tensor([remap[int(c)] for c in y_te[~is_shoe_te]])
print(f"own trunk, source task (7 classes): "
      f"{accuracy(own_trunk, X_src_te, y_src_te):.1%}")

# the real thing: ImageNet SqueezeNet, from committed weights (no download)
sq = squeezenet1_1(weights=None)
sq.load_state_dict(torch.load("../../data/squeezenet1_1-imagenet.pt"))
sq.eval()
for p in sq.parameters():
    p.requires_grad = False
```

```
IMNET_MEAN = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
IMNET_STD = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)

@torch.no_grad()
def squeeze_features(X):                        # (N,1,28,28) -> (N,512)
    x = X.repeat(1, 3, 1, 1)
    x = F.interpolate(x, size=96, mode="bilinear", align_corners=False)
    f = sq.features((x - IMNET_MEAN) / IMNET_STD)
    return F.adaptive_avg_pool2d(f, 1).flatten(1)

@torch.no_grad()
def own_features(X):                            # (N,1,28,28) -> (N,64)
    return F.adaptive_avg_pool2d(own_trunk.features(X), 1).flatten(1)

def linear_probe(Z_tr, y_f, Z_te, seed):
    torch.manual_seed(seed)
    head = nn.Linear(Z_tr.shape[1], 3)
    opt = torch.optim.Adam(head.parameters(), lr=1e-2,
                           weight_decay=1e-3)   # built-in L2 penalty (ch. 1's ridge)
    for _ in range(400):
        loss = F.cross_entropy(head(Z_tr), y_f)
        opt.zero_grad(); loss.backward(); opt.step()
    return (head(Z_te).argmax(1) == y_shoe_te).float().mean().item()
```

```
own trunk, source task (7 classes): 79.7%
```

```
K = 10
rows = []
for seed in [0, 1, 6050]:
    torch.manual_seed(seed)
    idx = torch.cat([(y_tr == c).nonzero().squeeze(1)
                     [torch.randperm(int((y_tr == c).sum()))[:K]] for c in shoe])
    X_few = X_tr[idx]
    y_few = torch.tensor([new_label[int(c)] for c in y_tr[idx]])

    scratch = train_model(lambda: Trunk(3), epochs=100, seed=seed,
                          data=(X_few, y_few))
    rows.append((accuracy(scratch, X_shoe_te, y_shoe_te),
                 linear_probe(own_features(X_few), y_few,
                              own_features(X_shoe_te), seed),
                 linear_probe(squeeze_features(X_few), y_few,
                              squeeze_features(X_shoe_te), seed)))

print("seed     scratch   own-trunk probe   ImageNet probe")
```

```
for seed, r in zip([0, 1, 6050], rows):
    print(f"{seed:4d}     {r[0]:.1%}       {r[1]:.1%}           {r[2]:.1%}")
mean = [sum(c) / 3 for c in zip(*rows)]
print(f"mean     {mean[0]:.1%}       {mean[1]:.1%}           {mean[2]:.1%}")
```

```
seed     scratch   own-trunk probe   ImageNet probe
   0     86.4%       68.9%           88.1%
   1     85.3%       65.5%           85.9%
6050     88.1%       77.4%           87.6%
mean     86.6%       70.6%           87.2%
```

Read that table slowly, because it does *not* say what the textbook story predicts, and the discrepancy is the best lesson in this chapter. Training from scratch on thirty images fights the mighty ImageNet backbone to a dead heat — the means differ by less than a point, well inside seed noise. And our own pretrained trunk, perfectly competent on its source task per the printout above, transfers *worse than nothing*. Three reasons, each a general principle:

1. **A trunk can only donate what its data taught it.** Our own trunk saw 863 shirts, bags, and trousers. Nothing in that curriculum required foot-shaped detectors, so its 64 features flatten sandals, sneakers, and boots into nearly the same point. Pretraining is not magic; it is *curriculum*.
2. **Domain and resolution gaps tax the donation.** SqueezeNet's filters expect 224-pixel color photographs; we feed it 28-pixel grayscale icons inflated to 96. Its early layers hunt for detail that simply is not there. (The lecture's recipe — resize and normalize *to match the pretraining pipeline* — is doing real work; we complied as far as the data allows.)
3. **Transfer wins when the target is big relative to your labels — not small.** This is the quiet one. Three shoe silhouettes at 28×28 is a *small* problem: thirty images genuinely suffice, so the scratch baseline is strong and there is little room for imported knowledge to pay rent. The lecture's numbers show the other regime: linear-probing an ImageNet ResNet-18 on the *full* FashionMNIST task reaches about 93%, and fine-tuning about 94% — at 224 pixels, with a task rich enough that features matter more than labels.

> **When to reach for a pretrained backbone**
>
> The honest decision rule our experiment and the lecture's jointly support: transfer pays when (your labels are scarce) *and* (the target task is feature-hungry) *and* (the pretraining data plausibly covers the target's features at a matched scale). At 224 pixels and 18 landmark classes — the course assignment — all three hold, and transfer is the winning move by a wide margin. At 28 pixels and 3 silhouettes, the conditions fail and scratch fights the superpower to a draw. Knowing *which regime you are in* is the skill; the mechanics (`requires_grad`, learning-rate splits) are the easy part.

The mechanics, for completeness, since you will use them constantly from Part V onward — fine-tuning SqueezeNet's last block with the lecture's two-learning-rate recipe (fresh head fast, pretrained layers slow):

```
def prep(X):
    x = X.repeat(1, 3, 1, 1)
    x = F.interpolate(x, size=96, mode="bilinear", align_corners=False)
    return (x - IMNET_MEAN) / IMNET_STD

torch.manual_seed(6050)
idx = torch.cat([(y_tr == c).nonzero().squeeze(1)
                 [torch.randperm(int((y_tr == c).sum()))[:K]] for c in shoe])
X_few, y_few = prep(X_tr[idx]), torch.tensor([new_label[int(c)]
                                              for c in y_tr[idx]])

ft = squeezenet1_1(weights=None)
ft.load_state_dict(torch.load("../../data/squeezenet1_1-imagenet.pt"))
head = nn.Linear(512, 3)
for p in ft.parameters():
    p.requires_grad = False
for p in ft.features[-1].parameters():         # last Fire block only
    p.requires_grad = True

opt = torch.optim.Adam([
    {"params": head.parameters(), "lr": 1e-3},          # fresh head: normal speed
    {"params": ft.features[-1].parameters(), "lr": 1e-4},  # pretrained: gentle
], weight_decay=1e-3)

ft.train()
for _ in range(60):
    z = F.adaptive_avg_pool2d(ft.features(X_few), 1).flatten(1)
    loss = F.cross_entropy(head(z), y_few)
    opt.zero_grad(); loss.backward(); opt.step()
ft.eval()
with torch.no_grad():
    z = F.adaptive_avg_pool2d(ft.features(prep(X_shoe_te)), 1).flatten(1)
    print(f"fine-tuned (last block + head): "
          f"{(head(z).argmax(1) == y_shoe_te).float().mean():.1%}")
```

```
fine-tuned (last block + head): 88.7%
```

Fine-tuning edges the frozen probe on this seed and lands in the same tie with scratch — consistent with the regime analysis above: adaptation helps at the margin, but no amount of it makes a small target task need imported features. Keep the pattern, though: *this* loop, scaled to real backbones and feature-hungry tasks, is how most of applied deep learning ships today. It is also this book's destination: Part V is what happened when the field noticed that "pretrain big, adapt cheaply" was not a vision trick but *the* recipe — for language too. The BERT moment (Chapter 15) is this section's idea, taken seriously at scale.

9.8. Okay, so —

1. **Small kernels, stacked**: two 3×3s see what a 5×5 sees, for $18C^2$ against $25C^2$, with twice the nonlinearity — provided width holds constant through the stack. Design in blocks, repeat the block.
2. **BatchNorm** re-standardizes every channel over the minibatch and hands back two knobs (γ, β); conv→BN→ReLU with `bias=False`; *train and eval are different machines* — flip the switch. Its per-token cousin, LayerNorm, awaits in

    1.

3. 1×1 **convolutions** run Chapter 1's linear model across channels at every pixel: summarize channels, don't discard them. With **global average pooling** they fire the flatten head — parameters collapse (218k → 35k) and position sensitivity leaves the network for good, at a clean-accuracy price our small dataset makes visible.
4. **Depth hits an optimization wall, not a capacity wall**: our plain 40-layer net couldn't fit its own training set. The residual connection $H(x) = F(x) + x$ adds an identity lane to the Jacobian — Chapter 5's gradient superhighway as infrastructure — and the same-depth twin trains to 100%. Transformers will be built from nothing but such blocks.
5. **Transfer learning** = freeze + probe, or gently fine-tune. Our honest experiment found scratch unbeatable at 28-pixel scale — pretraining is curriculum, gaps are taxed, and small targets don't need imports — while the lecture's full-scale runs (93–94% via ImageNet ResNet-18) show the regime where transfer rules. Diagnosing the regime is the skill.

Exercises

1. **(Pencil.)** Generalize the kernel-stacking arithmetic: n stacked 3×3 layers versus one $(2n{+}1) \times (2n{+}1)$ kernel, at constant width C. Then redo it for a layer that grows channels $C \to 2C$: split into two 3×3s ($C \to C \to 2C$ and $C \to 2C \to 2C$) and find when the split stops saving parameters.
2. **(Pencil.)** Verify the Inception bottleneck arithmetic from the callout (819,200 versus 110,592), then design the cheapest 1×1-plus- 5×5 pipeline from 128 channels to 64 output channels that keeps the squeeze width at least a quarter of the input width.
3. **(Code.)** Make VGGSmall's blocks *depthwise separable*: replace each 3×3 conv with a per-channel 3×3 (`groups=c_in`) followed by a 1×1 mixer. Count parameters, retrain, and report the accuracy-per-parameter ratio against the original. (This is MobileNet's trick for phones — the lecture's fifth principle.)
4. **(Code.)** Ablate BatchNorm: retrain VGGSmall with the BN layers removed, same budget. Compare final accuracy *and* the first ten epochs' training accuracy. Then repeat the `bn-drift` cell's measurement on both trained networks — does training itself fix the activation scales?
5. **(Code.)** Data augmentation as a third road: on the 30-image shoe task, train from scratch with random horizontal flips and ±3-pixel shifts (Chapter 6's exercise, now as a tool). Does augmentation close the gap to the lecture's full-data numbers further than transfer did? Why might augmentation and transfer help in *different* regimes?

Part III.

III · Sequences

10. Sequences and Recurrence

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Order matters. Parameter sharing across time gives the RNN; products of Jacobians give vanishing gradients; gates (LSTM/GRU) give memory a highway.

11. Encoder–Decoder, Teacher Forcing, Beam Search

> **Warning**
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Compress a sentence into a vector, then unroll it in another language. Teacher forcing, scheduled sampling, beam search — and the fixed bottleneck that sets up everything in Part IV.

Part IV.

IV · Attention: Learning the Similarity

12. Kernel Regression: Attention Before It Was Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

A century-old idea: predict by averaging nearby evidence, weighted by similarity. Nadaraya–Watson in NumPy, bandwidths, and the observation that this is already attention — with a fixed similarity function. Open with the two Chapter 1 callbacks (prediction as a weighted combination of training targets via the projection view, and the dot product as similarity score) plus the Chapter 2 callback (softmax normalizes scores into weights).

13. Attention: Making the Kernel Learnable

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Replace the fixed kernel with learned projections: queries, keys, values. Additive vs scaled dot-product (and why GPUs prefer the latter), the $\sqrt{d}$ scaling, masking, multi-head. The seq2seq bottleneck falls. Harvest two Chapter 2 seeds by name: softmax as the scores→weights machine, and the "softening the hard" motif — attention IS the differentiable lookup promised there (the 8.1 seed's own "soft way to access memory" language).

14. Self-Attention and the Transformer

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Let a sequence attend to itself and recurrence becomes optional. Positional encodings as rotations, multi-head subspaces, LayerNorm and residuals, the FFN as associative memory — the full block, built and trained.

15. The BERT Moment: Pretraining as the New Regime

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Stop training from scratch. Masked language modeling, bidirectional context, fine-tuning — and why 2018 changed the default workflow of the field.

16. Vision Transformers and Scaling Laws

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Patches as tokens: the transformer eats images, needs more data (inductive bias strikes again — this time as a trade), and obeys power laws. Chinchilla's 20-tokens-per-parameter arithmetic.

Part V.

V · The Pretrained Era

17. Adapting Pretrained Models: Prompting, PEFT, Quantization

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

When the model is too big to fine-tune, get surgical: prompting and in-context learning, LoRA's low-rank updates, quantization and QLoRA.

18. Alignment and RL Fine-Tuning

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

From next-token prediction to helpfulness: instruction tuning, preference learning, and RLHF in outline.

19. Generative Models: From PCA to Diffusion

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

One more make-it-learnable arc: PCA → autoencoder → VAE → diffusion (and GANs as the adversarial detour). Latents, ELBO, noise schedules.

A. Linear Algebra and the SVD

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

The geometry the book leans on: matrices as transformations, eigenvectors, and the SVD as the master decomposition.

B. Tensors in Practice

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Shapes, broadcasting, indexing, and the layer-algebra habits that prevent 90% of PyTorch bugs.

C. Notation

> ⚠ Warning
>
> **Chapter stub.** This chapter is planned but not yet written. The abstract below is its contract; drafting sources are listed for provenance.

Symbols used throughout the book.

D. Floating Point and Machine Precision

> ⚠ Warning
>
> **Appendix stub.** Planned (author request, July 2026). See `docs/backlog.md` §1.

Every number in this book is a lie of finite precision. What a float actually is (sign, exponent, mantissa), machine epsilon, why $0.1 + 0.2 \neq 0.3$*, catastrophic cancellation, and the patterns deep learning uses to survive it: log-sum-exp (the trick inside every softmax), solving instead of inverting, and mixed-precision training (fp16/bf16) — the bridge to quantization in Chapter 17.*